

# Tabular In-Context Learning: Priors, Architecture, and Training

A research proposal for standard benchmarks and rare-event prediction

Research assessment prepared for Niraj Kumar

3 October 2026

## Abstract

We propose a training distribution and an in-context model for binary classification, multiclass classification and regression. The aim is to improve on current tabular foundation models on standard benchmarks, then adapt the pretrained model to large financial tables with missing values and fraud rates near  $10^{-4}$ . The model learns from synthetic prediction tasks during pretraining. At deployment, its weights remain fixed and labeled rows from the new table provide the context for prediction.

The starting architecture is a dense model with 317M parameters. It encodes individual cells, summarizes each row and processes the row representations with a 24-layer transformer. The initial prior mixture assigns 50/20/15/10/5 percent of tasks to reference graphs, direct forests, categorical hierarchies, smooth or local functions, and sparse interactions. Missingness and coarsening are sampled separately from the target-generating mechanism. We also implement a 41M model that retains feature-level representations through its attention stages. A controlled comparison will determine which prior and architecture to use for the main training run.

The report explains the evidence behind these choices and gives the generator laws, model dimensions, losses, optimizer settings, curriculum, evaluation procedure and code needed to reproduce them. The full study has a budget of 50,000 GPU-hours. Local tests establish implementation correctness for the cases tested; benchmark performance, accelerator throughput and the best training scale remain to be measured.

|                                                                                                                                                                                                                                                                                                                                                                                    |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Starting configuration.</b> The current configuration trains the 317M compressed-row model on the five-family prior Q. The 41M persistent-cell model is a separate comparison. Figure 1 shows both architectures. The main pretraining run follows the prior and architecture study; its provisional targets are 64M standard tasks and 1.024M fraud-continuation tasks.</p> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

# Contents

|          |                                                                           |           |
|----------|---------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>The prediction problem and the proposed model</b>                      | <b>4</b>  |
| 1.1      | Starting architecture . . . . .                                           | 4         |
| 1.2      | Pretraining data and task sequence . . . . .                              | 4         |
| 1.3      | How to read the report . . . . .                                          | 5         |
| <b>2</b> | <b>How a prior teaches in-context prediction</b>                          | <b>7</b>  |
| 2.1      | Prior, world, table and episode . . . . .                                 | 7         |
| 2.2      | A Bayesian example . . . . .                                              | 8         |
| 2.3      | Amortized Bayesian prediction . . . . .                                   | 8         |
| 2.4      | Observation and selection mechanisms . . . . .                            | 9         |
| 2.5      | The role of each implemented starting generator . . . . .                 | 10        |
| <b>3</b> | <b>Previous work: priors, model scale and architecture</b>                | <b>10</b> |
| 3.1      | Synthetic task distributions and their limits . . . . .                   | 11        |
| 3.2      | Implications for experimental design . . . . .                            | 12        |
| 3.3      | Checkpoint size and disclosed training scale . . . . .                    | 12        |
| 3.4      | Availability of pretraining code . . . . .                                | 13        |
| 3.5      | Representation and attention designs . . . . .                            | 14        |
| 3.6      | Sparse experts and the Xiaomi comparison . . . . .                        | 15        |
| <b>4</b> | <b>Research hypotheses and supporting analysis</b>                        | <b>15</b> |
| 4.1      | Context size and learnable mechanism complexity . . . . .                 | 15        |
| 4.2      | Conditional targets as a lower-variance training estimator . . . . .      | 16        |
| 4.3      | Numerical check of the gradient identity . . . . .                        | 17        |
| 4.4      | Adaptive sampling as a controlled hypothesis . . . . .                    | 17        |
| 4.5      | Preserving information and choosing auxiliary objectives . . . . .        | 18        |
| <b>5</b> | <b>Selecting a stronger standard prior</b>                                | <b>18</b> |
| 5.1      | What the evidence supports . . . . .                                      | 18        |
| 5.2      | A specified starting mixture . . . . .                                    | 19        |
| 5.3      | Generating the proposed mechanisms . . . . .                              | 20        |
| 5.4      | Observation law and information boundaries . . . . .                      | 23        |
| 5.5      | What can be established without pretraining . . . . .                     | 23        |
| 5.6      | Choosing the mixture within the compute budget . . . . .                  | 25        |
| <b>6</b> | <b>Prior bank for rare-event financial tables</b>                         | <b>27</b> |
| 6.1      | Finite positive evidence . . . . .                                        | 27        |
| 6.2      | Classification and regression targets . . . . .                           | 28        |
| 6.3      | Covariate-first finite-population prior . . . . .                         | 28        |
| 6.4      | Shared population rules and the earlier class-conditional model . . . . . | 30        |
| 6.5      | Shortcut and coverage audit of the finance worlds . . . . .               | 33        |
| <b>7</b> | <b>The model architecture</b>                                             | <b>33</b> |
| 7.1      | Architecture overview . . . . .                                           | 34        |
| 7.2      | Three choices in the input, output and attention layers . . . . .         | 34        |
| 7.3      | Preprocessing and cell embeddings . . . . .                               | 37        |
| 7.4      | Feature attention, row compression and support attention . . . . .        | 38        |
| 7.5      | Prediction heads and parameter counts . . . . .                           | 39        |
| 7.6      | Compact persistent-cell comparator . . . . .                              | 41        |
| 7.7      | Finance inputs and the optional realized-loss head . . . . .              | 42        |
| <b>8</b> | <b>Context selection and caching</b>                                      | <b>44</b> |
| 8.1      | Dense context cost and exact support caches . . . . .                     | 44        |
| 8.2      | Bounded finance contexts and shared routing . . . . .                     | 45        |
| 8.3      | Full-history preparation and burst inference . . . . .                    | 46        |
| 8.4      | The information limit of fixed-size compression . . . . .                 | 47        |
| <b>9</b> | <b>End-to-end training protocol</b>                                       | <b>47</b> |
| 9.1      | Standard pretraining and fraud specialization . . . . .                   | 47        |

|           |                                                                  |           |
|-----------|------------------------------------------------------------------|-----------|
| 9.2       | Typed data boundaries and deterministic replay . . . . .         | 49        |
| 9.3       | Sampling, validity and retry semantics . . . . .                 | 49        |
| 9.4       | Joint task mixture and finance-world reuse . . . . .             | 50        |
| 9.5       | Likelihoods, preprocessing and logical-batch reduction . . . . . | 51        |
| 9.6       | Population-risk estimation for rare-event queries . . . . .      | 51        |
| 9.7       | Gradient scale and fraud specialization . . . . .                | 53        |
| 9.8       | Muon and AdamW: partition, numerics and schedule . . . . .       | 53        |
| 9.9       | Shape and class curriculum . . . . .                             | 54        |
| <b>10</b> | <b>Model size and allocation of 50,000 GPU-hours</b>             | <b>55</b> |
| 10.1      | Model-size recommendation . . . . .                              | 55        |
| 10.2      | Training volume and computational units . . . . .                | 56        |
| 10.3      | Planned volume and reference-pilot exposure . . . . .            | 56        |
| 10.4      | Compute model and profiling protocol . . . . .                   | 58        |
| 10.5      | Model size and training-volume experiments . . . . .             | 59        |
| 10.6      | Allocation of the project budget . . . . .                       | 60        |
| 10.7      | Compute accounting tool . . . . .                                | 61        |
| <b>11</b> | <b>Evaluation and model selection</b>                            | <b>61</b> |
| 11.1      | Generator and model acceptance checks . . . . .                  | 61        |
| 11.2      | Standard benchmarks and inference tracks . . . . .               | 62        |
| 11.3      | Finance evaluation against full-history trees . . . . .          | 62        |
| 11.4      | GPU batch-latency measurements . . . . .                         | 64        |
| 11.5      | Statistical criteria and controlled comparisons . . . . .        | 64        |
| 11.6      | Joint-model selection within the budget . . . . .                | 65        |
| <b>12</b> | <b>Implementation and validation</b>                             | <b>66</b> |
| 12.1      | Map from the recipe to the implementation . . . . .              | 66        |
| 12.2      | Running the candidate and selection experiments . . . . .        | 68        |
| 12.3      | From local checks to a cluster pilot . . . . .                   | 69        |
| 12.4      | Numerical and statistical checks . . . . .                       | 70        |
| 12.5      | Design implications of the implementation audit . . . . .        | 70        |
| 12.6      | Validation and limitations . . . . .                             | 70        |
| <b>A</b>  | <b>Reference generator and earlier prior designs</b>             | <b>72</b> |
| A.1       | Reference prior and earlier extensions . . . . .                 | 72        |
| A.2       | Generating an episode in the earlier R/P1 pilot . . . . .        | 73        |
| A.3       | P0/P1/P4 replacement control: mixture and variants . . . . .     | 76        |
| A.4       | Sampling conventions and base task shapes . . . . .              | 76        |
| A.5       | Shared features and the observation process . . . . .            | 77        |
| A.6       | Response distributions and task construction . . . . .           | 78        |
| A.7       | P0: five conditional and joint mechanisms . . . . .              | 79        |
| A.8       | P1: hierarchical categorical effects . . . . .                   | 81        |
| A.9       | P4: count and compound-amount outcomes . . . . .                 | 82        |
| A.10      | Generation order, shifts and validity checks . . . . .           | 82        |
| A.11      | Context-conditioned complexity . . . . .                         | 83        |
| A.12      | Conditional-law outputs and information boundaries . . . . .     | 84        |
| <b>B</b>  | <b>Deferred architecture and training experiments</b>            | <b>85</b> |
| B.1       | Optional feature feedback . . . . .                              | 85        |
| B.2       | Optional MoE: matched active work and extra capacity . . . . .   | 85        |
| B.3       | Deferred local/global attention and cache experiments . . . . .  | 87        |
| B.4       | Optional consistency and masked-conditional objectives . . . . . | 90        |
| B.5       | Optional adaptive prior sampling . . . . .                       | 91        |
| <b>C</b>  | <b>Earlier screening allocation and controls</b>                 | <b>91</b> |

# 1 The prediction problem and the proposed model

The goal is to pretrain a model that can solve a new tabular prediction problem from labeled examples, without updating its weights on that table. We call the labeled examples the *support set* and the rows to be predicted the *query set*. The model must use the support set to infer which features matter, how they relate to the target and how much uncertainty remains. We evaluate binary classification, multiclass classification and regression separately, even when one checkpoint serves all three.

Two applications guide the design. Standard benchmarks test transfer across tables with different feature types and target relationships. Financial prediction adds large histories, rare fraud, missing values, delayed labels and bursts of arriving records. A history may contain millions of rows while offering only a few useful positive examples. The model therefore needs both a suitable training prior and a way to select informative context from a much larger table.

## 1.1 Starting architecture

**The starting configuration is the 317M compressed-row model**, named `base` in the code. Each cell is first embedded in 128 dimensions. Two stages of column and within-row attention produce eight learned summary vectors per row. Concatenating those vectors gives a 1,024-dimensional row representation, which enters a 24-layer row transformer. A classification head produces class probabilities; a regression head predicts 999 quantiles. Figure 1 gives the complete outline, and Section 7 specifies every layer.

The model uses categorical hash bits, explicit missing-value indicators, logarithmic attention scaling and two shared key/value heads on the query path. Support processing in the row transformer still uses all 16 attention heads. With the finance adapter and optional realized-loss head included, it has exactly **317,116,304 parameters**. The realized-loss head contains a zero atom and a 1,025-bin positive density; it is reserved for the separate finance-regression experiment and is not trained by the standard and binary-fraud runs.

The alternative, `persistent_small`, is a persistent-cell model with **41,089,248 parameters**. It keeps feature cells through twelve attention stages and discards them only at the final readout. It has no separate row transformer. This is a competing architecture, trained as a separate model. It is not an additional branch inside the 317M model, and the two are not combined into an ensemble by default. The planned comparison asks whether retaining feature detail can give better predictions at the available training and inference cost. The existing 94M model and released compact architectures provide further points of comparison during scaling.

No architecture has yet won that comparison. The 317M model is the runnable starting choice; the full pretraining allocation goes to the model and prior supported by the experiments in Sections 5 and 10. Mixture-of-experts layers, feature feedback and the general local/global attention proposal are deferred studies in Appendix B. They are not part of either configuration in Figure 1.

## 1.2 Pretraining data and task sequence

The current standard-training configuration samples from prior Q: reference graphs, direct forests, categorical hierarchies, smooth/local functions and sparse interactions with probabilities .50/.20/.15/.10/.05. An independent observation process introduces missingness or coarsening. Classification and regression each receive half the task probability. The prior study varies these choices while keeping the model, optimizer and inference procedure fixed, then compares the two architectures under the selected prior.

The earlier R/P1 pilot remains a control. It adds P1 with probability .05 when the requested width is at least eight and otherwise uses the released TabICLv2 graph generator R. R keeps its native filters; P1 uses numerical validity checks without a teacher filter. Both Q and this earlier pilot use wider feature and class limits for 20% of requests in stages two and three. Appendix A retains that recipe and the older P0/P1/P4 generators in full. These earlier mixtures should not be confused with Q.

After standard pretraining, we retain the general model and initialize a second checkpoint for binary fraud detection. This continuation uses finite simulated populations, delayed labels and the same context-selection procedure intended for deployment. It starts a fresh optimizer and learning-rate schedule. The standard checkpoint therefore remains available for general classification and regression. Final training reserves 9,000 GPU-hours for the standard model and 3,000 for the fraud model. An 80% standard/20% finance mixture is retained as a separate parameter-sharing comparison; finance regression also requires its own evaluation.

Muon updates the hidden matrices, while AdamW updates the remaining parameters. The starting configuration uses zero weight decay, following the released TabICLv2 training script; the original P0/P1/P4 optimizer control uses .01. Auxiliary, contrastive and adaptive-mixture losses are disabled. One *episode*, also called a macroepisode in the code, is a supervised prediction task from one simulated data-generating process. A finance episode can route its queries to several selected contexts, but it still contributes one task to the logical training batch.

Use `configs/selection_candidate_standard.json` and `configs/selection_candidate_fraud.json` for the provisional full-volume configurations, and `docs/prior_selection.md` for the experiment and launch instructions. The older `configs/candidate_standard.json` and `configs/candidate_fraud.json` files are bounded reference pilots, documented in `docs/main_recipe.md`. Section 10.3 explains how measured throughput determines the feasible number of tasks.

### 1.3 How to read the report

We first explain how a synthetic prior teaches prediction from context, then review the relevant models and the evidence for extending their priors. The main methods sections describe Q, the finance prior, the two implemented architectures, context selection and training. The remaining sections cover scaling, evaluation and the code. Earlier generators and deferred experiments are collected in the appendices so that their details remain available without interrupting the current training plan.

The mathematical examples explain particular properties of a generator, estimator or representation. Predictive claims require trained checkpoints evaluated on the same splits and under stated inference budgets. The study therefore compares components on a fixed model, confirms promising combinations with fresh seeds and dataset lineages, and then tests model size and training duration. This is necessary because a gain can disappear at larger scale or when combined with another change. Context retrieval and permutation ensembles are reported as explicit inference settings; per-dataset gradient fine-tuning is a separate adaptation baseline.



Two architectures to be trained separately at the same allocated budget.  
 In both, support states depend only on support; queries never update them.

Figure 1: Two implemented architectures for the same prediction task. The standard training configuration uses the 317M model on the left. The 41M model on the right is the compact alternative in the architecture study. Each column-attention block is an induced set-attention block (ISAB): it forms summaries from support and lets feature cells attend to those summaries. Each within-row block mixes the cells and eight learned summary tokens of one record. The left model compresses before its row transformer; the right model keeps cells until the output. The arrows describe computation within each model, not an ensemble connecting them. Both support binary and multiclass classification with up to 256 classes, and regression.

## 2 How a prior teaches in-context prediction

Prior-based pretraining begins by choosing a distribution of prediction problems. Each draw specifies how columns relate to one another, how the target is generated, how noise enters and which measurements are visible. From this simulated data-generating process, we construct a table and ask the network to predict withheld outcomes using the labeled rows. Across many such tasks, the network learns how the evidence in a new table should change its predictions.

The target might depend almost linearly on one feature, change abruptly at a threshold or include a shared effect for records with the same product code. Labeled support rows provide evidence for distinguishing these possibilities. The prior determines how frequently each arises, including cases in which several explanations agree with the available observations. Choosing a prior therefore determines the kinds of inference the network can learn, as well as the amount of synthetic data available for training.

### 2.1 Prior, world, table and episode

| Object                  | Meaning in this project                                                                                                                                                          |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Prior family            | A distribution over possible rules and data sets, such as R, F or H. A family is a generator, not one stored table.                                                              |
| World $\phi$            | One sampled data-generating process, specified by hidden rules such as graph/functions, category effects, noise parameters, or a finite finance population.                      |
| Table                   | Records produced under a world. Their features and outcomes obey the same sampled mechanism, subject to its declared observation law.                                            |
| Episode                 | A prediction exercise: labeled support, query features, and separate query targets used to score the prediction. A finance macroepisode may use several routed support contexts. |
| Neural weights $\theta$ | The shared predictor learned across many episodes. These are different from the hidden parameters $\phi$ of a particular synthetic world.                                        |

Table 1: Generating 64 million tasks counts accepted episodes, not 64 million prior families, independent rows or parameters.

We call one sampled data-generating process a *world*, denoted by  $\phi$ . Its parameters are drawn before its records and shared across them. For example, in  $y = ax + \epsilon$ , the coefficient  $a$  remains fixed while each row receives new  $x$  values and noise. A new process can have a different  $a$ . If every row instead had an unrelated slope, its label would generally tell us little about the query. P1 uses the same principle: each entity’s latent effect is shared across its support and query occurrences. The finance generator reuses one finite population for four macroepisodes, so these are four uses of the same sampled process, not four independent populations.

Records generated by the same process need not be independent. They may be related through a finite population, temporal effects or sampling without replacement. The native R generator also transforms complete tables and rejects some proposed tables. These operations affect the prediction problem presented during training and are part of the prior specification.

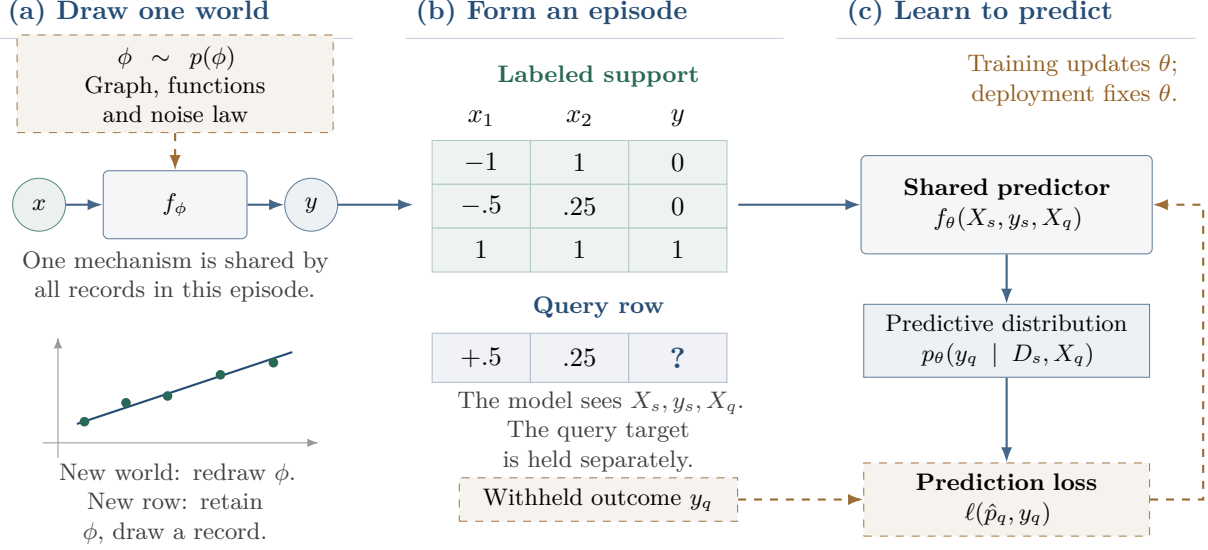


Figure 2: Synthetic pretraining episode. A sampled mechanism generates both support and query records. The predictor receives support labels and query features; withheld query outcomes supply the loss. Outcomes may also enter the declared acceptance or query-sampling procedure. Pretraining updates  $\theta$ , whereas deployment follows the prediction path with fixed weights. The miniature table and line are illustrative, not outputs of a particular generator.

## 2.2 A Bayesian example

A coin-toss example illustrates the Bayesian interpretation. Each simulated process has an unknown positive-outcome probability  $p$ , sampled from  $p \sim \text{Beta}(1, 1)$ , the uniform distribution on  $[0, 1]$ . Outcomes are independent conditional on this shared  $p$ . If the support contains eight positives and two negatives, Bayes' rule gives

$$p \mid D_s \sim \text{Beta}(9, 3), \quad P(Y_{\text{next}} = 1 \mid D_s) = \mathbb{E}[p \mid D_s] = \frac{9}{12} = 0.75.$$

The observed positive fraction is 0.8, but the predictive probability is 0.75. The prior contributes one count to each outcome, drawing the estimate toward 0.5 when observations are scarce. As support grows, these two prior counts have less influence. Changing the prior therefore has its largest effect when little evidence is available.

Training a network on many such problems can teach it to map support observations to predictions without explicitly programming a Beta-distribution update. Tabular pretraining extends this idea to feature-dependent relationships, interactions, categories and observation patterns; the transformer need not implement the analytic calculation above. The sampling process remains essential to the interpretation. Eight positives among ten selectively reviewed transactions do not form a representative binomial sample, so inference must account for how those transactions were chosen.

## 2.3 Amortized Bayesian prediction

Let  $I$  contain everything available to the predictor: labeled support, the features of one query, permitted class information and other permitted metadata. The target distribution  $q$  describes how the data-generating process and support are sampled, and which query population the loss should represent. For standard tasks, it includes table acceptance and query-row sampling. In finance, the target is a

record drawn uniformly from the finite future population, conditional on the sampled process and its history. Balanced training proposals estimate this objective through importance weighting. Without those weights, they would train against a different target distribution.

With classification log loss, the objective for an unrestricted predictor is

$$\mathcal{L}(\theta) = \mathbb{E}_{(I,Y) \sim q}[-\log f_{\theta}(Y | I)].$$

For any fixed  $I$ , the expected log loss equals the entropy of  $q(Y | I)$  plus the KL divergence from  $q(Y | I)$  to the predicted distribution. The minimum is therefore attained at  $f^*(Y | I) = q(Y | I)$ . By Bayes' rule, this prediction averages over the possible hidden data-generating processes:

$$q(y | I) = \int q(y | I, \phi) q(\phi | I) d\phi.$$

The posterior  $q(\phi | I)$  gives greater weight to processes that better explain the evidence. Query features belong in  $I$  because they too can help identify the process that generated the table. When several mechanisms remain plausible, the prediction averages their implications for the query outcome. This per-query objective requires predictive marginals; it does not require the output head to model the joint dependence among all query outcomes.

This reusable approximation is called *amortized inference*. Sampling a table is often easy, but integrating over all graphs, functions and latent variables consistent with it can be expensive. Pretraining learns a prediction procedure that can be applied to many new tables. At deployment, a forward pass approximates the integration: support observations change the activations and predictions while the network weights remain fixed. The Bayesian interpretation thus concerns the process that generated the task, rather than uncertainty over neural weights, as in TabPFN and related prior-fitted models [1, 8].

Finite model capacity, imperfect optimization and differences between synthetic and real tasks all limit this approximation. The training loss also determines the quantity being approximated. Pinball loss estimates conditional quantiles, while the optional hurdle likelihood models a point mass at zero and a density over positive outcomes. Averaging 999 quantiles approximates the predictive mean, but can miss the contribution of outcomes rarer than the finite quantile grid can resolve.

## 2.4 Observation and selection mechanisms

A real table contains measurements, not the simulator's complete state. Values can be rounded or missing, and a category may first appear in the query set. Prediction must account for the uncertainty left by these observations. The generator therefore samples the latent process first and then determines what the learner observes. Missingness and the conversion of latent values into recorded features are parts of the statistical problem.

Rejecting proposed episodes also changes the training distribution. Let  $c$  specify the task type, table shape and source selected before generation. Write  $q_0(e | c)$  for the distribution of proposed episodes after any split repair, and  $a(e, c)$  for the probability of accepting episode  $e$ . Among episodes generated under these fixed settings, the accepted distribution is

$$q_{\text{accepted}}(e | c) = \frac{a(e, c)q_0(e | c)}{\int a(e', c)q_0(e' | c) de'}.$$

Retries keep  $c$  fixed, so they do not change its probability in the outer sampling process. Settings with low acceptance rates consequently require more generation time without losing their assigned share of training tasks. In R, graph checks, split repair and the predictability filter all affect which proposed

tables reach the learner. Generated outcomes may be used in these construction steps while remaining excluded from predictor inputs. P1 keeps arithmetically valid tasks without applying the native teacher filter.

Finance introduces a further issue: how historical evidence is selected. Including more known fraud cases can make a limited context more informative, but the event fraction in that context no longer estimates the population rate. The generator must preserve the relationship between the selected support, any valid historical reference and the naturally rare future outcomes. Population-weighted query losses then target the future population even when rare training queries are oversampled. These observation and selection rules matter as much as the response equation.

## 2.5 The role of each implemented starting generator

| Source  | What varies across worlds                                                                                       | What the learner practices                                                                             |
|---------|-----------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| R       | Vector-valued graphs, nonlinear mechanisms, feature/target positions, noise and native transformations          | Inferring broad numerical relationships, interactions and dependence from a labeled table.             |
| P1      | Entity frequencies, shared group effects, entity-specific effects and their relationship to observable metadata | Combining repeated-category evidence with group/feature information when a category is rare or unseen. |
| Finance | Finite risk regions, temporal populations, event counts, missingness, delayed labels and selected contexts      | Predicting rare future events from the evidence actually available at a cutoff.                        |

Table 2: The earlier executable pilot uses R/P1 for the standard checkpoint and the finance law for a separate binary specialist. P0 and P4 remain historical comparison controls, with no additional sample quotas in the main plan.

The earlier reference pilot spans three related inference problems. R provides broad combinations of numerical relationships. P1 introduces repeated category identities and partial pooling, with controlled frequencies of category observations. The finance generator combines rare outcomes with the process by which historical evidence becomes available. Learning from these tasks requires the representation and context-selection method to preserve the relevant information, which is why prior and architecture changes must be evaluated together.

## 3 Previous work: priors, model scale and architecture

We reviewed nineteen retained papers together with official repositories and checkpoint metadata. The local PDFs and their searchable text are stored under `papers/`; page numbers in this section refer to those copies. Source fingerprints and page-level notes are available in `papers/source_manifest.json` and `research/reviews`.

Recent models often change their synthetic generator, representation and training scale at the same time. Better benchmark performance establishes the value of the resulting system, but cannot by itself identify which change produced the gain. We therefore distinguish the methods each paper discloses from its system-level results, and use that distinction to design comparisons that control the relevant factors.

### 3.1 Synthetic task distributions and their limits

| Work         | Prior/model evidence                                                                                                                             | Consequence                                                                                                  |
|--------------|--------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| TabPFN v1/v2 | Simplicity-biased BNN/SCM priors; v2 expands tasks, types and missingness alongside architecture changes [1, 2].                                 | Bayesian task pretraining and mixed causal structures are established ingredients.                           |
| TabPFN-2.5   | Improved priors/training and separate real-data continuation variants [3].                                                                       | Report whether training used only synthetic data or also real-data continuation.                             |
| TabPFN-3     | Graph/function composition, categoricals, dynamic/spatial/high-frequency and extrapolation priors; Sec. 2.5, pp. 10–12 [4].                      | A new function or shift family needs evidence of a useful difference.                                        |
| TabPFN-3.5   | High-cardinality categories and wide tables; Fourier/rank features and shared classification/regression training, pp. 8–11 [5].                  | Encoding, capacity and prior changes need separate controls.                                                 |
| TabICL/v2    | Open row-compressed ICL; v2 has eight mechanism classes, structured hyperparameter sampling and ExtraTrees filtering, pp. 5–7 and App. E [6, 8]. | Use the released generator as the reference for new prior comparisons.                                       |
| TabFM        | September 400M-scale release combines spectral feature encoding, two column/row stages and deep ICL; Table 1 and Sec. 3 [9].                     | A larger row trunk is credible, but the earlier 1.64B release is a different artifact.                       |
| Xiaomi       | Compositional SCMs plus feature grouping, attention changes, sparse experts and staged training [10].                                            | Prior gains cannot be inferred from full-system gains.                                                       |
| EXAONE       | Small architecture with feature/item interaction and synthetic SCM training; architecture/pretraining sections [11].                             | A more expressive interaction path may outperform indiscriminate parameter growth.                           |
| Kumo Tabular | Mixed SCMs, missingness, coarsening, conflicting labels, high cardinality and tails, pp. 5–6 [12].                                               | These data properties are already covered by competing priors. Its 35/71/137M numbers count training tables. |

Evidence from prior ablations also cautions against adding mechanisms indiscriminately. Mitra-v2 combines base and hybrid SCMs with five tree generators, yet its negative-results appendix reports limited benefit from changing the outer mixture weights and worse performance for some correlation-matched continuation settings [13]. In a smaller nanoPFN experiment, O’Prior reports TabArena AUC of 0.8194 for the full G4 composition and 0.8335 for hybrid G1b in Table 2 [14]. These findings apply to the particular models and training conditions studied. They nevertheless give a clear reason to test proposed additions individually before combining them.

Other studies examine why a prior transfers to real data. “Mind the Gap?” and controlled prior comparisons find that measured similarity between real and synthetic data explains only part of downstream performance [15, 16]. Measures of structural coverage provide another diagnostic, but omit some relevant relationships [17]. These results motivate examining what a learner can infer from a finite context. A list of function families, marginal histograms or generated-table counts cannot fully describe that inference problem.

### 3.2 Implications for experimental design

Our internal review considered priors, architectures, training and evaluation separately across the nineteen papers, then checked the proposal against the executable generators and predictor. The resulting design decisions are supported by source-level findings in `research/reviews/independent_audit.md`.

TabICLv2 provides direct evidence that the value of a prior depends on the architecture. Section 7 and Appendix C show that its new prior only matches the older system when used with the older architecture; conversely, performance degrades when the newer architecture uses the older prior [8]. A prior that performs well on a small proxy model therefore need not be the strongest choice for a different backbone.

The planned comparison crosses two priors with two architectures: one compresses rows, and the other retains feature-cell states. Within each comparison intended to identify the effect of a change, the heads, optimizer, table-shape schedule and total cost are held fixed. The prior comparison also keeps the current shared checkpoint and output-head conventions. Separately trained classification and regression models remain an available control, but a dedicated study of weight sharing is deferred under the revised budget.

Many mechanisms considered here already appear in competitive systems. Kumo includes coarsened measurements, conflicting duplicates, structured missingness, high-cardinality categories and heavy-tailed regression [12]. TabPFN-3/3.5 expands function composition, distribution shifts, high-dimensional structure and categorical coverage [4, 5]. Yet more mechanisms do not consistently produce better results: O’Prior’s hybrid-only generator outperforms its full composition on TabArena AUC/F1 in the reported nano-scale setting, and Mitra-v2 finds relatively small effects from mixture adjustments within its complete training and adaptation system [14, 13]. We therefore begin with a strong reference, add mechanisms in controlled comparisons and judge proposed realism by its effect on transfer.

The reference follows the released implementation, including known differences from the intended method. The official TabICL repository states that cautious weight decay was not connected to the released Muon runs and leaves it disabled in the v2 scripts [7]. Appendix E also describes discrepancies in correlated categorical sampling and feature warping. We retain the unchanged source revision as the reference and treat any repair as a separate experiment.

Identifying the exact released artifact is equally important for other models. Some width descriptions in TabFM’s model card disagree with the inspected weight metadata; tensor shapes and checkpoint configuration therefore determine the artifact used in our comparison. RTFM reports 90,000 datasets together with a batch size of 64, 3,000 steps and 30 iterations [27]. Taken literally, the latter quantities imply 5.76 million nominal tasks, so the headline dataset count does not support an unambiguous compute estimate.

### 3.3 Checkpoint size and disclosed training scale

| Model/version | Size per checkpoint                  | Qualification/source                                                     |
|---------------|--------------------------------------|--------------------------------------------------------------------------|
| TabPFN-3.5    | 220M shared; Fast 84M                | Supplied p. 4; both classification and regression in the main model [5]. |
| TabICLv2      | 27.55M classifier / 28.54M regressor | Official weight metadata; distinct checkpoints [18].                     |
| Kumo Tabular  | Approximately 28M / 62M / 215M sizes | Small/medium/large; separate task files, not 35/71/137M parameters [19]. |

| Model/version | Size per checkpoint                   | Qualification/source                                                                           |
|---------------|---------------------------------------|------------------------------------------------------------------------------------------------|
| Xiaomi-TabLDM | 70.08M / 71.08M total                 | Active weights about 61.67M / 62.68M; sparse expert design, supplied App. A [10].              |
| EXAONE        | 20.81M / 21.11M                       | Small separate classifier/regressor, supplied Table 2 [11].                                    |
| TabFM 1.1.0   | 410.22M / 412.29M                     | Current public metadata consistent with September paper’s roughly 400M label [20, 9].          |
| TabFM 1.0.0   | 1.639B / 1.648B                       | Older public artifacts; explains billion-scale references in earlier comparisons [21].         |
| LimiX-2       | Roughly 400M; measured grid to 406.2M | Capacity scaling through 406.2M is measured; 2B extrapolation is not [22].                     |
| Mitra-v2      | 75.7M / 76.7M                         | Default inference includes task fine-tuning/bagging; separate from frozen-ICL comparison [13]. |
| TabDPT v1.3   | 63.04M stored elements                | Current official artifact differs from original 78M paper model [23, 24].                      |

The checkpoint sizes above count stored tensor elements. This can include frozen positional parameters. Exact trainable-parameter counts also require checking how modules are registered and whether parameters are aliased, although small positional tensors do not materially change these rounded totals. The audit in `research/reviews/model_scale_audit.md` records release revisions and weight headers; it did not execute the models. Storage and computation for ensembles, frozen text encoders and task-specific fine-tuned copies must be counted separately.

TabICLv2 reports 35.2M nominal episodes per task model. Its staged schedule implies approximately 48.15B expected row presentations and about 588 H100 GPU-hours per model [8]. A row presentation counts a row each time it is processed; it is neither a raw-cell count nor a count of independent data-generating processes. H100 hours also cannot be converted directly into MI355X hours. TabPFN-3 reports more than 8T “tokens”, but the disclosed tokenizer and training stages are insufficient for a reliable comparison; v3.5 does not provide an equivalent disclosed total [4, 5]. The compute analysis leaves these quantities unknown.

### 3.4 Availability of pretraining code

Released weights and inference code do not necessarily make pretraining reproducible. We inspected the complete public repository trees for PriorLabs, NVIDIA Kumo, Xiaomi and AutoGluon’s Mitra, and traced Mitra-v2 to its separate Hugging Face source. Revision identifiers and file inventories are recorded in `research/results/upstream_repository_audit.json`; function-level findings and permanent links are in `research/reviews/upstream_code_audit.md`. The following table separates released generators and pretraining code from inference and downstream fine-tuning code.

| Release       | Finding at the inspected public revision                                                                                                                                                                                                 |
|---------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PriorLabs 3.5 | Architecture, preprocessing, caching and downstream fine-tuning are exposed. No complete current synthetic generator and pretraining entry point was found. The actual ICL code uses full support heads and a query-only KV prefix [53]. |
| Kumo Tabular  | <code>recipe.py</code> is a processing/ensemble pipeline, not the synthetic prior. The release article explicitly says training recipe and generators will follow. The inspected medium/large models use two query KV heads [12, 54].    |
| Xiaomi        | <code>learning.py</code> implements contextual prediction. <code>moe.py</code> provides expert routing and initialization components. The inspected tree does not include a complete generator, optimizer and launch procedure [55].     |
| Mitra         | AutoGluon contains a pretrain configuration schema, but its available trainer and neighboring YAML concern fine-tuning. The v2 package supplies adaptation and results; Appendix A explicitly excludes synthetic pretraining [13, 56].   |
| TabICLv2      | The actual prior, training scripts and trainer are available. The retained source is pinned at <code>0dbff3ec8fc68c123c87af77b0ea8b25cd2d23f3</code> ; hashes are verified before execution [7].                                         |

Three recent studies inform narrower parts of the design. A controlled sparse-prior study finds that alternating-axis attention outperforms row compression on sparse linear tasks, with a much smaller difference on dense tasks [57]. This supports comparing architecture families under more than one prior. A quantization study substantially reduces compressed weight size while leaving inference memory and speed largely unchanged [58]; smaller weight files alone therefore do not solve context-latency limitations. A less-constrained latent-prior synthesis method learns a generator for an existing table [59]. That objective differs from learning a distribution of supervised ICL tasks, so its results do not establish a reason to replace this bank.

### 3.5 Representation and attention designs

TabICLv2 separates table representation from contextual inference in a compact architecture. Appendix A, pp. 18–19, describes three induced column-attention blocks at width 128 with 128 inducing points, three row blocks with four summary tokens, and twelve ICL blocks at width 512. Classification and regression use separate models. Its optimizer experiment favors Muon over AdamW, but Appendix C finds no reliable log-loss improvement from simply increasing the depth of the column, row and ICL stacks. These results support studying capacity together with the generator.

TabPFN-3.5 combines standardized values, learned Fourier features and empirical ranks fitted on support. It shares a backbone across classification and regression, normalizes queries and keys, and uses eight summary tokens to construct a width-1,024 row representation (supplied paper §3.1, pp. 8–9). Its grouped attention on the query path is also a candidate for a controlled efficiency comparison. TabFM uses a related larger design: Table 1, p. 4, and the mask description on p. 5 specify two column/row stages, eight summary tokens and 24 ICL blocks at width 1,024, totaling 408.7M parameters. These systems provide precedents for the proposed components; experiments must determine which combination suits the present prior and compute budget.

LimiX-2 takes a different approach by retaining feature-cell states through 24 dual-axis blocks. It has separate feature and target pathways, with four target slots at cell width 256. Its asymmetric attention

and shallow reconstruction loss illustrate how a target stream can operate differently from the feature stream. This design motivates both a comparison with permanent row compression and the optional feedback experiment in Appendix B.1. The architecture alone does not establish an advantage; that requires an empirical comparison. The design is described in the [LimiX-2 report](#), §2, pp. 4–7.

### 3.6 Sparse experts and the Xiaomi comparison

Xiaomi’s architecture illustrates why sparse experts should be evaluated in terms of active computation as well as stored parameters. Appendix A, Tables 6–10, describes 24 ICL blocks at width 512. In the final eight blocks, each width-1,024 two-layer FFN is replaced by *two routed experts, top-1 selection, and one always-on shared expert*; every expert has the width of the dense FFN. Each input executes both a routed expert and the shared expert. Across the trunk, this amounts to 32 FFN units of computation instead of 24 in the corresponding dense design, before routing overhead. The classifier has 70.08M total parameters and 61.67M active parameters, so the increase in stored capacity over active computation is modest [10].

Xiaomi reports a regression prediction time of 3.12 seconds per 1,000 rows, compared with 9.67 for TabFM. These measurements cover complete systems, including their architectures, preprocessing and inference settings; they do not isolate the effect of sparse experts. The benchmark’s dataset “training time” likewise measures task-side fitting or setup, rather than foundation-model pretraining. These results motivate measuring expert routing directly, without attributing the entire latency difference to it. The proposed advantage is **greater conditional capacity at similar active FFN arithmetic**; runtime measurements will determine whether it is useful in practice.

## 4 Research hypotheses and supporting analysis

For a prior to teach useful prediction, the support must reveal something about the sampled mechanism, and the model must preserve that information. We examine four aspects of this problem: mechanism complexity relative to support size, shared effects among categories, conditional-label supervision and information lost during row compression. The main configuration focuses on extending the reference prior and comparing representations. Additional complexity controls and training objectives are separate experiments. The analysis below explains the reasoning behind these choices before the implementation details.

### 4.1 Context size and learnable mechanism complexity

A complicated mechanism may provide little useful evidence if support examples rarely cover the regions relevant to a query. Consider a known tree with  $L$  equally likely leaves. Each leaf receives an independent fair binary label that is then fixed for all records in that leaf, and the learner is told each record’s leaf identity. With  $n$  support examples, the probability that a query falls in a leaf never observed in support is

$$P_{\text{empty}} = (1 - 1/L)^n. \quad (1)$$

For an unobserved leaf, the posterior probability of either class remains one-half. The expected Bayes error is therefore  $P_{\text{empty}}/2$ . At  $n = 128$ , using  $L = 32$  leaves gives  $P_{\text{empty}} = 0.01718$  and a maximum expected accuracy of 0.99141. At  $L = 1024$ , these values become  $P_{\text{empty}} = 0.88244$  and 0.55878. Requiring the learner to infer the tree structure can only add difficulty. The calculation changes if leaf labels are correlated, leaf frequencies are unequal, boundaries have shared structure or the context is larger. Tree depth alone consequently says little about how much evidence is available.

Correctly specified Gaussian linear regression provides another example of how model complexity interacts with support size. Assume no intercept, isotropic  $d$ -dimensional inputs and independent noise with variance  $\sigma^2$ . When  $n > d + 1$ , ordinary least squares has prediction risk

$$\mathbb{E}[(Y_q - X_q^\top \hat{w})^2] = \sigma^2 \left(1 + \frac{d}{n - d - 1}\right). \quad (2)$$

With  $n = 64$  and unit noise, the risks are 1.068, 1.340, 2.032, 4.200, 21.000 for  $d = 4, 16, 32, 48, 60$ . These are OLS risks under the stated assumptions, not Bayes risks for a sparse prior or forecasts of neural-model error. They illustrate how independently sampling table width, active dimension and context size can produce prediction problems with very different amounts of usable evidence.

The `context_v1` experiment in Appendix A tests this issue by making active dimension and nominal tree depth depend on support size, while retaining a branch that samples uncapped problems. Its control samples complexity independently of support size. For asymmetric or correlated trees, we measure how calibration records occupy the leaves;  $2^{\text{depth}}$  does not count the leaves that provide useful evidence. Results on wide development tables will show whether the cap removes poorly identifiable problems or also removes useful high-dimensional structure.

We propose measuring effective rank, conditional entropy, category frequencies and the improvement that a reference learner can achieve from the available context. Similarity between marginal distributions is only one useful descriptor. Structural-coverage work reports associations with performance while leaving some local and higher-order relationships unmeasured [17]. A task-centric study reports transfer from small real corpora and relates performance to learned retrieval [25]. Both motivate measurements of the prediction task itself. Neither identifies a universally best synthetic distribution or establishes that fitting benchmark densities should be the training objective.

## 4.2 Conditional targets as a lower-variance training estimator

Some synthetic generators can return a conditional distribution of outcomes as well as a sampled label. Using that distribution in the loss can reduce supervision noise while keeping hidden parameters out of the predictor. Let  $H = (X_s, Y_s, x_q)$  be the learner’s information, and let  $Z$  contain the parameters of the simulated process and any further state required to specify a valid conditional label distribution. The network receives only  $H$ . When the observation process hides values,  $Z$  also retains the query coordinates before rendering and their random effects. Conditional resampling fixes these quantities and  $H$ , then draws fresh outcome noise. The resulting distribution may therefore depend on hidden information; it need not be the outcome distribution conditioned on visible features alone.

For classification, suppose the generator can compute  $q_Z(c | H)$  exactly. Then the expected supervised loss is

$$\mathbb{E}[-\log p_\theta(Y | H)] = \mathbb{E}_{H,Z} \left[ -\sum_c q_Z(c | H) \log p_\theta(c | H) \right]. \quad (3)$$

The conditional label distribution is used only as a *loss target*. Averaging over data-generating processes drawn from the specified distribution accounts for posterior uncertainty without giving  $Z$  to the network. Under the usual conditions for interchanging differentiation and expectation, and with finite second moments, the conditional gradient  $\bar{g} = \mathbb{E}[g | H, Z]$  obeys

$$\mathbb{E}[\bar{g}] = \mathbb{E}[g], \quad \text{Cov}(g) - \text{Cov}(\bar{g}) = \mathbb{E}[\text{Cov}(g | H, Z)] \succeq 0. \quad (4)$$

This standard conditional-expectation identity shows that the estimator preserves the mean gradient while removing a positive-semidefinite component of its covariance. Synthetic-data distillation has used the same principle for time-series models [26]. Here, the open questions are whether a tabular ICL

generator supplies a valid conditional distribution and whether computing it is worthwhile. Gains reported for time series do not establish the size of any gain in this setting.

Forward logistic and multinomial generators provide exact row probabilities after intercept calibration and label-noise mixing. In the deferred initial ablation, soft cross-entropy would replace half the eligible classification losses, while realized query labels would be retained for independent accuracy evaluation. The full proposed comparison uses 0/50/100% replacement at equal total generation and training cost. It integrates out only *query label noise*. Support labels remain sampled observations, because replacing them with probabilities would change the information available for contextual prediction.

For regression, the analogous target is the conditional expectation of the entire loss. If that expectation cannot be computed directly, the generator can draw four independent labels for the same process, features and realized support, then average their losses. This estimator remains unbiased and, under independence, reduces conditional sampling variance by four. Applying MSE only to the conditional mean does not train calibrated uncertainty; similarly, bin probabilities omit learned density terms within the tails. Where necessary, the proposed comparison therefore averages samples of the full loss, keeps the head fixed and counts the additional generation work in the budget.

A forward simulator is not sufficient to justify conditional-label supervision; its conditional distribution must be correct for the information being held fixed. In an SCM where the target causes some features, conditioning on those descendants generally changes the target distribution from its structural equation to  $p(Y \mid X_s, Y_s, x_q, Z)$ . Selection that depends on labels can also change it. These cases retain `label_law_valid=false` and ordinary hard targets unless a correct conditional sampler is implemented. A privileged teacher used to select task difficulty answers a different question: low teacher error does not establish that a learner with finite support has access to the same prediction.

### 4.3 Numerical check of the gradient identity

A numerical check in `research/probes/algorithmic_probes.py` uses seed 20261003 to simulate 30,000 logistic processes, each with a random positive or negative coefficient, 32 sampled support labels and eight queries. A fixed two-parameter predictor uses statistics of the support and query features; the generating probabilities appear only in the loss. Across 32 resamplings of the query labels, the measured mean-square gradient component removed with label noise was 0.02187749, compared with the analytic value 0.02183528 and Monte Carlo standard error 0.00002784. The check passed its six-standard-error criterion. Across simulated processes, the soft-gradient variance trace remained 0.00134728, reflecting variation in contexts and generating parameters.

This toy experiment agrees with the conditional variance identity. An improvement in transformer training would also depend on the cost of obtaining conditional targets and the gradient variation that remains across contexts and generating processes. The proposed comparison at equal total cost is designed to test that practical benefit.

### 4.4 Adaptive sampling as a controlled hypothesis

The reference samples from a fixed bank. A deferred adaptive sampler would increase the frequency of mechanisms for which a fixed set of teachers identifies a reproducible prediction gap, while continuing to sample broadly from the bank. High loss alone is insufficient: irreducible noise, hidden information or a weak teacher can all cause it. Appendix B.5 describes a policy based on independent diagnostic simulations. This policy is outside the current executable baseline.

Under this policy, calibration simulations are kept separate from those used for scoring. The label or loss of an emitted query does not determine whether its task enters the nominal prior. Any change in

sampling weights is compared with the unchanged mixture on development tasks from disjoint source lineages.

## 4.5 Preserving information and choosing auxiliary objectives

Row compression makes wide contextual transformers affordable by summarizing the feature cells early. Some interactions or rare thresholds, however, may become relevant only after the model has considered the support labels. The funded persistent-cell comparison tests whether retaining feature-level states improves prediction within the available budget. Appendix B.1 describes a smaller, deferred feedback alternative that does not update feature-cell states at every trunk layer. Either approach must justify its additional computation through downstream performance.

All auxiliary-loss coefficients are zero in the main configuration. A deferred consistency objective would compare predictions after changes that preserve the task, such as permuting feature order or renaming nominal categories. Deleting features, applying arbitrary monotone transformations or adding observations can change the available information or useful distance structure, so we do not assume invariance to those operations. Nor do rows from different simulated processes automatically provide valid contrastive negatives. Appendix B.4 specifies the limited consistency and masked-conditional experiments.

## 5 Selecting a stronger standard prior

The earlier R/P1 configuration provides a reproducible historical baseline, but the available evidence does not establish it as the best prior for the final model. We therefore define a broader candidate bank and a controlled experiment to determine which additions help. Existing generators, configurations and volume calculations remain available as controls. The candidate sampler is implemented in `challenger_prior.py`; its response and observation distributions are implemented in `selection_laws.py`. It uses the same trainer as the reference. The experiment planner, phase materializer and paired real-data evaluator supply the remaining interfaces needed to run the comparison.

### 5.1 What the evidence supports

We extend R to test whether additional, explicitly controlled prediction problems complement its graph-based mechanisms. In Mitra’s prior ablation, SCM plus ExtraTrees scores 1063 Elo compared with 1000 for SCM alone. Adding gradient boosting raises the score to 1076, while the complete bank scores 1062 [30]. These results justify testing a direct-tree prior, but do not establish that it will improve R, which already includes tree mechanisms. O’Prior similarly reports .8335 AUC for its hybrid-only configuration and .8194 for its full composition in a small TabArena experiment [14]. A larger variety of tasks may spread a fixed training budget too thinly. Mitra-v2 also finds limited benefit from coarse changes to outer mixture weights [13].

Recent leading models point to several types of data worth covering. Kumo includes structured missingness, coarsened measurements, high-cardinality categories and heavy-tailed targets. TabPFN-3.5 emphasizes categorical, wide and grouped data, together with changes to its representation and architecture [12, 5]. These descriptions identify candidate mechanisms, but do not determine how much training each deserves. We study them within ordinary single-table classification and regression. Financial label delays and support acquisition are reserved for the separate specialist.

Our hypothesis is that useful additions will teach the model when a simple rule is adequate, how much to pool evidence across rare categories and how much uncertainty to retain when measurements lose

information. Direct trees and smooth functions allow closer control of these questions than randomly composed graphs. Sparse interactions test feature selection when individual features carry little marginal signal. Each task also requires an encoder that preserves the information needed to solve it. TabICLv2’s ablations show that gains from prior and architecture changes need not be independent [8].

## 5.2 A specified starting mixture

We denote the candidate mixture by  $Q$  and begin with the following requested probabilities for its five mechanism families:

$$(w_R, w_F, w_H, w_S, w_I) = (.50, .20, .15, .10, .05).$$

R receives half the draws, retaining broad reference coverage while leaving half the tasks for mechanisms whose behavior can be examined directly. Forests receive the largest added share because controlled experiments motivate testing their complementarity. Hierarchical tasks provide repeated and rare category identities for training the nominal representation. Smooth/local functions and sparse interactions receive smaller shares because R already contains related mechanisms. These coarse weights specify an initial experiment. They have neither been inferred from competitor checkpoints nor established as optimal.

| Source | Requested share | Inference problem emphasized                                                                 |
|--------|-----------------|----------------------------------------------------------------------------------------------|
| R      | 50%             | Broad vector-valued graph composition, with the native response and filtering laws.          |
| F      | 20%             | Direct axis-aligned or oblique piecewise rules, beyond their occurrence inside a random DAG. |
| H      | 15%             | Repeated, rare and unseen category identities with observable metadata and group structure.  |
| S      | 10%             | Additive smooth responses and local functions with controlled intrinsic dimension.           |
| I      | 5%              | Sparse feature interactions, including cases with little marginal feature signal.            |

Table 6: Initial candidate mixture. F, H, S and I refer to the distributions specified in this section; similarly named historical controls do not implement all of these distributions.

Every comparison first draws the task type and requested table shape from the same reference distribution. Classification and regression each retain half the probability, and class budgets, row ranges and specified later-stage width ranges remain fixed. H requires at least eight columns; a narrower H request falls back to R. F, S and I support every positive width. If  $e$  is the probability that the requested shape is eligible for H, the expected realized source probabilities are

$$w'_H = .15e, \quad w'_R = .50 + .15(1 - e), \quad (w'_F, w'_S, w'_I) = (.20, .10, .05).$$

Because this fallback transfers tasks from H to R, the planner integrates the width distribution for each stage and reports both requested and eligibility-adjusted shares. For a provisional total of 64M tasks, the requested counts are 32M R, 12.8M F, 9.6M H, 6.4M S and 3.2M I. Only the R/H counts change after fallback. These are allocation targets; hardware measurements are still needed to determine whether the total volume fits the budget.

The observation mode is sampled independently of the mechanism: identity with probability .50, missing completely at random (MCAR) .15, missing at random conditional on observed drivers (MAR) .15, missingness that depends on the hidden value itself (MNAR) .10 and coarsening .10. Identity adds no measurement corruption, although the underlying process may still include noise and hidden variables.

Before the eligibility fallback, unchanged R with identity observation therefore accounts for 25% of all tasks. The reference control uses R with identity probability one, providing an unchanged baseline for assessing both new mechanisms and altered observations.

These weights determine the distribution of training problems, not fixed contributions to every prediction. Let  $c$  denote the requested task and shape,  $q_j$  the joint distribution of component  $j$ , and  $w_j(c)$  its probability after the eligibility adjustment. For an ideal Bayesian predictor, the evidence  $I$  also determines each component’s predictive weight:

$$q_w(y \mid I, c) = \sum_j \frac{w_j(c)q_j(I \mid c)}{\sum_k w_k(c)q_k(I \mid c)} q_j(y \mid I, c).$$

Allocating half the episodes to R therefore does not make R responsible for half of every prediction, nor does it bound the model’s loss relative to R on arbitrary real tables. It ensures that R remains well represented during training. Whether the learned inference procedure transfers better must be established empirically.

### 5.3 Generating the proposed mechanisms

Each new family uses separate random streams, identified by distinct keys, for process parameters, 4,096 calibration rows, support rows, query rows, measurements and category relabeling. Calibration rows are independent draws from the same sampled process. They set generator scales and thresholds but never appear in the model context. R needs a different approach because its unchanged adapter does not expose an independent pool from the same process. Its observation wrapper fits scales, quantiles and mask-rate intercepts using clean support X without labels, then fixes them for the queries. Adding native calibration rows would change R’s table shape and filtering distribution. In all cases, the input codec is fitted only on observed support.

If a new family produces a numerically invalid process, generation retries with the same source, task and shape, aborting after 100 attempts. R retains its native attempt limit and acceptance rules. Valid draws from the new families are kept even when they have missing classes, noisy labels or constant outcomes.

**Features and effective dimension.** For F, S and I, the number  $d$  of active feature coordinates depends on support size in 80% of sampled processes. These draws use  $\rho \sim \text{LogUniform}(8, 128)$  and set

$$d = \min\{F, \max(1, \lfloor n_s/\rho \rfloor)\}.$$

The remaining 20% draw  $d$  discrete-log-uniformly from one through  $F$ . This branch retains difficult or weakly identified problems and removes the older bank’s universal sixteen-feature cap. Tying dimension to context size is a sampling hypothesis, not a sample-complexity theorem. A matched comparison with complexity conditioning disabled uses the latter dimension distribution for every draw.

The fraction of active coordinates treated as categorical is drawn from  $\{0, .25, .5, 1\}$  with probabilities  $\{.35, .30, .25, .10\}$ . After rounding to a count, the generator chooses those coordinates randomly. Numerical coordinates are generated in equicorrelated Gaussian blocks of four, with correlation 0, .3, .8 at probabilities .4, .4, .2. A common Student- $t_5$  row scale is added with probability .2.

Draw each categorical cardinality from 2, 3, 5, 10, 32, 128. Use probabilities .25, .15, .20, .20, .15, .05, respectively. Category frequencies follow a power law with exponent  $U(0, 1.5)$  and a random ordering of the categories. A random lookup, centered and scaled to unit variance, maps each category to its latent

coordinate in the mechanism. The model receives the category ID rather than this hidden numerical effect.

The other  $F - d$  columns are sampled as proxies for active coordinates with probability 30% and as independent distractors with probability 70%. Numerical proxies add noise with scale  $\text{LogUniform}(.01, .3)$ ; categorical proxies replace their value with probability .05 using the same marginal distribution. To render numerical features, each column is multiplied by an independent random sign and a scale drawn from  $\text{LogUniform}(.1, 10)$ , with no offset. Columns and their type flags are then permuted together. Category names are independently recoded for every column and sampled process, using the same map for support and query. R still presents numerical columns: an observation wrapper cannot recover nominal types already discarded by its adapter.

**F: directly sampled forests.** F samples axis-aligned and sparse-oblique forests with equal probability, and chooses the number of trees uniformly from  $\{1, 4, 16, 64\}$ . In the branch conditioned on support size, a nominal number of support observations per leaf,  $m \sim \text{LogUniform}(8, 128)$ , determines depth through

$$D = \text{clip}(\lfloor \log_2 \max(1, n_s/m) \rfloor, 1, 10).$$

The unconditioned branch draws depth uniformly from one through ten. The conditioned formula corresponds to a balanced tree, whereas random split thresholds usually create unequal leaf frequencies. We therefore record actual leaf occupancy instead of assuming that the depth rule guarantees coverage.

For a numerical split, the threshold is a  $U(.1, .9)$  quantile of the calibration records that reach the node. A categorical split draws a nonempty proper subset uniformly from the levels present among those records, without imposing an ordering on category IDs. The node remains unsplit if the selected coordinate has only one observed level. Oblique splits combine two through  $\min(4, d)$  latent mechanism coordinates with unit-normalized Gaussian weights; when  $d = 1$ , the split is axis-aligned. Nodes receiving fewer than sixteen calibration rows also remain unsplit. Each leaf receives independent Gaussian scores for every output coordinate. Scores are summed across trees, divided by  $\sqrt{T}$  and calibrated separately to zero mean and unit variance. This is a directly sampled forest prior, distinct from Mitra’s fitted ExtraTrees and gradient-boosting generators.

**H: support-dependent categorical hierarchy.** H extends P1’s hierarchy while retaining its observable entity ID, parent-group ID and two metadata coordinates. Shared group effects, entity intercepts and optional slopes follow Appendix A.8. Half the sampled processes use the existing catalog-size distribution. The other half draw a target mean occupancy  $m \sim \text{LogUniform}(.5, 64)$  and set catalog size to  $\text{clip}(\text{round}(n_s/m), 8, 262144)$ . The mixture of uniform and power-law frequencies and the effect scales remain unchanged. For large catalogs, effects are generated only when needed from a deterministic key identifying the process and entity; the frequency distribution remains fixed and finite.

The base P1 mechanism uses the remaining  $F - 4$  coordinates. The dimension rule for F/S/I does not apply to H’s four reserved columns. Category counts and class coverage vary naturally across samples. H bypasses every information-losing step in the legacy P1 renderer: masking, grid rounding and clipped exponential warping. Before the common observation mode is applied, it retains only invertible feature transformations and consistent category renaming. Thus identity adds no hidden legacy corruption, and each other observation mode is applied only once.

For an entity absent from support, metadata and the parent group can still reveal shared effects, but its independent residual effect remains uncertain. The arbitrary category name provides no further measure of similarity. Evaluation therefore separates well-observed, rare and unseen identities, and includes simulated processes in which metadata is only weakly informative.

**S: smooth and local responses.** Half of S uses additive piecewise-linear splines. For every active mechanism coordinate, the generator selects four, eight or sixteen knots with equal probability and places them at equally spaced calibration quantile levels including zero and one. Knot heights are Gaussian, with linear extrapolation beyond the endpoints. Tied knots are removed before heights are assigned, and distinct knots have a minimum spacing of  $10^{-6}$  in standardized units. A single remaining knot defines a constant function. Independent Gaussian coefficients, normalized by their joint Euclidean norm, combine the coordinate functions.

The other half of S uses local radial basis functions (RBFs). The generator draws an intrinsic rank  $r$  discrete-log-uniformly from one through  $\min(8, d)$ , then projects the mechanism coordinates through a Gaussian matrix with unit-norm columns. It draws  $J \in \{16, 32, 64, 128\}$  uniformly and samples  $J$  independent anchor rows from the same process, in addition to the 4,096 response-calibration rows. With independent  $a_j \sim N(0, 1)$ , the score is

$$f(z) = J^{-1/2} \sum_{j=1}^J a_j \exp\{-\|z - c_j\|^2 / (2\ell^2)\}.$$

The RBF length scale approximately follows the density of the context. The generator draws an intended neighbor count  $k \sim \text{LogUniform}(4, 64)$  and an additional independent projected calibration sample of 512 rows. For each row, it excludes the row itself and finds the neighbor at rank  $\text{clip}(\text{round}(512k/n_s), 1, 511)$ . The median of these distances sets  $\ell$ , with a positive numerical floor. Because this rule only approximates the desired occupancy, actual neighbor counts are recorded. The final score is calibrated before the response is drawn.

**I: sparse interactions.** I draws interaction order from  $\{2, 3, 4, 6\}$  with probabilities .45, .30, .20, .05, capped by  $d$ . It requests a uniformly drawn number of distinct coordinate subsets from one through eight, using every available subset when there are too few. Each term is equally likely to be a product of bounded tanh coordinates or a parity function of threshold/category-subset bits. Numerical thresholds are independently drawn calibration quantiles in  $U(.2, .8)$ , and categorical subsets are nonempty proper subsets. The final column permutation means predictive coordinates can occur anywhere in the table.

Let  $h$  be the calibrated interaction score and  $g$  an independent linear score with Gaussian coefficients normalized to unit Euclidean norm. For  $\alpha \sim U(.25, 1)$ , the generator forms  $\sqrt{\alpha}h + \sqrt{1-\alpha}g$  and recalibrates it before drawing the response. Exact pure parity and independent random truth tables are used as diagnostics rather than assigned large shares of ordinary pretraining. These examples distinguish high-order interactions that have shared structure from tasks with little structure to learn.

**Responses.** R keeps its native response distribution. For each new classification process, the generator draws one score coordinate per requested class  $K$ , a temperature  $T \sim \text{LogUniform}(.25, 4)$  and target class masses from a symmetric Dirichlet distribution with concentration  $a \sim \text{LogUniform}(.2, 5)$ . It fits sum-zero intercepts on the independent calibration pool so that the mean softmax of  $s/T + b$  matches the target masses within absolute tolerance  $10^{-3}$ . The solver uses at most 200 damped Newton iterations. The audit records requested and achieved masses, maximum absolute error, total variation and solver failures; an absolute tolerance does not guarantee low relative error for rare classes. A failed solve aborts the process. After a successful solve, support and query labels are sampled independently, with no resampling to enforce class coverage.

For regression, new processes add standardized noise at scale  $\text{LogUniform}(.03, 1)$ . The noise distribution is Gaussian, Student- $t_5$  or centered lognormal with probabilities .6, .3, .1, with lognormal log-scale equal to one. Half the processes use constant noise variance. In the other half, the noise is multiplied by

$\exp\{\text{clip}(a^T z, -2, 2)\}$  and normalized by its calibration RMS. Here  $a$  is a unit-normalized Gaussian vector defined on a uniformly sampled number of coordinates from one through  $\min(4, d)$ .

The target then undergoes identity, asinh or signed-square rendering with probabilities .6, .2, .2. H instead uses identity/asinh with probabilities .75/.25. Its unbounded pair interactions can have an infinite fourth moment under shared Student- $t_5$  features, making signed-square rendering potentially yield infinite MSE. At the model interface, target standardization uses support only. A calibration standard deviation below  $10^{-6}$  results in a retained zero score and a no-signal flag; all scale denominators have a  $10^{-6}$  floor. Finite tail observations are retained without clipping. Initial training uses sampled-label supervised losses. Conditional supervision remains a separate, deferred estimator comparison.

## 5.4 Observation law and information boundaries

The generator draws labels from the complete simulated process before modifying what is observed. Recomputing labels from the damaged features would erase the uncertainty these tasks are intended to teach. Each episode uses one observation mode, with parameters shared across support and query and independent row-level realizations. The initial candidate adds no further support/query shift and does not combine multiple corruption modes.

Whenever a mode selects half the columns, the count is rounded upward. MCAR selects a nonempty random half and masks their cells independently at rate  $r \sim U(.01, .30)$ . MAR first reserves one through  $\min(3, F - 1)$  driver columns that always remain observed, then targets half the others. Its masking probability is a sigmoid of a unit-normalized linear combination of standardized driver values. For nominal drivers, fixed Gaussian-effect lookup tables are standardized using the specified calibration data; category IDs are never interpreted as ordered numbers. The intercept is fitted to the sampled masking rate, using independent calibration rows from the same process for new families and clean support X for R. If  $F = 1$ , MAR falls back to MCAR and records the fallback.

MNAR also selects a random half of the columns, but the probability of hiding a value depends on that column’s own complete standardized value or nominal effect. The slope is drawn from  $\{-2, -1, 1, 2\}$ , and the intercept is calibrated on the same data source used by MAR. In this way, missingness can convey outcome information through covariates without using a query label.

Coarsening selects a nonempty random half of the columns. Each selected numerical column is divided into two, four, eight, sixteen or thirty-two equiprobable calibration bins, with the number of bins chosen uniformly. A bin is represented by its calibration median, or its support median for R. Duplicate edges are removed, a column with only one observed value remains constant, and an empty requested bin uses the nearest existing representative. Nominal columns with more than two levels are merged by a random map into  $\max(2, \lceil C/2 \rceil)$  groups.

These transformations can make distinct latent records look identical even when their outcomes differ. The generator preserves these apparent contradictions and supplies explicit missing-value flags. It applies no ExtraTrees filter after observation. R’s native filter runs earlier, and new families use numerical-validity checks alone. An observed improvement must therefore be attributed to the specified distribution as a whole unless a further experiment separates the effect of function choice from filtering.

## 5.5 What can be established without pretraining

Before training a neural model, we can check whether a generator provides the intended information and whether the resulting inference problem is solvable. The four calculations below address these questions. `research/probes/prior_information_analysis.py` computes exact values and checks them with independent Monte Carlo simulations; `research/results/prior_information_analysis.json` records

seeds, uncertainty and results. These checks help constrain the design, but cannot identify mixture weights that will work best on real data.

First, consider a category effect  $\theta \sim N(0, \tau^2)$  with observations  $y_i \mid \theta \sim N(\theta, \sigma^2)$ , assuming the mean and variances are known. The posterior mean and its Bayes MSE are

$$\mathbb{E}[\theta \mid D] = \frac{n\tau^2}{\sigma^2 + n\tau^2}\bar{y}, \quad R_\theta(n) = \frac{\tau^2\sigma^2}{\sigma^2 + n\tau^2}.$$

For  $\tau^2 = 1, \sigma^2 = 4$ , the Bayes MSE for the latent effect is .8 after one observation, .2 after sixteen and .05882 after sixty-four. The appropriate degree of pooling depends on both the observation count and the ratio of noise variance to effect variance, motivating joint variation of these quantities. This calculation assumes known hyperparameters and does not specify the share of hierarchical tasks needed in the training mixture.

Second, let  $p_j$  denote the population probability of category  $j$ . The probability that an independent query belongs to a category absent from  $n$  independent support draws is

$$U(n) = \sum_j p_j(1 - p_j)^n.$$

For 128 categories and 128 support rows, uniform frequencies give  $U = .36644$ . A Zipf distribution with exponent 1.25 instead gives  $U = .17770$ , even though 69.73% of category identities remain unseen. At 512 support rows, uniform frequencies give .01803 and Zipf frequencies .06520, reversing the ordering. Thus neither cardinality nor the fraction of unseen identities is sufficient: coverage must account for how much query probability those identities carry.

Third, an exactly enumerated measurement example shows the uncertainty introduced by coarsening. Let  $Z$  be uniform on  $\{-2, -1, 1, 2\}$ , set  $Y = \mathbf{1}[Z > 0]$  and add independent sensor error  $E \in \{-2, 0, 2\}$  with probabilities .2, .6, .2, giving  $W = Z + E$ . Quantize  $W$  to  $X = 0$  below  $-1$ ,  $X = 1$  on  $[-1, 1]$  and  $X = 2$  above one. Even with the mechanism known, prediction from visible  $X$  has Bayes error .25, binary Brier risk .125 and log loss .346574 nats. Access to unquantized  $W$  reduces log loss to .294249; access to  $Z$  makes it zero. A predictor with latent information and a predictor conditioned on visible features therefore face different tasks. Correct conditional-label averaging in the loss can remain unbiased, but measuring performance against unattainable latent certainty would count irreducible uncertainty as a model error.

Finally, consider twelve uniform binary features and thirty-two noiseless context rows. If a prior assigns an independent fair label to every one of the  $2^{12}$  possible input cells, its Bayes error is

$$\frac{1}{2}(1 - 2^{-12})^{32} = .4961085.$$

On the same input distribution, a uniformly drawn affine parity rule has error  $4.7672 \times 10^{-7}$ , computed by exact propagation of the binary design-matrix rank. The two problems have identical feature occupancy, but parity supplies shared algebraic structure that makes inference much easier. Occupancy is therefore informative within a specified mechanism family; it cannot rank all priors or justify excluding every high-order interaction.

These checks use seed 20261003, with 50,000 independent replicates for the normal/coarsening calculations and 4,000 for occupancy/parity. Seven independent identity tests include exhaustive enumeration of small state spaces. The results support the correctness of the toy calculations. They do not measure neural transfer or latency.

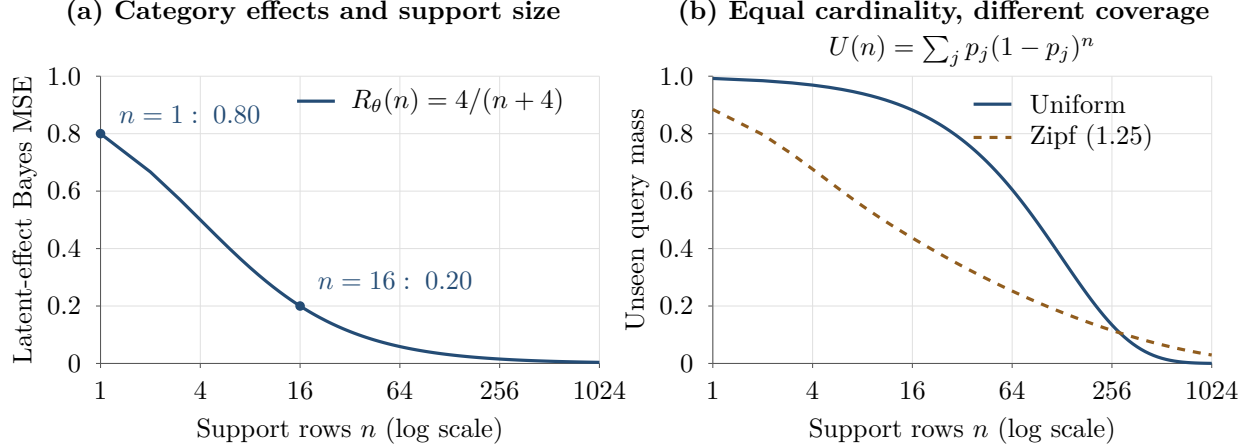


Figure 3: Exact diagnostics for the distribution of evidence in a synthetic prior. (a) For a category effect  $\theta \sim \mathcal{N}(0, 1)$  and  $y_i | \theta \sim \mathcal{N}(\theta, 4)$ , the posterior-mean risk for  $\theta$  is  $4/(n + 4)$ ; risk for a new noisy response adds 4. (b) Support and query categories are independent draws from the same law over 128 categories: either uniform or  $p_j \propto j^{-1.25}$ . Curves connect exact values at integer support sizes. Unseen query mass weights categories by their population frequency and therefore differs from the fraction of category identities absent from context. Neither calculation estimates benchmark accuracy or identifies optimal mixture weights.

## 5.6 Choosing the mixture within the compute budget

Phase P tests eight distributions: unchanged R; R with the observation mixture; R with 20% F; R with 15% H; R with 10% S; R with 5% I; full Q with identity observation; and full Q with the observation mixture. Each individual mechanism addition uses identity observation. Every configuration receives seeds 0 and 1 and 300 GPU-hours per seed, giving 4,800 hours in total, shared between classification and regression. This phase first measures individual additions, then tests their combination and the additional effect of the observation process.

Phase M changes the graph share to .70, .50, .30, dividing the remainder among F/H/S/I in the fixed ratio 4 : 3 : 2 : 1. The resulting vectors are (.70, .12, .09, .06, .03), (.50, .20, .15, .10, .05) and (.30, .28, .21, .14, .07). All use the same observation distribution, with fresh seeds 2 and 3 and 300 hours per seed, totaling 1,800 hours. Fresh seeds keep the central mixture from duplicating the Q runs in phase P. If diagnostics identify a more pressing uncertainty, one recorded slot may instead compare complexity conditioning on/off or test filter sensitivity. This substitution must be specified before launch and stay within the same allowance. Removing a family similarly defines a new candidate whose weights must be recorded before training.

Phase A crosses R and the selected challenger with the implemented compressed-row architecture and a compact alternative that retains feature-cell states. The four combinations, each run with two seeds at 400 hours per run, require 3,200 hours. Both architectures are implemented. Because their parameter counts differ, this tests complete systems at equal cost rather than isolating topology at equal model size. Phase V then confirms the complete reference and finalist with fresh seeds 4 and 5, allowing 700 hours per run for a total of 2,800 hours.

Phase F uses the remaining 2,400 hours for three continuations from a common standard parent: standard replay, fraud training with a context combining reservoir rows and observed positives, and fraud training with the full selector. Each continuation uses seeds 6 and 7 at 400 hours per seed. Both fraud comparisons hide the explicit hard-negative flag H, follow the permitted label-availability rules

and apply the correct population weights.

Together these five phases use 15,000 GPU-hours, replacing the earlier screening allocation in Section 10.6. Optimizer, head, codec, length-scaling and sharing settings remain fixed; dedicated sweeps of these factors are deferred. The existing 10,000-hour phase for size comparisons and intermediate-length training remains: two sizes with two seeds at 500 hours each, reference and challenger training extensions of 3,000 hours each, and two 1,000-hour runs for unresolved questions. R proceeds to the longer training extension even if a small pilot ranks it lower, because rankings can change with scale. Final training, evaluation and reserve keep the full project within 50,000 hours.

**Real-data selection.** Prior selection requires disjoint development, confirmation and final evaluation data. Before training, register at least twenty independent dataset lineages for each of binary classification, multiclass classification and regression in the development panel, and at least ten per task type in the confirmation panel. These counts are coverage targets rather than a power calculation. The panels should represent the intended feature types, widths, context sizes and missingness patterns. Duplicates, derived targets and related datasets count as one lineage. Official final benchmark folds and customer evaluation periods are excluded from mixture selection. If enough disjoint data cannot be obtained, the resulting claim must be narrower; final benchmark data must remain held out.

For each dataset, first average folds within each training seed, then compare paired seed averages. The primary improvement is the reduction relative to the reference in classification log loss or regression MSE, normalized by the corresponding constant-predictor loss estimated on that dataset’s training split. Denominator floors are fixed before evaluation, and raw metrics are also reported. Datasets receive equal weight within a task type, and the three task types receive equal weight overall. Binary AUC, multiclass log loss, regression RMSE and distributional calibration are reported separately to reveal deteriorations that an aggregate could obscure.

Exploratory configurations are ranked on development data with uncertainty reported. The finalist is then evaluated on the untouched confirmation panel. Its 95% paired dataset-lineage bootstrap interval must have a positive lower bound for overall improvement, and a lower bound above  $-.005$  normalized units for every task type. The  $.005$  tolerance is a stated design choice. The bootstrap resamples all paired predictions together by lineage, using 10,000 draws. Variation across training seeds is reported separately because two seeds estimate it poorly. If no candidate meets these criteria, the experiment has not identified a stronger bank. Repeatedly using confirmation results to revise the configuration turns that panel into development data and requires a new holdout.

Preprocessing, support size and inference policy are held fixed while selecting the prior. The surviving systems are subsequently compared on cold context setup, cached GPU batch prediction, peak memory and accuracy with  $1/4/8$  views. Predictive performance takes priority when assessing the measured accuracy–latency frontier. Changing a prior alters what the model learns without changing its arithmetic at a fixed shape. It may nevertheless improve latency indirectly if comparable accuracy becomes attainable with less context, fewer views, a smaller model or a more efficient implementation.

The classification constant predictor uses training class counts with Jeffreys smoothing,  $(n_k + 1/2)/(n + K/2)$ . For regression, the selection objective computes losses after the existing support-only target standardization. In both tasks, the paired loss reduction is divided by  $\max(10^{-6}, L_0)$ , where  $L_0$  is the training loss of the corresponding constant predictor in the same units. Thus the denominator is fixed before evaluating queries. Regression RMSE in raw units remains a separate endpoint. Bootstrap conclusions are conditional on the trained checkpoints; limited training-seed replication remains an additional source of uncertainty.

**Cheap diagnostics before GPU screens.** Before GPU training, we audit at least 1,000 independently sampled processes for every family and stage, stratified by observation mode and task type. The audit records actual informative dimensions, type masks, support occupancy, missingness, contradictions introduced by coarsening, class frequencies and the cost of rejection. It also checks independence from query labels, consistent category renaming and separation of calibration data.

A smaller stratified subset provides learning curves at finite context sizes. Ridge/logistic, kernel or nearest-neighbor and tree learners receive the same support data. Any selection among these learners uses cross-validation within support, followed by evaluation on a separate query set. Their losses provide attainable reference values rather than Bayes risks. Teacher comparisons retain negative estimated gaps and their uncertainty; failure of one learner does not justify removing an entire prior family.

The planner `scripts/plan_prior_selection.py` reads `research/specs/prior_selection_v1.json` and produces forty trial records listing costs, mixture shares and unresolved dependencies. Operating instructions are in `docs/prior_selection.md`. These files make the experiment reviewable without launching it. The sampler, shared distributions, compact architecture and training path are implemented. `scripts/materialize_prior_selection.py` creates runnable configurations for each phase, and `scripts/submit_training.py` submits a resolved run with a scheduler wall-time limit. Actual stage costs and real-data results remain to be measured.

## 6 Prior bank for rare-event financial tables

The financial candidate uses the implemented `risk_partition` prior with `hide_h` and target historical prevalence  $10^{-4}$ , approximately one event per  $10^4$  records. It first generates a finite population of covariates and then samples outcomes from overlapping risk regions. This creates the central inference problem: millions of transactions may contain few events, and only some of those events have known labels when a prediction is made. Bursts in arrivals and substantial missingness further shape the available evidence. Fraud detection and broad tabular performance are evaluated on separate panels. The candidate must meet both objectives; success on one panel does not establish success on the other.

A financial episode has three parts. The generator creates the transactions, features and label-reveal times. The context policy in Section 8.2 selects historical evidence for the predictor. The objective in Section 9.6 uses sampled future queries to estimate population risk. Selecting extra positives for context or training queries must not create evidence that the finite history lacks, or change the prediction target to the posterior of an enriched sample. We first describe the finite-evidence problem and the selected covariate-first candidate. We then give the shared arrival, observation and label-availability rules together with the earlier class-conditional generator, which remains a controlled stress test.

### 6.1 Finite positive evidence

At this prevalence, the amount of labeled positive evidence can matter more than total history size. Under an IID reference model, three million transactions contain about 300 positives in expectation, before label delays and selective verification reduce what is available. A uniform context of size  $s$  has no positive with probability

$$\Pr(M_s = 0) = (1 - \pi)^s. \quad (5)$$

With  $\pi = 10^{-4}$ , this probability is 81.5% for a 2,048-row reservoir and 3.77% for a 32,768-row context. Large random contexts are therefore a strong comparison for any proposed selector. Yet even 32,768 rows contain only about 3.28 positives on average, compared with about 300 in the full history. Bursts and shared campaigns can further limit their diversity. The intended selector retains available verified

positives, preserves representative background evidence, and retrieves negatives that help distinguish fraud from legitimate lookalikes.

The proposed contribution is a joint law for *finite evidence, observation, selection and prediction*. It includes histories with no observed positive evidence, fraud motifs that overlap with hard negatives, structured feature loss, and labels revealed late or selectively. Training and inference present the bounded-context selector with the same kinds of evidence. These conditions are poorly represented by ordinary balanced synthetic tasks. They still cannot make a new fraud mechanism identifiable when its features and history are indistinguishable from legitimate activity.

## 6.2 Classification and regression targets

The finance family includes multiclass outcomes as well as two regression targets. A multiclass world has one legitimate class and  $R \in \{2, 3, 4\}$  fraud types, drawn with probabilities .5/.3/.2. The declared alphabet is retained even when a type is absent from support. Regression tasks divide equally between transaction amount and realized fraud loss. Fraud loss has a large point mass at zero, so it uses the hurdle extension in Section 7.7. The ordinary continuous head remains an ablation.

If transaction amount  $a$  is observed and the target is exactly  $a\mathbf{1}\{\text{fraud}\}$ , classification supplies a separate expected-loss comparator:  $a p(\text{fraud} \mid x, H)$ . Partial recovery or additional variation in severity would need a separately specified outcome model.

## 6.3 Covariate-first finite-population prior

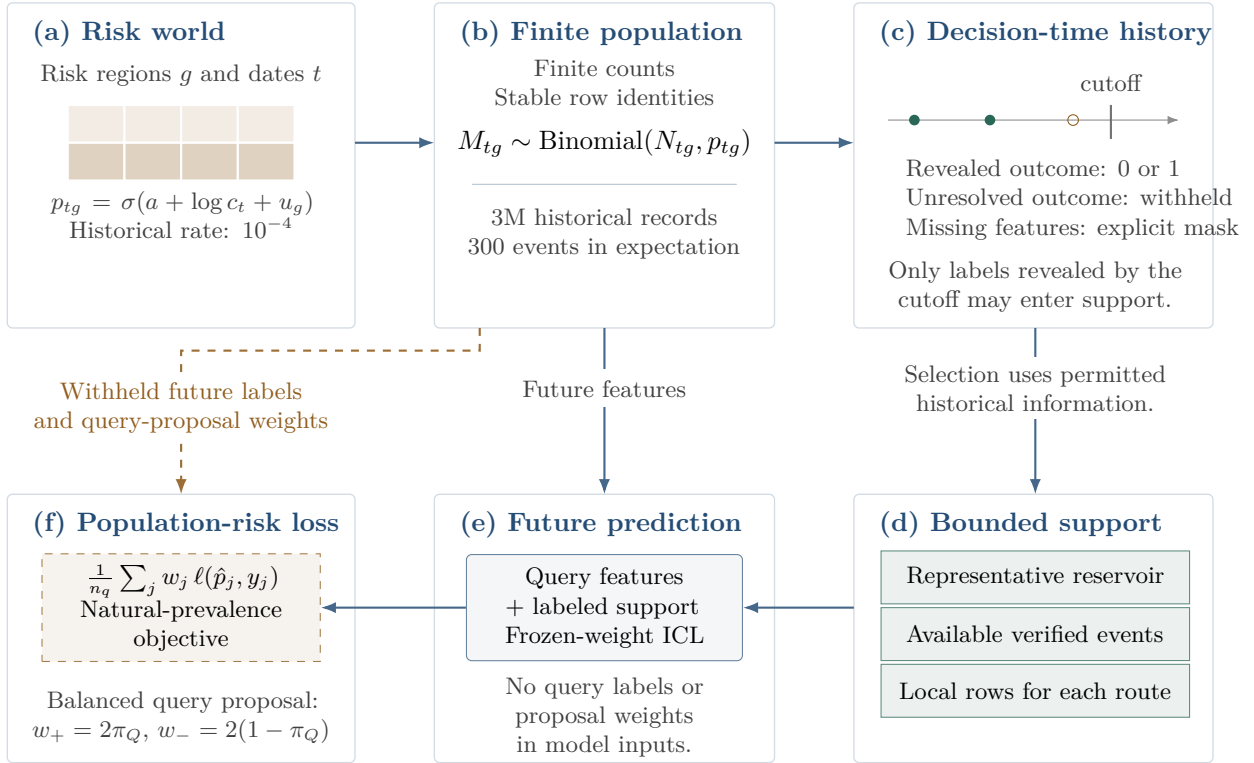
**Generation procedure.** The selected candidate generates covariates before outcomes. Two to five binary risk factors divide the finite population into four to 32 regions. Each region has a population frequency and a risk effect, including an interaction. Allocate historical and future records across dates and regions. Draw volume bursts separately from risk campaigns, so a rise in transaction count need not imply a rise in event probability.

Next, choose an intercept that makes expected historical prevalence  $10^{-4}$  conditional on the sampled covariate counts. Binomial draws determine the realized events. Thus three million historical records have 300 events in expectation, without a fixed quota of 300. Rendering and missingness determine observed features; reveal times determine which labels are known by the cutoff. The selector can then assemble its permitted reservoir, eventful rows and local context. The `hide_h` view removes the explicit operational cohort flag  $H$  from both selector and model, though indirect information about  $H$  can remain in observations.

Training samples future queries from a proposal that may oversample rare outcomes. When both classes exist and the finite future positive fraction is  $\pi_Q$ , a balanced proposal assigns loss weights  $2\pi_Q$  to positives and  $2(1 - \pi_Q)$  to negatives. At  $\pi_Q = 10^{-4}$ , these weights are .0002 and 1.9998. They make the 50/50 proposal estimate the naturally rare population objective. The predictor receives neither future class fractions nor query labels; it predicts from selected historical evidence and query features.

The following rules specify `risk_partition` precisely. Historical target prevalence is fixed at  $\pi = 10^{-4}$ , and outcomes are sampled from risk regions instead of selecting separated Gaussian motifs by class label. Draw the number of binary risk factors  $d$  uniformly from  $\{2, 3, 4, 5\}$ . Draw their independent Bernoulli probabilities from Beta(2, 2) clipped to  $[\text{.05}, \text{.95}]$ . Products of these probabilities and their complements define weights  $w_g$  for the  $2^d$  cells. Allocate each day's finite records as  $N_{tg} \sim \text{Multinomial}(N_t, w)$ .

Build cell risk scores from sparse signed main effects and one signed pair interaction. Center and normalize these scores under  $w$ , then multiply by strength  $U(.5, 4)$ . By default, the main-effect and



Volume bursts and risk changes are distinct.  
 The population is generated lazily; only selected rows enter context.

Figure 4: Generation and prediction under the finite-population finance prior. Only historical labels revealed by the cutoff may enter context. Future labels determine the training-query proposal and importance weights but remain hidden from the predictor. Counts and deterministic row IDs represent a large population, from which selected records supply a bounded context. Solid arrows carry data and predictions; the dashed arrow carries training targets.

interaction components receive equal weight. Apply rotation with probability one-half.

Cell effects  $u_g$  and risk-campaign effects  $c_t$  jointly set event probabilities:

$$p_{tg} = \text{sigmoid}(a + \log c_t + u_g), \quad \sum_{t \in \text{history}, g} N_{tg} p_{tg} / N_H = \pi.$$

Solve for the intercept by bisection using realized covariate counts, then draw independent event counts  $M_{tg} \sim \text{Binomial}(N_{tg}, p_{tg})$ . Calibration fixes expected prevalence conditional on features, without conditioning on observed outcomes. Actual event counts fluctuate and can be zero. The generator imposes neither a minimum positive quota nor a learnability rejection. The existing future-drift option can change later prevalence. Multiclass diagnostics allocate events using a cell-specific Dirichlet mode vector.

Within a cell, give each row signed half-normal risk coordinates and Gaussian nuisance coordinates, then apply the specified rotation and observation process. Conditional on day and cell, non-H covariates are independent of the realized outcome. Draw H afterward from its class-conditional probabilities. The candidate removes this column before retrieval and model input.

The finite-ID sampling and reveal-time rules are specified below. The natural reservoir, enriched support and importance-weighted future objective are retained as described in Sections 8.2 and 9.6. Other `feature_view` settings reveal H alone or the full feature set. The `positive_h_probability` setting also permits a control in which the generating probabilities make H nondiscriminative.

Changing feature view or world normalization preserves `population_id` and query draws. Changing the generative H probabilities changes the population law itself. Even when explicit H is hidden, missingness, reveal timing and selection can carry information. The resulting finite threshold-risk model includes overlap and conditional interactions, but is neither a general continuous hazard model nor an entity-specific attack model. Its oracle risk envelope is recorded across worlds; it is not used to retain only worlds with attractive AP.

Unit normalization preserves the intended population objective. Section 9.7 explains how the specialist uses it. Whether an unweighted mixture with standard tasks is safe remains a separate question.

## 6.4 Shared population rules and the earlier class-conditional model

The selected candidate and the earlier class-conditional model use the same arrival and label-availability rules except where a change is stated. This subsection specifies those shared rules and preserves the full earlier construction for stress tests of finite evidence, observation and selection. Its class-conditioned Gaussian features and sampled prevalence are replaced by the covariate-first risk model in Section 6.3. The label delays below are experimental assumptions, not measurements from the company. Implementation uses the seed namespaces and separate forward, loss and audit objects in Section 9.2.

**Population and arrivals.** Each world has a finite history and a finite future. Draw historical size  $N_{\text{total}}$  from  $\{300,000, 1,000,000, 3,000,000\}$  with probabilities  $.2/.3/.5$ . Historical records fall on days  $-90, \dots, -1$ , with the snapshot at day zero. The future contains  $\lfloor N_{\text{total}}/3 \rfloor$  records on days  $1, \dots, 30$ .

Half the worlds have uniform daily arrival weights. In the rest, draw a burst lasting 1, 3, 7 days with probabilities  $.5/.3/.2$ , a start uniformly among positions that fit the entire interval on the ordered 120-date axis, and a multiplier equally likely to be 5 or 20. Multiply weights within that nonwrapping interval, normalize history and future separately, then allocate daily counts with multinomial draws. These volume bursts are independent of changes in risk.

**Legacy prevalence and shared risk campaigns.** The earlier stress distribution varies event rarity around the target. Choose a log-prevalence interval from  $[-5, -4.5]$ ,  $[-4.5, -3.5]$  or  $[-3.5, -2]$  with probabilities .1/.7/.2, draw uniformly within it, and exponentiate base ten. Most mass lies around  $10^{-4}$ . Finance disables the standard class-probability floor, support-coverage floor and prevalence-contamination term, so observed classes can be rare or absent.

Half the worlds have no risk campaigns and use  $g_d = 1$ . The others draw one to four campaigns uniformly. Each campaign draws its date center uniformly, width 1, 3, 7 with probabilities .5/.3/.2, and amplitude  $A_c \sim \text{LU}(10, 100)$ . The legacy daily risk is

$$g_d = 1 + \sum_c A_c \exp \left[ -\frac{(d - t_c)^2}{2w_c^2} \right], \quad p_d = \sigma(\alpha + \log g_d).$$

Use 64 bisection steps on  $[-50, 20]$  to find  $\alpha$  so that the count-weighted historical mean of  $p_d$  matches the drawn  $\pi$ . Apply the same law to future dates. A separately flagged drift comparison adds  $\log \kappa$  to future log odds, with  $\kappa \in \{.3, 1, 3\}$  drawn with probabilities .2/.6/.2. The initial stationary comparison excludes this additional drift. The risk-partition candidate uses this campaign process together with its cell effects.

In the earlier model, draw daily fraud counts as  $M_d \sim \text{Binomial}(N_d, p_d)$ . A Dirichlet(.51 $\mathbf{1}_R$ ) vector, fixed for the world, allocates events among the  $R$  fraud types. Binary and regression worlds retain the types as latent structure. Zero-fraud histories and zero-fraud futures are valid. Under this count model, outcomes are conditionally independent given the world. Replacing binomial counts with beta-binomial counts would add an assumption about dependence.

**Finite identities and lazy generation.** Large populations are represented by counts and persistent ranges of row IDs for each date/class/cohort/reveal stratum. Allocate samples without replacement across strata using the multivariate hypergeometric law, then select distinct IDs within each stratum. Key all row randomness by world and row ID, so a repeated visit returns the same record. Only generated positives with revealed labels belong to the eligible positive pool.

An oversampled future-query proposal may revisit a finite ID when estimating loss, but the revisit adds no independent evidence. Retrieval generates projected features for every candidate it examines, then full features for the selected context and query records. For strata defined by label availability, reveal times are drawn from the corresponding conditional law so that each generated record is consistent with its stratum.

**Legacy signal, overlap and recurrent motifs.** The earlier model represents fraud motifs with eight signal coordinates. Each type draws  $\mu_r \sim N(0, 4I_4)$ . The world draws  $\sigma \in \{.25, .5, 1\}$  with probabilities .25/.5/.25 and  $\Delta \in \{0, 1, 2, 4, 6\}$  with probabilities .05/.15/.25/.35/.20. A fraud record of type  $r$  has

$$v_{1:4} \sim N(\mu_r, \sigma^2 I_4), \quad v_5 \sim N(\Delta, 1), \quad v_6 \sim N((-1)^r \Delta/2, 1), \quad v_{7:8} \sim N(0, I_2).$$

For fraud records, an observed operational cohort flag  $H$  follows Bernoulli(.8). For legitimate records it follows  $H \sim \text{Bernoulli}(h)$ , where  $h \in \{10^{-6}, 10^{-4}, 10^{-3}, .01\}$  has probabilities .1/.3/.3/.3. Legitimate  $H = 0$  records have eight standard-normal coordinates. Legitimate  $H = 1$  records follow the fraud mixture with the mean of  $v_5$  set to  $-\Delta$ , except that a fraction  $\xi$  retain  $+\Delta$ . Draw  $\xi \in \{0, .001, .01, .1\}$  with probabilities .4/.3/.2/.1.

This construction gives some legitimate records the obvious fraud motif, including a subset whose overlap cannot be resolved from these features. Worlds with  $\Delta = 0$  and worlds with high overlap are retained.

Shared entity offsets create recurrent feature patterns. Draw IDs from  $\min(N_{\text{total}}, 10,000)$  entities with Zipf exponent 1.1, and give each entity a fixed  $N(0, .25^2 I_8)$  offset. Campaign offsets follow  $N(0, .5^2 I_8)$ . Given date, assign a campaign in proportion to its contribution to  $g_d$ , with weight one for background. Fraud records and matching hard negatives use the same assignment law.

Before standard feature rendering, apply one random orthogonal rotation to the eight signal coordinates. Total width is at least 16. Four additional observable columns hold  $H$ , acquisition channel, entity ID and event date; the standard copy/distractor mechanism fills the remaining columns. Event IDs identify finite records internally and are never predictors.

Generate transaction amount as  $a = \exp(2 + .4v_1 + .2v_2 + \epsilon)$  for  $\epsilon \sim N(0, 1)$ . With probability .05, multiply by an independent Pareto draw with minimum one and shape three to add a heavy tail. Hide amount from inputs when it is the regression target. It may be observed when the target is  $a\mathbf{1}\{\text{fraud}\}$ . Event dates and historical summaries may likewise contain only information available when the prediction is made.

**Structured missingness.** Missing financial measurements can occur in related groups. Divide eligible predictor columns into four fixed blocks, excluding identifiers and  $H$ . Keep acquisition channel observed as an indicator of availability. A world chooses MCAR, channel-block, value-dependent or outage masking with probabilities .20, .40, .30 and .10.

MCAR draws cell-mask rate  $\text{LU}(.01, .20)$ . In a channel-block world, each record draws one of four channels uniformly. A fixed  $4 \times 4$  Bernoulli(.8) availability matrix gives block-missing probabilities of .02 or .95. One Bernoulli draw then masks each record/block. For value-dependent masking, each block selects one latent signal coordinate and uses probability

$$\sigma(b + .75v_j + .81\{\text{channel} = 0\}).$$

Choose the desired mean equally from .05, .20, .50. An independent 4,096-row calibration sample from the count-weighted historical mixture determines  $b$  through 64 bisection steps on  $[-20, 20]$ . Then use one Bernoulli draw to mask the whole block. An outage world chooses one block and a three-day interval uniformly; masking probability for that block is .95 inside the interval and .02 outside. Retain NaNs and explicit masks. At prediction time, a missingness indicator cannot encode the result of an investigation that has yet to occur.

**Label availability.** Past outcomes become usable evidence only after verification. Draw complete or incomplete adjudication with probabilities .4/.6. Complete adjudication reveals every label after delay  $D \in \{7, 30, 60\}$  days, chosen with equal probabilities. Incomplete adjudication provides no universal guarantee that a label will eventually be known.

Both settings permit three additional routes to verification. Independent random audits occur with probability .001 and reveal a label after two days. Operational review occurs with probability .001 for  $H = 0$  and .05 for  $H = 1$ , with a one-day delay. Positive self-reporting occurs with probability  $r \in \{.5, .8, 1\}$ , drawn with probabilities .2/.4/.4, and delay  $\min(60, \max(1, \lceil \text{LogNormal}(\log 7, 1) \rceil))$ . The earliest successful route reveals the label, assumed correct in this initial model. Full-amount regression has a separate fixed one-day observation delay and does not wait for fraud adjudication.

Reveal probabilities are constant within date/class/ $H$  strata. Exact binomial splitting can therefore determine availability without scanning every negative record’s features. The simulator stores the latent outcome, label mask and conditional reveal time separately, and derives availability from the verification rules. Support includes only labels known at the snapshot. An outcome that has not been reported cannot be treated as a negative.

A complete reference cohort contains all historical records old enough to fall under the universal adjudication rule, from both classes. Newer reviewed records can enlarge support, but the class frequency in that combined set no longer estimates population prevalence. Fraud research describes this operational validation problem [45, 46]. Corrections based only on positive labels also require assumptions about how labels are selected [47].

## 6.5 Shortcut and coverage audit of the finance worlds

The earlier class-conditional model contains a measurable shortcut. It is a controlled stress world rather than a validated model of company fraud, and strong performance on it can overstate detection ability. Its observable hard-negative flag  $H$  has  $P(H = 1 | Y = 1) = 0.8$  and  $P(H = 1 | Y = 0) = h$ , where  $h \in [10^{-6}, 10^{-2}]$ . In this subsection,  $H$  means this feature rather than the history object used elsewhere. Using the flag alone gives

$$\text{AUC}_H = \frac{1 + 0.8 - h}{2}, \quad P(Y = 1 | H = 1) = \frac{0.8\pi}{0.8\pi + (1 - \pi)h}.$$

At  $\pi = 10^{-4}$ , its AUC is 0.895–0.900 without learning a motif. When  $h = 10^{-6}$ , precision at recall 0.8 is 98.77%. With the usual threshold-step treatment of the two tied scores, population AP is  $0.8P(Y = 1 | H = 1) + 0.2\pi$ . It ranges from 0.00637 at  $h = 0.01$  to 0.79014 at  $h = 10^{-6}$ , with finite populations fluctuating around those values. High synthetic AP can therefore arise from the supplied flag alone.

The diagnostics compare this H-only baseline with a paired feature view that hides explicit H from *both* the predictor and the selector. H also affects features and observation channels, so removing the column does not erase every source of information about it. A second control weakens the generating mechanism itself: before producing dependent observations, set  $P(H = 1 | Y = 1) = P(H = 1 | Y = 0) = h$ . Repeat population and selection checks under this changed law. Where oracle ceilings can be derived, compare them with all-feature trees and the frozen ICL candidate on the same worlds.

The earlier motifs have other limits. They are rotated eight-dimensional Gaussians. Entity offsets change features but not entity-specific risk, and campaigns change volume or prevalence without creating a new attack structure. The risk-partition prior in Section 6.3 adds overlapping outcomes and conditional interactions. Emerging motifs, subgroup-specific risk and uninformative missingness still require broader evaluation. The earlier generator remains useful for isolating the effects of finite evidence, label availability and context selection.

## 7 The model architecture

The starting model is the 317,116,304-parameter compressed-row architecture introduced in Figure 1. It shares a backbone across classification and regression, with a separate output head for each task. Numerical features, categories, and precomputed temporal or customer features all enter as table columns. Two stages of feature processing produce one vector per row; a 24-layer transformer then predicts each query from those vectors and the labeled support. The older 315,021,456-parameter ICL-315 model is retained as a control. The 41M alternative is described in Section 7.6. Architecture comparisons hold the prior fixed.

We describe the computation in the order it is executed: preprocessing, cell embedding, feature interactions, row attention and prediction. We then specify the additional inputs used by the finance model. Section 8 explains how to cache support states and select context from a large history. The deferred architecture experiments are collected in Appendix B; none is enabled in the starting configuration.

## 7.1 Architecture overview

The model has to answer two questions: what is in this record, and what relationship do the labeled examples suggest? Preprocessing preserves numerical values, category identity and missingness. The first attention stages combine a record’s features while drawing information from the support table. The row transformer then compares its representation with labeled support rows. Figure 5 shows this computation and the support states reused when predicting another batch of queries.

Column attention collects information about one feature across the support rows. Within-row attention combines the features of one record. Eight learned summary tokens carry the result into the row transformer; they are trainable representations of the row, rather than eight selected raw features or examples. Compressing a row in this way makes a wide transformer affordable. It can also discard information, which is why the comparison with a model that retains feature cells is part of the study.

Once the support set and weights are fixed, we can fit preprocessing and compute the support caches once. Later query batches reuse them. A query’s prediction is mathematically independent of the other query rows, allowing for numerical tolerance in the implementation. Caching reduces repeated setup work, although each query still attends to its support. A change to the support, weights or relevant preprocessing requires the affected caches to be rebuilt.

The figure describes one prediction task. The implementation processes variable-sized episodes separately and accumulates their losses into a logical global batch. It does not combine unrelated tasks in one padded attention tensor. The following dimensions and masks therefore refer to a single episode.

## 7.2 Three choices in the input, output and attention layers

Three important choices concern the inputs, regression output and attention cache. Categorical hash bits specify how a category enters the network. The quantile head defines the output for regression. Shared query KV heads reduce the support state stored for repeated prediction.

**Categorical hash-bit inputs.** A categorical value identifies a group, such as a merchant or product, without defining a numerical order. Assigning merchant IDs 1, 2 and 3 does not make the third merchant three times the first. The encoder therefore gives each category a deterministic bit pattern. For a fixed column identifier and encoding seed, the same category receives the same pattern in support and query.

The implementation takes 64 bits from a SHA-256 hash of the typed category key, feature identity and seed, then represents each bit by  $-1$  or  $+1$ . For illustration, the four-bit pattern 1010 would become  $(+1, -1, +1, -1)$ ; the actual representation has 64 coordinates. It appends support frequency, an unseen-category flag and a missing-value flag, giving 67 input coordinates. A learned linear projection converts these coordinates into the cell representation used by the transformer. Thus the hash is fixed, while the network learns how to use its output.

This representation accommodates categories absent from pretraining without allocating a separate learned lookup entry to every possible ID. It also keeps each raw identity bit exactly representable in BF16, unlike nearby scalar codes that can become indistinguishable after rounding. The bit pattern itself carries no merchant semantics or target risk: those relationships must be inferred from the other features and labeled context. Hash collisions and information loss after the learned projection remain possible. Section 7.3 specifies the support-only collision checks, missing-category encoding and precision tests.

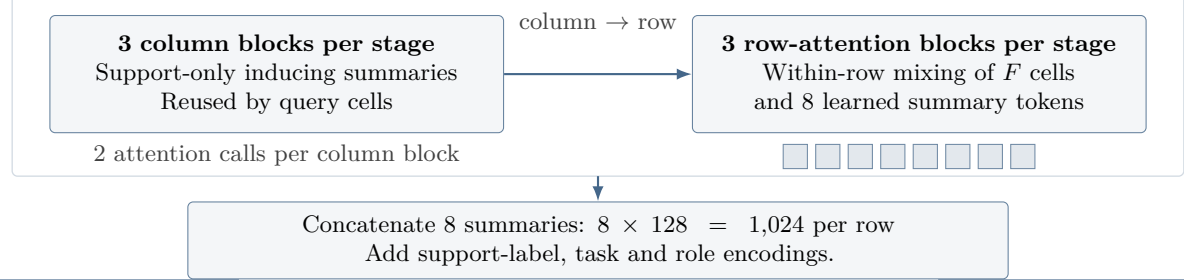
## (a) Typed cell inputs



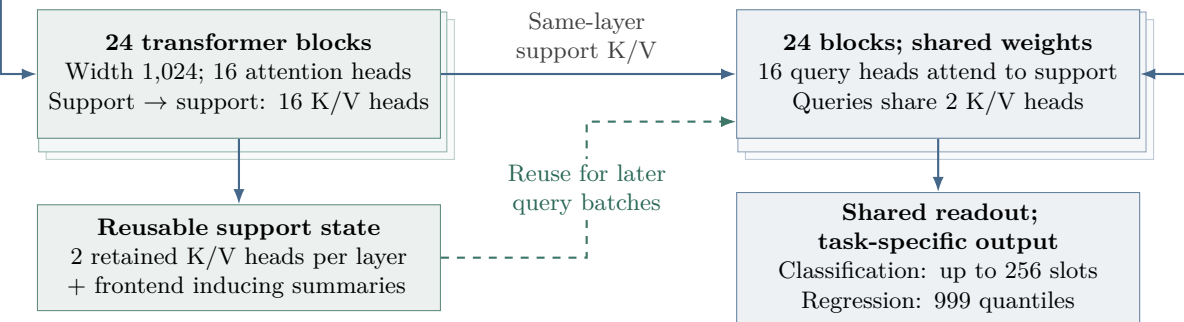
## (b) Support-fitted codec and cell encoder: width 128

Numerical values + ranks | 64 categorical hash bits | missingness flags  
 Support-label encoding is added; the query-label contribution is zero.

## (c) Two frontend stages



## (d) Support prefill



No query-to-query attention; queries never update support.

The frontend has 18 attention blocks in total.

The eight summaries are learned representations, not selected features.

Figure 5: Detailed computation in the 317,116,304-parameter starting model (base). Support and query paths share weights. Each column summary block contains two attention calls, giving 18 frontend attention blocks in total. The query path shares two KV heads, reducing retained cache storage; support prefill uses all 16 heads. Queries neither attend to one another nor update support states. The optional severity head is omitted here and is not trained by the standard and binary-fraud runs. Matrices and layer stacks are schematic; dimensions are stated explicitly.

**A quantile regression head.** A prediction head is the final learned mapping from a query’s hidden representation to its outputs. For classification, those outputs determine class probabilities. For regression, the head used here predicts conditional percentiles of the numerical target, given the query features and labeled support. The 0.5 quantile is a median; the 0.9 quantile is an upper percentile intended to place approximately 90% of conditional outcomes at or below it. For discrete targets, mass at the quantile can make this fraction exceed 90%.

For example, predicted 0.1, 0.5 and 0.9 quantiles of 20, 35 and 80 describe a median prediction of 35 and an intended central 80% prediction interval from 20 to 80. This is an interval for an individual outcome, not a confidence interval for a fitted mean. Its coverage must be measured on held-out data. The model predicts 999 such values, at levels  $\tau_k = k/1000$  for  $k = 1, \dots, 999$ , through one shared network.

Each output is trained with the asymmetric absolute-error loss called the *pinball loss*. If  $e = z - \hat{q}_\tau$  is the difference between the support-standardized target and its predicted quantile, then

$$\rho_\tau(e) = \begin{cases} \tau e, & e \geq 0, \\ (\tau - 1)e, & e < 0. \end{cases}$$

At  $\tau = 0.9$ , an underestimate has nine times the penalty of an equally large overestimate. This asymmetry makes the population loss favor the requested percentile. Training averages the 999 losses on the raw outputs. Prediction transforms them back to the target’s original units and sorts them to give an ordered quantile curve. The point prediction averages the grid, approximating a conditional mean; it is distinct from taking the median. A finite grid does not specify a complete density or resolve arbitrarily rare tails. Section 7.5 gives the head specification, its comparison with histogram outputs and a rare-tail counterexample.

**Shared query key/value heads.** Attention uses learned *query* (Q), *key* (K) and *value* (V) vectors. In the row transformer’s prediction path, an unlabeled row supplies Q, and labeled support rows supply K and V. Comparing Q with each K determines attention weights; the weighted combination of V supplies information from support. An attention head performs this operation in one learned subspace. Different query heads can assign different weights to the same support rows.

The candidate retains 16 query heads but shares two sets of support K/V vectors among them. The first eight query heads use the first K/V head, and the remaining eight use the second. Their Q vectors are still distinct, so sharing K/V does not force their attention weights or outputs to be identical. The implementation first computes support self-attention with all 16 heads, then keeps the first two K/V heads in the cache used for queries. This is query-only grouped attention, applied consistently during training and inference.

At fixed support size and head dimension, retaining two K/V heads instead of sixteen makes the persistent trunk K/V cache eight times smaller. It does not remove support rows, reduce the number of query heads, or make the complete model eight times faster. Support preparation still uses full-head attention, other caches and activations remain, and backend kernels determine the realized bandwidth benefit. It also changes the predictor relative to independent K/V heads, so predictive quality requires comparison. Section 8 gives the exact cache sizes and latency protocol.

These mechanisms are implemented in `codec.py`, `model.py` and `distributions.py`: the codec constructs category identities, the model projects inputs and produces quantiles and attention states, and the loss module evaluates pinball scores. Section 12.1 locates these modules in the repository.

### 7.3 Preprocessing and cell embeddings

Let  $n_s$  and  $n_q$  be the numbers of support and query rows,  $N = n_s + n_q$  their total, and  $F$  the number of features. We omit a batch dimension because each forward call handles one episode. The inputs are  $X_s$ ,  $y_s$ ,  $X_q$ , feature-type masks, the task type and the declared class vocabulary. Query targets are passed only to the loss function.

The class vocabulary must be known without examining the query labels. Synthetic training declares it before generating outcomes. At deployment, we use an independently supplied vocabulary or, if none is supplied, the classes in the support/training fold. A query belonging to an unknown class is reported as unsupported and remains in the evaluation. It does not cause the vocabulary to be expanded or the task to be rejected. The implemented head supports up to 256 classes.

Numerical features are normalized using finite support values only. For each feature, take their median  $m$  and median absolute deviation  $\text{MAD} = \text{median}(|x - m|)$ . The preferred scale is  $s = 1.4826\text{MAD}$ . If  $\text{MAD}$  is at most  $10^{-12}$ , use the population standard deviation; if that is also at most  $10^{-12}$ , use one. Finally, floor the scale at  $10^{-3}$ . With no finite support values, use  $m = 0$ ,  $s = 1$ . Even-length medians average the two middle values. Compute  $z = (x - m)/s$  in a sufficiently wide accumulator and retain indicators of clipping.

Both magnitude and ordering can matter in tabular prediction. We therefore represent a numerical value by eight channels:  $\text{clip}(z, -100, 100)$ ,  $\text{sign}(z) \log(1 + \min(|z|, 10^6))$ , its support midrank, a missingness flag, positive- and negative-infinity flags, and the indicators  $z < -100$  and  $z > 100$ . With  $n_f$  finite support values, midrank is  $(\#\{x_s < x\} + \frac{1}{2}\#\{x_s = x\} + \frac{1}{2})/(n_f + 1)$ . A nonfinite value, or a value with an empty reference set, receives rank  $1/2$ , zero numerical channels and its explicit flags.

We add 32 learned sine/cosine pairs of the clipped value and eight fixed pairs of its rank at frequencies  $1, \dots, 8$ . Together with the eight channels above, this gives 88 inputs. The learned positive frequencies start logarithmically spaced from 0.01 to 10. Their logarithms are optimized, and the frequencies used in the forward pass are clipped to  $[10^{-4}, 100]$ . Every Fourier argument includes  $2\pi$ .

Missing features do not need to be filled in before entering the model. A numerical NaN uses a finite placeholder and an explicit missingness flag; it is excluded when computing support location, scale and ranks. A missing category is distinguished from an observed category that was absent from support. This treatment concerns missing features. Support target labels are still required.

The categorical encoder described above uses 64 signed bits from a SHA-256 prefix. Support frequency, an unseen-category flag and a missing-value flag bring its input to 67 coordinates. Each bit is exactly representable in BF16. The hash depends on the typed category key, column identifier and encoding seed; collision checks on support do not change the mapping when queries arrive. This projection uses 384 fewer parameters than the earlier scalar-Fourier control.

For an ideal random hash, the collision probability among a million keys is approximately  $2.7 \cdot 10^{-8}$  before checking the observed support. The precision probe below found that hash bits separated the problematic pair under all six tested initializations. This establishes separation for that pair at initialization; it does not show how well a trained projection preserves category information.

The earlier scalar-Fourier encoder remains available for comparison. It uses a deterministic random code in  $[-1, 1)$ , support relative frequency, an unseen-in-support flag and a missingness flag. Adding 32 learned Fourier pairs gives 68 channels. The scalar code is formed from the first 53 SHA256 bits of a length-prefixed serialization of the encoding seed, stable feature identity and type-tagged category value. The resulting code identifies a category; it carries no language semantics.

Only observed support keys are checked for hash collisions. If a collision occurs, the encoding stream is redrawn up to eight times, after which the episode fails explicitly. Query values never change the

encoding seed, support codes or acceptance of the task. Consequently, the current implementation neither detects nor logs query-code collisions, and unseen keys can still collide. Support and query use the same hashing rule. Missing categories have zero value and Fourier channels, together with their missingness flag.

Preprocessing currently computes codes, normalization, ranks, Fourier phases and trigonometric features in FP64. Completed vectors are cast to FP32 before BF16 projection. An FP32 preprocessing path would need separate validation. The audit checks collisions after casting as well as at the source hash, since distinct hashes can still lead to indistinguishable numerical representations.

A precision probe shows why this distinction matters. Among 50,000 categories with encoding seed zero, `category_28838` and `category_39200` receive different scalar codes separated by  $1.6093 \times 10^{-9}$ . They pass the support hash check, yet a seeded tiny encoder maps them to identical vectors under CPU BF16 autocast; the largest FP32 difference is  $3.58 \times 10^{-7}$ . This is a reproducible failure for one encoder and one pair, not a result about every trained encoder or GPU implementation. Hardware validation must therefore check identity after projection, using different recodings and held-out category counts.

Before attention, each feature embedding also incorporates two neighboring positions. At anchor position  $j$ , three slot-specific bias-free projections read  $j$ ,  $(j + 1) \bmod F$  and  $(j + 3) \bmod F$ . Their outputs are summed into a width-128 vector. Numerical and categorical projections, and frequency banks, are separate for each slot. This operation keeps every anchor position: the output has shape  $N \times F \times 128$ , rather than reducing the number of features to  $F/3$ .

To each cell vector we add learned encodings of the anchor feature type, task, support/query role and support label. Classification labels use a  $256 \times 128$  lookup. Regression labels use a bias-free projection of the standardized target, its signed logarithm and a clipping indicator. The raw target is clipped at 20 and the logarithm at  $10^6$ . Queries receive exactly zero in the label channel. An affine LayerNorm with epsilon  $10^{-5}$  finishes the cell embedding.

## 7.4 Feature attention, row compression and support attention

A column block shares information across rows for one feature; a row block combines features within one record. There are two feature-processing stages, each containing three column ISABs and three row self-attention blocks. Here ISAB means induced set-attention block, and MAB denotes the multihead attention block used inside it. These blocks use width 128, eight heads of dimension 16 and SwiGLU hidden width 512. Each ISAB has 128 learned inducing vectors. For feature  $j$ , it computes

$$H_j = \text{MAB}(I_j, Z_{s,j}), \quad Z_{:,j} = \text{MAB}(Z_{:,j}, H_j).$$

The inducing parameters are shared across feature positions, but the summaries they produce depend on the feature and episode. Only support rows provide keys and values to the first attention call. Both support and query cells then read those summaries. Missing cells remain valid tokens because the missingness flag can be informative. If padding is introduced, only padding is masked.

Eight learned summary tokens are appended to each row before its first row-attention block. Every row block attends over that row’s  $F + 8$  tokens. Only this within-row attention uses parameter-free rotary positions: the base is 10,000, features occupy positions  $0, \dots, F - 1$  and summary tokens occupy  $F, \dots, F + 7$ . The summary tokens persist between stages. The second column stage updates feature cells while leaving those tokens unchanged; the next row blocks update both. Concatenating the final eight summaries gives  $R \in \mathbb{R}^{N \times 1024}$ . We then add separate width-1,024 task, role and support-label encodings, using the same label definitions as in the cell encoder.

The row transformer has 24 blocks with independent parameters, width 1,024, 16 attention heads of dimension 64 and SwiGLU hidden width 2,816. It has no row-position encoding. Support rows attend

to all support rows, including themselves. Queries attend only to support. A query never provides a key or value, even for itself; its own features remain in its residual stream. The visibility mask is

$$M = \begin{pmatrix} 0_{n_s \times n_s} & -\infty_{n_s \times n_q} \\ 0_{n_q \times n_s} & -\infty_{n_q \times n_q} \end{pmatrix}.$$

This mask makes support caching and separate query batches possible: adding or removing another query cannot change the prediction for a fixed row. Any implementation that pads or packs episodes must also mask support padding and prevent communication between episodes. The current API keeps them separate. Standard training requires nonempty support. For the empty-support case used by the finance extension, an attention branch with no valid keys returns zero, leaving the residual computation and default preprocessing statistics in place.

All attention blocks use the same residual structure. Each MAB contains bias-free Q, K, V and output projections, followed by three SwiGLU matrices:

$$\begin{aligned} U &= Q + \text{RMS}_2(\text{Attn}(\text{RMS}_1(Q), \text{RMS}_1(KV))), \\ O &= U + \text{RMS}_4\{W_d[\text{SiLU}(W_g \text{RMS}_3(U)) \odot W_v \text{RMS}_3(U)]\}. \end{aligned}$$

The first RMSNorm uses shared parameters for the query and key/value inputs. Each of the four RMSNorms has a learned gain, no bias and epsilon  $10^{-6}$ . Projected Q and K vectors are also normalized within each head, using separate learned head-dimension gains shared across heads. When rotary positions are used, this normalization comes first.

Attention scaling changes with the number of available keys. In the older control, each head’s  $QK^\top/\sqrt{d_h}$  is multiplied by

$$\tau_h(n) = \exp\{\tanh(a_h) \text{clip}[\log(\max(n, 1)/1024), -\log 4, \log 4]\},$$

where  $n$  is the number of permitted keys and  $a_h$  is initialized at zero. The starting configuration replaces this bounded rule in support-reading inducing attention and the ICL trunk with

$$T_h(n) = \frac{\text{softplus}(a_h)}{\log 2} \frac{\log(\max(n, 2))}{\log 1024}, \quad a_h(0) = 0.$$

This multiplier equals one for 1,024 keys at initialization and keeps increasing beyond the old rule’s saturation point of 4,096 keys. Attention over fixed-size inducing summaries and within-row feature tokens retains the older scaling. Query attention uses the cached support count, so its temperature does not depend on query-batch size. This particular logarithmic rule is an implemented alternative, not a reproduction of every detail of QASSMax.

We use zero dropout. Ordinary linear weights start with standard deviation  $1/\sqrt{\text{fan\_in}}$ , embeddings with 0.02, and output-head weights with  $0.02/\sqrt{\text{fan\_in}}$ . RMS gains start at one, except for residual-output gains: these start at  $1/\sqrt{36}$  in the 18 frontend MABs and  $1/\sqrt{48}$  in the 24 row-transformer blocks.

## 7.5 Prediction heads and parameter counts

A width-1,024 RMSNorm and a shared bias-free  $1024 \rightarrow 1024$  exact-GELU layer feed the prediction heads. Classification uses 256 logits, with cross-entropy computed over the declared classes. During training, the episode’s  $K$  labels are assigned to an ordered  $K$ -element subset sampled without replacement from the 256 output slots. The remaining logits are masked. This exposes every output slot to training even when most tasks have few classes. Deployment can map labels deterministically to the first  $K$  slots. Standard targets are sampled outcomes; finance queries additionally carry the importance

weights used only by the loss in Section 9.6. Ordinary regression uses the 999-quantile head explained above.

For comparison, the older regression head defines a normalized density with explicit tails. It emits 132 values: 130 bin probabilities and two tail scales. Targets use the support median/MAD transform and its numerical fallbacks. The density consists of 128 uniform bins on  $[-5, 5]$  and two Lomax tails of shape three, with scales  $0.05 + \text{softplus}(u)$ . The left tail covers values below  $-5$ , the finite half-open bins cover  $[-5, 5)$ , and the right tail starts at 5. Likelihoods are evaluated in FP32. In original units the density is  $p_z((y - m)/s)/s$ , so the log score includes the affine Jacobian. The mean combines bin midpoints and masses with tail means  $-5 - s_L/2$  and  $5 + s_R/2$ , then inverts the support transform. A prior-only comparison keeps its chosen head fixed; the current prior screen uses the same quantile head in every arm.

At distance  $v \geq 0$  from a tail boundary, the tail density is  $3s^{-1}(1 + v/s)^{-4}$ . Its piecewise analytic CDF gives quantiles, and its first two moments are finite. No bin edges are fitted to query data. The optional realized-loss model adds a point mass at zero, as specified in Section 7.7.

If an integer target is scored as a discrete outcome, it needs a probability mass rather than a density value. A separate discrete evaluation can use the CDF to assign mass to  $[k - 1/2, k + 1/2)$  and condition on the declared integer support. For nonnegative counts, normalize by the mass above  $-1/2$  and reject negative targets. Means and quantiles then come from this PMF with a certified tail bound, or from an explicit count head. The continuous mean is not exactly the mean after rounding. The initial general-regression comparison may instead use continuous scores and point metrics, provided every prior is evaluated the same way.

For the older dense control, a MAB with width  $d$ , hidden width  $h$ , head dimension  $d_h$  and  $H$  heads contains  $4d^2 + 3dh + 4d + 2d_h + H$  parameters. The 18 frontend MABs contain 4,728,528 parameters and the row transformer contains 308,383,104. The other components are: 98,304 inducing-vector parameters; 1,024 summary-token parameters; 59,904 cell-projection weights; 192 Fourier frequencies; 4,864 type/task/role parameters; 294,912 class-label parameters; 3,456 regression-label weights; 256 input-LayerNorm parameters; 1,445,888 head weights; and 1,024 final-RMSNorm parameters. They sum to **315,021,456**. The accompanying JSON records this breakdown, and the implementation checks the total before a run. The 317M starting configuration differs through its category encoder and output heads, as detailed below.

The attention mask makes the predictor mathematically invariant to support-row order and independent of other query rows; implementation tests allow numerical tolerance. Other symmetries are only approximate. Feature grouping and rotary positions make column order matter. Hash codes can make category renaming matter, and learned class embeddings and output matrices can make class relabeling matter. Augmentation encourages robustness to those transformations without proving invariance. An exactly permutation-equivariant set encoder would be a separate comparison.

**Why use quantiles rather than the older density head?** The old finite-bin likelihood cannot distinguish some targets whose point predictions should differ. For example,  $z = 0.005$  and  $z = 0.070$  occupy the same bin, giving identical NLL and all 132 output gradients. In a noiseless constant task, support normalization gives center  $c$  and scale one, putting  $z = 0$  at the left boundary of  $[0, 0.078125)$ . The population-optimal NLL places all mass in that bin, but its predicted mean is  $c + 0.0390625$ . A larger backbone cannot remove a bias caused by the output representation itself.

One implemented alternative retains the density but increases its resolution to 1,025 equal-width bins on  $[-5, 5]$ . With the same two tails and scales, it has 1,029 outputs. The central bin has midpoint zero, and the bin width  $10/1025$  is about eight times smaller. At width 1,024, the change adds 918,528 output

weights. It removes the constant-target bias in the example and narrows the range of indistinguishable values, although values within a bin remain indistinguishable to that likelihood.

The starting configuration instead uses 999 quantiles trained with pinball loss. TabICLv2 reports strong RMSE results with this approach; LimiX-2 uses a much finer histogram [8, 22]. The network emits unconstrained values at levels  $k/1000$ ,  $k = 1, \dots, 999$ , and minimizes their mean pinball loss in support-standardized units. Inference sorts the quantiles and averages the finite grid for the point prediction. This supports quantile and point-error evaluation, but it does not provide a complete density likelihood.

The finer histogram and quantile heads are implemented; the 132-output head remains a control. A separate mean output trained with squared error could be useful, but has not been implemented. Quantiles also have a tail limitation: if  $P(Y = 10^6) = 10^{-4}$  and the target is otherwise zero, every quantile on this grid is zero even though the mean is 100. Learning the finite grid perfectly would therefore still miss that mean.

**A small learned example.** We tested whether optimization reproduces the analytical resolution problem. Identical two-input MLPs predicted noiseless targets .005 and .070, with an independent nuisance input. Across three seeds, 1,000 Adam steps, 64 training rows and 512 held-out rows, RMSE was .032537–.032538 for 128 bins, .0035745–.0035752 for 1,025 bins, and .0000409–.0002501 for 999 quantiles. Every seed is recorded in `research/results/learned_regression_head_probe.json`. Thus the old head’s defect persists after training in this example, while both alternatives distinguish the targets more accurately. This tests output resolution in a small supervised problem, not foundation-model transfer or a universal ranking of regression heads.

## 7.6 Compact persistent-cell comparator

Most parameters in the compressed model operate on rows after feature cells have been summarized. The `persistent_small` comparator instead retains cells through twelve stages before final compression. Every stage has one column ISAB and one row MAB, with cell width 256, eight attention heads and SwiGLU hidden width 1,024. Column attention uses 128 inducing summaries per feature, and row attention mixes feature cells with eight summary tokens. The final eight tokens are concatenated and projected from 2,048 to 512 dimensions for the shared prediction heads. No row-level ICL trunk follows.

Support labels enter the early cell representation. Each column stage forms inducing summaries from support alone, and queries read their cached keys and values. Row attention stays within a record, so queries neither alter support nor communicate with one another. This model shares the compressed model’s input codec, declared class vocabulary, 999-quantile target, finance adapter, loss reduction and optimizer partition. Query-KV grouping has no effect because the trunk has zero layers. The common interface retains late support-label parameters, but without a trunk their support outputs cannot affect queries.

With the candidate heads and finance adapter, `persistent_small` has 41,089,248 parameters. The smaller `persistent_tiny` supports local execution tests. This comparison model is neither a reproduction of TabPFN or EXAONE nor unrestricted cell-to-cell attention: all cross-row information still passes through inducing summaries. Delaying the permanent row bottleneck may preserve feature detail, but the inducing path imposes its own compression. Since width, depth, stored state and parameter count all differ from the 317M model, the funded experiment compares the complete systems’ use of available compute.

## 7.7 Finance inputs and the optional realized-loss head

The binary finance model uses extra inputs to describe how its context was selected. The same implementation also includes an optional realized-loss head for finance regression, specified later in this subsection; that head is not trained in the initial binary continuation. A selected context combines a representative reservoir  $R$ , a capped sample of observed events  $P$ , and nearby records  $L_c$  for route  $c$ . After deduplication, the context is  $C_c$ . Here  $N$  is the number of eligible labeled historical records,  $M$  their event count and  $M_R$  the event count in  $R$ . This  $N$  describes the history, not the number of rows in one forward pass. The selection procedure is given in Section 8.2. All counts concern information available at prediction time.

A context enriched with known fraud does not have the same class distribution as the history it represents. We therefore give the model a 14-coordinate metadata vector. Ordinary feature normalization would erase these quantities because they are constant within a context:

$$\begin{aligned} &\log(1 + N), \log(1 + M), \log(1 + |R|), \log(1 + |P|), \log(1 + |L_c|), \\ &\log(1 + |C_c|), \log(1 + M_R), \log(|R|/N), \log(|P|/M), \mathbf{1}\{M = 0\}, \\ &\text{reference kind}, \text{clip}(\text{logit } \pi_{\text{ref}}, -40, 40)/20, \log(1 + |G|), \log(1 + N_{\text{ref}}), \end{aligned}$$

where  $G$  is the searched candidate pool and  $N_{\text{ref}}$  is the size of a representative adjudicated reference cohort, or zero when no such cohort is available. Including cohort size distinguishes references that estimate the same event rate with different uncertainty. Undefined log fractions and absent reference log odds are encoded as zero.

The reference-kind field tells the model where a prevalence estimate came from. Zero means no estimate is available. One means an independent earlier cohort has complete labels. Two means the entire eligible support pool has complete labels. Kind two uses  $(M + .5)/(N + 1)$ ; kind one applies the same smoothing to the separate cohort. If selectively reviewed new records are added to an older complete cohort, the reference remains kind one. Full-amount regression zeros the positive-count, fraction, absence and reference fields, including reference cohort size. The remaining fields describe the uniform and local sample sizes.

The metadata affects both cell and row representations. A bias-free  $14 \rightarrow 128 \rightarrow 1152$  GELU MLP produces a width-128 addition before feature attention and a width-1,024 addition before the row transformer. Separate bias-free  $3 \rightarrow 128$  and  $3 \rightarrow 1024$  projections encode membership in the three context sources. Query membership is 000. These additions let the model interpret both feature values and task evidence in light of how the context was collected.

The first metadata matrix starts with standard deviation  $1/\sqrt{14}$ . The second matrix and both source projections start at zero. AdamW updates all these parameters. Their total is

$$14(128) + 128(1152) + 3(128) + 3(1024) = 152,704,$$

which brings the original control to 315,174,160 parameters before the zero-atom readout. The additional bias-free  $1024 \rightarrow 1$  projection gives the finance-capable legacy control **315,175,184 parameters**. Substituting the selected quantile head and adding the separate finer hurdle head gives the 317,116,304 total in Section 1.1.

For standard episodes, the metadata and source vectors are zero, so this additive path contributes exactly zero. This does not protect shared backbone weights from changing under joint training. Any parameter-sharing experiment must therefore check general tabular performance again.

When a representative cohort is available, the classifier predicts a correction to its class frequencies. Binary finance adds  $(\log(1 - \pi_{\text{ref}}), \log \pi_{\text{ref}})$  to the two learned logits. Multiclass finance adds log frequencies estimated with Dirichlet(.5) smoothing and aligned with the declared class permutation. The

scalar event-rate metadata separately uses Jeffreys smoothing of legitimate versus any-fraud counts, so the two estimates need not be numerically identical. Logs and likelihoods are computed in FP32.

Without a reference cohort, use zero logit offsets and loss normalization one, as in Section 9.6. A zero offset means no reference information was supplied; it is not an estimate of 50% fraud prevalence. Continuation retains the trained shared classification head. Resetting a separate residual head to zero would be a different experiment, not a reason to reset the shared head. Regression uses no class-logit offset.

**A point mass at zero for realized losses.** A realized-loss target is zero for a no-loss event and positive otherwise. A purely continuous density is therefore insufficient. Let  $f(z)$  be the base standardized severity density with CDF  $F(z)$ , let  $z = (y - m)/s$  be the support-fitted transform, and let  $z_0 = -m/s$  correspond to zero loss. A scalar output gives  $p_0 = \sigma(a)$ , the probability of exactly zero. Its learned projection starts at zero. If metadata supplies a valid event-rate reference, add  $-\text{logit}(\pi_{\text{ref}})$  before the sigmoid; otherwise add zero. A valid rare-event reference then determines the initial zero mass, rather than forcing a 50% event rate. Writing  $Z = 1 - F(z_0)$ , the distribution is

$$P(Y = 0) = p_0, \quad p(y > 0) = \frac{(1 - p_0)f((y - m)/s)}{sZ}.$$

The loss is  $-\log p_0$  for zero targets and  $-\log(1 - p_0) - \log f(z) + \log s + \log Z$  for positive targets. Negative realized losses fail target validation. Compute survival probabilities directly in log space from their components; subtracting a rounded CDF from one can lose precision.

The predictive mean is  $(1 - p_0)[m + sE(Z_{\text{base}} \mid Z_{\text{base}} > z_0)]$ , using truncated finite-bin and Lomax first moments. The point mass removes the ordinary histogram’s bias at zero: with  $m = 0, s = 1$ , assigning all mass to  $[0, .078125)$  would otherwise predict .0390625 for an identically zero target. Check normalization, zero probability, positive-tail moments and gradients before training. Choose the hurdle or continuous likelihood from the declared target type, not from whether a particular support sample happens to contain zeros.

Selection metadata does not by itself undo selection bias. The source fractions are not inverse inclusion probabilities for their union: local selection depends on reservoir-fitted preprocessing and centers, and deduplication links the sources. Nor does a selective-review flag identify population prevalence when the review process is unknown. Without representative adjudicated outcomes, probability estimates depend on assumptions learned from the prior. Results under incomplete adjudication must therefore be reported separately from calibration verified on representative outcomes.

**Scaling positive severities.** At prevalence  $10^{-4}$ , a natural 512-row reservoir often contains only zeros. A target transform fitted to that reservoir can put most positive severities in one tail, even with many histogram bins. The `positive_support` option instead fits the hurdle target transform to observed positive support outcomes. If none are available, it uses the original permitted-reservoir fallback. Feature scaling still uses the reservoir. This changes neither the affine Jacobian nor positive truncation, and query outcomes never enter the fit. It addresses the scale problem; the tail shape still needs validation on company losses.

The configuration with hash bits, 999 regression quantiles and a separate 1,025-bin hurdle density contains **317,116,304 parameters**. The finer-histogram-only alternative contains 316,093,328. Query KV sharing and the target-scaling option add no parameters.

## 8 Context selection and caching

GPU batch prediction requires both preparing a labeled context and reusing it efficiently. The dense implementation stores exact support caches. For financial histories with millions of rows, it first selects bounded contexts that fit dense attention. These are the current inference methods. Appendix B describes feature feedback, MoE and general local/global attention as separate experiments.

We first establish the cost of correct caching and query chunking, which preserve the predictor, and then profile the remaining bottlenecks. Those measurements determine whether to test approximations that discard evidence or changes that add conditional capacity. Section 11.4 gives the common timing and accuracy protocol.

### 8.1 Dense context cost and exact support caches

A new context incurs preparation work that later requests can reuse. Let  $n = n_s$ ,  $q = n_q$ , trunk width  $d = 1024$  and depth  $L = 24$ . We account for batch prediction time as

$$T_{\text{batch}} = T_{\text{codec}} + T_{\text{transfers}} + T_{\text{support prefill}} + T_{\text{query batch}} + T_{\text{output}}.$$

Model loading and kernel compilation are separate startup costs. Reusing a support cache removes prefill from later requests, but the first request for a dataset still pays for it. Benchmark evaluations and offline batches may reuse a context less often than an online service, making this distinction important to the reported cost.

Prefill contextualizes support rows against one another. Counting a multiply-add as two operations, the dense trunk’s attention products take approximately

$$4Ld(n^2 + nq)$$

forward FLOPs. Projections, FFNs, normalization and the frontend contribute additional work. Support attention is quadratic in  $n$ , whereas cached query attention is linear in  $n$  and  $q$ . Doubling support size therefore quadruples the *support-attention component* and doubles the *cached-query-attention component*; other operations determine how much total wall time changes. Each frontend stage costs roughly  $O(FnId_c + nF^2d_c)$ , with  $I = 128$ ,  $d_c = 128$ , so feature processing can remain the bottleneck on wide tables.

The visibility mask makes exact reuse possible. After computing support states, the model retains the fitted preprocessing state, each column ISAB’s support-induced summaries or projected K/V, and projected support K/V for every trunk layer. A query uses only these stored objects, its own features and fixed model weights. By induction over layers, no other query can affect its prediction. Cached/uncached and chunk-invariance tests check this property within the stated numerical tolerances.

The full-head control gives a memory reference: BF16 trunk K/V require  $2Lnd \times 2 = 98,304n$  bytes per estimator, or 0.75 GiB at 8,192 support rows, 3 GiB at 32,768 and 6 GiB at 65,536. The six frontend ISABs also store projected inducing K/V of size  $6F \cdot 128 \cdot 2 \cdot 128 \cdot 2$  bytes, which is 48 MiB at  $F = 128$ . These totals omit weights, preprocessing indices, temporary cells, outputs and allocator overhead. Separate ensemble views can multiply cache storage.

The two frontend stages cannot be prefetched independently. Stage-two inducing summaries depend on support cells after stage-one row attention has mixed the features within each row; constructing all six summaries in independent passes would require recomputation. The initial implementation keeps intermediate cells in GPU memory when they fit and tiles attention and workspace. One BF16  $n \times F \times 128$  tensor uses 1 GiB at  $n = 32768$ ,  $F = 128$ . Replay or offload could save memory, but their added computation and transfers would need separate timing.

The initial performance reference combines tiled/Flash-style exact attention, strict support/query separation, support K/V reuse and query chunks sized to available memory. FlashAttention reduces IO and score-matrix storage while retaining quadratic attention arithmetic; bit-identical results are not assumed [37]. We choose chunks from shape and memory profiles, then verify prediction agreement within tolerance. These choices mathematically preserve the specified predictor. Context removal, learned context summaries, fewer K/V heads, lower precision and early exits can change it and require accuracy comparisons.

**Implemented query-only grouped attention.** The 317M starting model reduces query-cache storage while preserving full support contextualization. Support self-attention uses all 16 heads. Query attention retains the first two KV heads, assigning each to a consecutive group of eight query heads during both training and inference. Cached tensors are cloned into independent storage; this includes one-row slices, which can otherwise retain a full backing allocation.

The resulting BF16 trunk cache takes  $12,288n$  bytes: 96 MiB at 8,192 rows, 384 MiB at 32,768 and 768 MiB at 65,536. At the finance limit of 64 routes and 7,168 rows per context, it uses 5.25 GiB per estimator, plus unchanged frontend caches and working memory. These savings concern storage. Training still holds full support activations, and query attention still has 16 query heads; eightfold reductions in FLOPs or latency do not follow. CPU tests check cache, chunk and gradient consistency. GPU profiling must establish whether the chosen kernel materializes repeated heads and how much bandwidth it saves [53, 54].

## 8.2 Bounded finance contexts and shared routing

Finance contexts combine a representative sample of the labeled pool with deliberate retention of rare positives. We freeze the eligible labeled history at a stated cutoff and denote its size by  $N$  and its verified positive count by  $M$ ;  $N$  differs from  $N_{\text{total}}$ . A simple random reservoir  $R$  supplies the data used to fit the numerical/category codec. A separate sample  $P$  is drawn uniformly without replacement from the  $M$  positives, keeping all positives when fewer than the cap are available.  $R$  represents the *eligible labeled pool*, which may differ from the transaction population under selective review.

| Stage        | Reservoir $R$ | Positive cap $P$ | Local negatives $L$ | Maximum union |
|--------------|---------------|------------------|---------------------|---------------|
| Initial      | 512           | 256              | 1,024               | 1,792         |
| Intermediate | 1,024         | 512              | 2,048               | 3,584         |
| Final        | 2,048         | 1,024            | 4,096               | 7,168         |

Table 7: Source caps, before overlap removal. Insufficient available rows reduce the actual context.

Many queries can share a bounded context through fixed routes. To build the routing index, we map observed features into 128 dimensions using a fixed seeded signed random projection of numerical rank coordinates, missing flags and feature-specific categorical indicators, normalized by  $\sqrt{p}$ . Seeded hash/sign generation applies the projection without materializing the full one-hot representation. Privileged latent coordinates are excluded.

Route centers cover both positive cases and the broader observed history. Farthest-first traversal chooses up to 16 centers from  $P$ , starting with the lowest persisted randomly assigned row ID. It then fills the remaining 64-center budget from  $R$ , maximizing distance to all centers already chosen. Row ID resolves ties, and traversal ends when every remaining distance is zero. If  $P$  is empty, all centers come from  $R$ . Support labels may define  $P$ ; query labels never affect routing.

For center  $c$ , the local set contains the nearest eligible negative rows up to the cap, ranked by squared Euclidean index distance with stable ID tie-breaking. The deployment reference prioritizes accuracy by searching **all verified negatives**, streaming candidates in chunks of 1,024 and maintaining exact top- $k$  lists.

Subsampling the search pool can remove rare negative regions. An IID region of mass  $10^{-6}$  is missing from 65,536 candidates with probability about 93.7%; that cap is therefore a latency control, not the full-history reference. Pretraining searches 65,536, 262,144 or all available negatives with probabilities .70/.20/.10. Capped pools are uniform samples without replacement, and the actual searched counts are logged. Their nearest-neighbor distributions differ deliberately: proximity within a small candidate pool does not imply proximity within the full history.

Each context is  $C_c = R \cup P \cup L_c$ . We deduplicate by row ID and retain three source-membership bits per row. Reservoir positives remain in the union even if the capped  $P$  excludes them. A query selects its nearest fixed center independently of other queries and uses that context. Grouping queries by route changes GPU execution but leaves selection unchanged.

The approximation selects neighbors of a center instead of neighbors of each query. Reservoir-only centers, smaller negative search pools and feasible per-query retrieval provide controls for measuring the cost of sharing routes. TabR and TabDPT offer precedents for retrieval on large tables [49, 24]; they do not establish this selector’s accuracy.

Regression uses sources appropriate to its target. Full-amount regression replaces  $P$  with an additional uniform labeled sample, searches all eligible labeled records for  $L_c$ , chooses centers only from the reservoir, and disables fraud-prevalence metadata and offsets. For realized-loss regression, a known positive loss defines an eventful row.

When  $N = 0$ , support is empty, codec statistics use their zero/default values, and a single dummy route has zero selection fractions. Attention with no valid keys contributes zero through the attention branch while retaining the residual path, avoiding a softmax over all negative infinity. Prediction without history is thus defined without fabricated labels or extra parameters.

### 8.3 Full-history preparation and burst inference

Bounding the context limits transformer cost, but selecting it still requires searching historical evidence. For 64 fixed centers and a 128-dimensional index, streamed negative search costs  $O(64|G|128)$  in distance work, and routing  $Q$  future queries costs  $O(64Q128)$ . Feature generation, codec fitting and projection add to this. The 10% all-history pretraining branch can therefore dominate generation time despite bounded transformer inputs.

Burst memory depends on how many contexts are cached at once. With 64 routes at the final 7,168-row cap, the two-head query caches use 5.25 GiB of trunk K/V per estimator; the full-16-head control uses 42 GiB. At 128 features, frontend inducing caches add 3 GiB to either model. If only 300 positives are eligible, the maximum context before deduplication is 6,444 rows, giving about 4.72 GiB of trunk cache for the candidate or 37.76 GiB for the control across 64 routes.

Weights, activations, codec and index storage, and workspaces are additional. Feature- or class-permutation ensembles can multiply these requirements. Reservoir rows shared across routes are contextualized differently on each route, so matching their original IDs does not make their deeper K/V shareable.

Timing covers snapshot preparation, feature and index construction, source selection, prefill for every active route, query routing, grouped inference and output assembly. We report cold total time and warm batch time separately for 512 queries, 4,096 queries and the actual production burst size.

Training uses the same accounting. A macroepisode with  $J$  active routes processes  $\sum_{c=1}^J |C_c|$  support rows; logs therefore count contexts, cells and row tokens as well as macroepisodes. Four macroepisodes may reuse an index, but support activations must be computed at current weights and remain in the training graph. Reusing the index does not justify detached or stale activations across updates. Feasible update counts require profiling this complete workload.

Successive bursts may reuse a snapshot while support, codec, reference metadata and model weights stay fixed. New evidence ends that lifetime: a newly verified positive, for example, may invalidate every route cache. Past validation data determine the update cadence by measuring freshness against latency. Earlier transactions use only labels confirmed by their cutoff; later fraud confirmations are never added retroactively. Bounded selection is the immediate approach for million-row finance histories. General local/global attention and reduced-projection GQA remain optional experiments, while MoE tests conditional FFN capacity.

## 8.4 The information limit of fixed-size compression

A finite-bit argument explains why compression must be tested on rare and local evidence. Take  $n$  distinct fixed keys with independent binary labels, and let a query request one key’s label. The labels admit  $2^n$  assignments. A cache of  $b < n$  bits has at most  $2^b$  states, so two assignments that differ on some query must share a state. Such a cache cannot perform exact lookup for every assignment.

Learned summaries can still work well when a distribution has exploitable structure. That structure must be tested, however. The prior and large-context training should include the rare and local cases the efficient architecture is meant to retain, and evaluation should check whether compression preserves their evidence.

# 9 End-to-end training protocol

Training turns sampled worlds into masked prediction problems and uses their query losses to update a shared predictor. The starting recipe uses the fixed five-way Q prior, sampled labels for ordinary tasks and population-risk weights for finance queries. A faithful reproduction of the released reference keeps its original recipe. Controlled comparisons then vary the prior, architecture, head and optimizer separately.

The unit of loss reduction and compute accounting is a logical batch of 256 macroepisodes. An ordinary task contributes one support/query problem. A finance macroepisode may contain several route groups from one history and future query sample; dividing it among routes does not create new independently weighted tasks.

## 9.1 Standard pretraining and fraud specialization

Pretraining and deployment use the same forward computation. During pretraining, query losses from 256 macroepisodes are backpropagated into shared weights. At deployment, a fixed checkpoint conditions on a new labeled context and predicts query rows without a loss calculation or weight update.

The current starting run trains on binary, multiclass and regression tasks from Q; the earlier R/P1 pilot remains a separate reference configuration. The final run will use the bank selected in Section 5. We retain the standard checkpoint and copy its weights into a synthetic binary-fraud continuation with a fresh optimizer and schedule. This produces a separate deployable specialist while preserving the

standard model. Fraud continuation is a pretraining phase, not a gradient fit repeated for each finance customer.

**(a) Standard pretraining: Q challenger and selected variants**

50% classification / 50% regression; 64M accepted episodes; 250,000 updates; 9,000 GPU-hour ceiling.

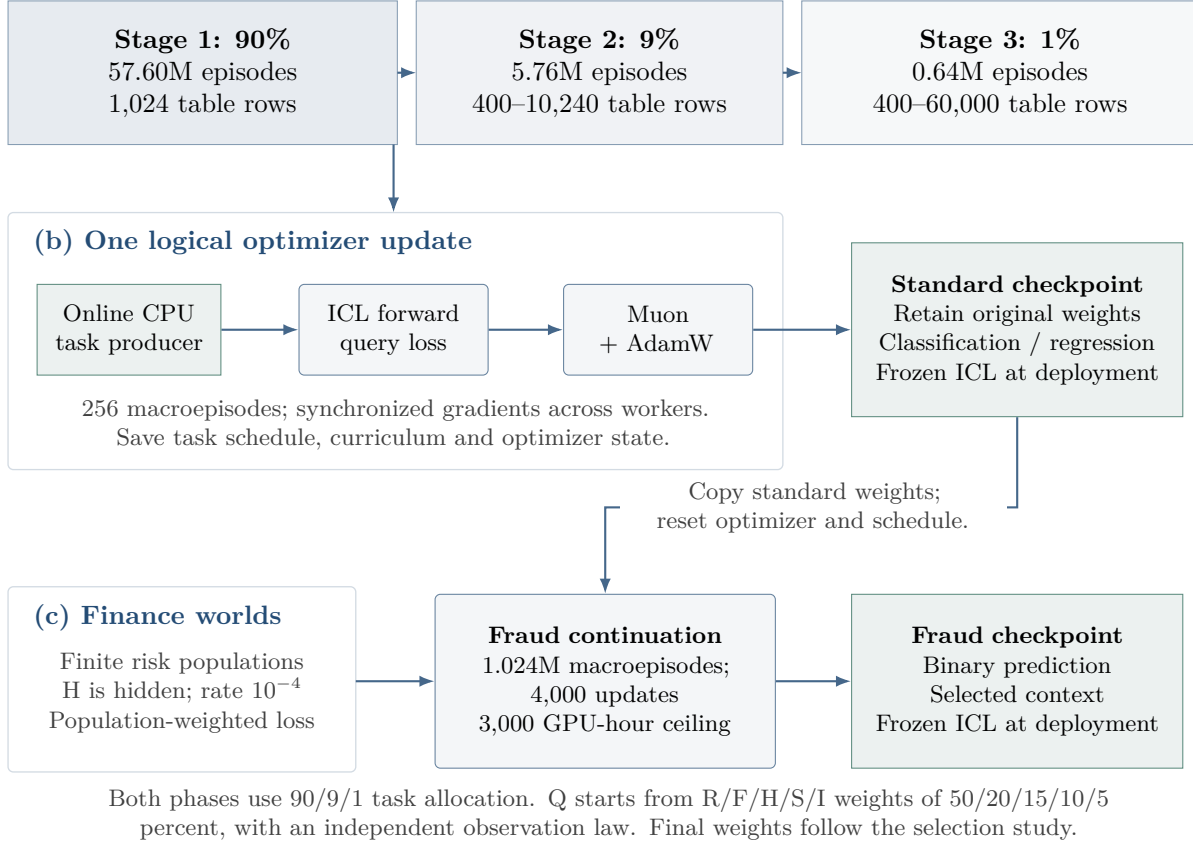


Figure 6: Pretraining and synthetic fraud specialization for the implemented Q candidate and subsequent selected variants. Each run allocates 90/9/1% of its accepted tasks to the three curriculum stages; stage boxes are not drawn in proportion to exposure. Task counts are planning targets constrained by the GPU-hour ceilings and measured throughput; the repository contains both shorter reference pilots and provisional full-volume Q configurations. Both resulting checkpoints use frozen-weight ICL at deployment. The final run uses the prior and architecture chosen by the controlled comparison.

Both phases repeat the same training cycle:

1. A deterministic schedule fixes task seeds and curriculum stages. While the model runs, the CPU producer prepares upcoming tasks; the trainer consumes them in the scheduled order.
2. The predictor sees support labels and query features, with query labels held in a separate loss object. Classification uses cross-entropy and ordinary regression uses mean pinball loss. Binary-fraud losses retain finite-population importance weights.
3. Route losses keep the original macroepisode query denominator. The trainer averages macroepisodes over the global batch, synchronizes gradients across ranks and clips their global norm at ten.
4. Muon updates registered hidden attention and FFN matrices; AdamW updates the remaining parameters. Only a valid update advances the fixed warmup/cosine schedule. Updated weights invalidate model caches.
5. Logs record consumed tasks, rejected generator attempts, total rows and cells, requested and observed

class counts, update status and allocated time. Observed class counts refer to the support/query union rather than full support-only coverage histograms. Checkpoints retain the state needed to replay the deterministic schedule.

Tasks are generated from seeds as needed, without storing the full bank. Bounded prefetch can overlap CPU generation with model computation; hardware measurements must establish how much overlap and GPU utilization it achieves. Section 10.3 uses the complete update cost to set a feasible horizon, and Section 9.8 gives optimizer equations and parameter groups.

## 9.2 Typed data boundaries and deterministic replay

Generation first defines a prediction problem, then constructs what the learner can observe. It samples task and shape, hidden mechanisms and any independent calibration data, simulates records, and assembles the episode. Rendering assigns visible values and category identities. Preprocessing fits only permitted support data, and the adapter separates model inputs from query targets and simulator diagnostics. The `Episode` interface in Section 12.1 encodes these roles.

The CPU reference uses a documented counter-based RNG or NumPy `SeedSequence`. Stable byte serialization and hashing key streams by `(run_seed, global_episode_id, recipe_version, component_id, attempt_id)`; Python’s process-randomized `hash()` cannot provide this behavior. Profile, mechanism, calibration, support, query, observation and permutation draws have separate stream IDs, so a rendering change does not perturb the underlying score or label noise.

To replay a run after interruption, checkpoints retain accepted IDs, retry counters, queue offsets and sampler state. Comparisons pair mechanism seeds, while training, calibration, synthetic validation and final audit use disjoint namespaces.

The following minimal record makes these roles explicit. Arbitrary IDs indicate equality and do not represent scalar measurements:

```
EpisodeSpec:
  recipe_version, episode_id, family, task_kind
  support_size, query_size, feature_budget, shape_bucket
  split_policy, class_vocabulary, active_modifiers
WorldParameters: mechanism graph, coefficients, latent effects, render rules
SafeInputs:
  support: features, categorical types, masks, observed_targets
  query: features, categorical types, masks
  metadata: declared task type and legal label vocabulary
LossOnly:
  query_targets, optional exact conditional label law
AuditOnly:
  latent states, true parameters, oracle predictions, rejection reasons
```

Task vocabulary and target definition are chosen before scored outcomes, so neither may encode the number of classes realized in a query set. `model.forward` receives no fields from `LossOnly` or `AuditOnly`. Debug fields are also excluded from retrieval, cache keys, scaling and attention.

## 9.3 Sampling, validity and retry semantics

The active Q sampler uses the mechanism and observation weights specified in Section 5. The legacy replacement-bank control instead samples P0/P1/P4 with probabilities 0.85/0.10/0.05, then draws mechanisms and modifiers conditional on family. Its legal conditional probabilities balance binary, mul-

ticlass and regression tasks. Finance uses its separate finite-population sampler. Logs record accepted task frequencies and loss normalization so the requested mixture can be distinguished from realized exposure.

The shared head takes a label vocabulary declared before outcomes and allows classes to be absent from support. Legacy rules requiring class coverage do not apply by default. In the replacement-bank generator, an exception terminates generation immediately, including through the process producer. Native R retains the released filtering and retry rules with explicit counters and an abort bound.

Invalid keys, nonfinite latent propagation, impossible shapes and simulator-budget violations are validity failures. Query imbalance and high query loss are not. A fully masked target cannot be replaced by zeros. The replacement-bank control accepts tasks on validity alone, whereas native R preserves the released predictability filter.

The proposed queue adapter below allows up to eight attempts for a request, retaining its original family and assigning new streams to each attempt. After exhaustion it records a dropped request and replenishes that quota later. This is not the current training loop. It neither substitutes an easy linear task nor hides repeated failures.

The replacement control’s `validity_only` rule and R’s native teacher filter therefore imply different training distributions. The optional adaptive policy in Appendix B.5 would update future mixture weights from independent calibration worlds, without adding an easy/medium/hard acceptance mixture to the base sampler.

```
for request_id in deterministic_request_stream:
    spec = sample_family_shape_task(request_id)
    for attempt in range(8):
        params = sample_parameters(spec, stream("mechanism", attempt))
        cal = independent_calibration(params, stream("calibration", attempt))
        freeze_mechanism_scales_and_task_thresholds(params, cal)
        world = simulate_world(params, stream("world", attempt))
        roles = choose_support_query_roles(spec, world)
        safe = compile_feature_views(world, roles, spec)
        loss = compile_targets_in_separate_object(world, roles, spec)
        if not validity_checks(safe, loss, params):
            log_invalid_request(spec, attempt)
            continue
        safe = support_only_preprocess(safe)
        yield safe, loss, separate_audit_record(spec, params, cal)
        break
    else:
        log_dropped_quota_request(spec)
```

This pseudocode fixes the computation order; the information boundary must also be enforced within its steps. A simulator may create a full joint table before extracting inputs, but realized scored query outcomes cannot determine feature selection, eligibility or task thresholds.

## 9.4 Joint task mixture and finance-world reuse

The historical replacement-bank sharing experiment assigns 80% of emitted macroepisodes to ordinary tasks and 20% to finance. Its ordinary-task family probabilities are .85/.10/.05 for P0/P1/P4, while finance chooses binary, multiclass and regression tasks with probabilities .70/.10/.20. To maintain one-third of the total marginal probability for each head, the ordinary component uses

(.2416666667, .3916666667, .3666666667).

as its head probabilities. Since P4 draws binary/regression tasks with probabilities .20/.80, P0/P1 use head probabilities (.2438596491, .4122807018, .3438596491). This adjustment applies only to the joint experiment; pure-standard training retains its balanced marginal task distribution. Reports include raw profile frequencies, gradient shares and compute shares, because 20% of macroepisodes need not mean 20% of contexts or GPU-hours.

Profile membership is drawn per emitted macroepisode. Drawing an 80/20 mixture of worlds and reusing finance worlds four times would instead give 50% finance episodes. Each finance world, history and index is used for four macroepisodes, no more than once in a logical optimizer batch. Model activations are recomputed at current weights.

Each macroepisode draws 128 query IDs using Section 9.6’s proposal, routes them from their features and evaluates every nonempty route group. First-stage finance widths are 16, 32, 64, 128 with probabilities .45/.25/.15/.15; intermediate widths are 64, 128, 256, 512 with probabilities .25/.35/.25/.15; final widths are 128, 256, 512 with probabilities .25/.375/.375. The stage cell cap applies to each routed forward at its actual support/query counts. Profiling totals all routes within a macroepisode.

## 9.5 Likelihoods, preprocessing and logical-batch reduction

Input and target transforms use only permitted support information, as specified in Sections 7.3 and 7.5. The finance codec fits the selected reservoir from Section 8.2. Query features do not update these statistics under the inductive protocol. Section 7.4’s masks keep support independent of queries and each query independent of the others. Preprocessing and batching preserve the same separation of model inputs, loss targets and hidden audit state.

The original control uses classification cross-entropy and histogram/Lomax NLL for ordinary regression. The starting candidate keeps cross-entropy and replaces regression NLL with the averaged 999-quantile pinball score. Realized finance loss uses Section 7.7’s zero-atom likelihood.

For ordinary tasks, we average valid query losses within each macroepisode and then average macroepisodes within the logical batch. Head multipliers start at one. Finance keeps the original query denominator and the world multiplier defined below; route groups divide execution and do not receive separate episode averages.

Funded prior comparisons keep the compressed architecture, preprocessing, heads and optimizer fixed. Reproducing the complete open reference with its own training recipe is a separate experiment. We monitor per-head gradient norms and validation quality, since proper losses in different units are not directly comparable measures of difficulty. Exact conditional targets, when eligible under Appendix A, affect loss estimation only and leave forward inputs unchanged.

Packing and accumulation must preserve episode isolation: changing query labels or another packed episode cannot change the relevant forward prediction. We sum macroepisode losses and divide by the global accepted macroepisode count. Taking an unweighted average of unequal microbatch means would change task weights.

## 9.6 Population-risk estimation for rare-event queries

Balanced query sampling makes rare outcomes appear more often during training, while importance weights recover the risk over the finite population. Let the future population  $Q$  have size  $n_Q$  and class counts  $m_y$ . The proposal first samples a class uniformly from those present, with probability  $r_y$ , then samples a uniform finite ID in that class. Training allows replacement. For binary populations containing both classes,  $r_0 = r_1 = 1/2$ ; absent classes have zero proposal probability. Sampling precedes

feature-based routing. The loss estimator is

$$\hat{\mathcal{L}}(H) = \frac{w(H)}{q} \sum_{i=1}^q \frac{m_{y_i}/n_Q}{r_{y_i}} \ell(f_{\theta}(x_i; C_{c(x_i)}, H), y_i) \quad (6)$$

Conditional on the world and history, this is an unbiased estimate of  $w(H)$  times finite-population query risk: the proposal probability cancels within each class to recover its population contribution. It is a weighted mean, not a self-normalized ratio. Every route group’s loss sum retains the original denominator  $q$ ; equally averaging route means would change the objective. Finite-population class fractions stay in the loss object and never enter model inputs or metadata.

Amount regression samples queries uniformly with unit weights. Realized-loss regression applies the same importance construction to zero and positive outcomes and scores them with the hurdle likelihood. Finance retains sampled hard labels. Exact conditional supervision would need the outcome law given observed features and masks; the law conditional only on a hidden risk cell is insufficient.

The starting specialist sets  $w(H) = 1$  for every world. A separate entropy-normalization control uses the following weight for finance classification and event-loss regression when a reference is available:

$$w(H) = \frac{1}{\max\{h_2(\pi_{\text{ref}}), 10^{-4}\}}, \quad h_2(p) = -p \log p - (1 - p) \log(1 - p).$$

For multiclass tasks,  $\pi_{\text{ref}}$  is the scalar Jeffreys-smoothed any-fraud reference; where this reference weighting does not apply,  $w = 1$ . The floor bounds amplification at 10,000. Because the multiplier depends only on permitted learner information, it leaves the ideal conditional optimum of a proper loss unchanged, although finite capacity may be allocated differently. Using *hidden true* world entropy would reweight posterior worlds and could change the probability target.

The normalized and unit-weight comparisons receive equal compute and record clipping frequency at the norm-10 limit. The normalization cap remains an optimization choice to tune. Balanced proposals provide more positive query draws, but do not create independent historical positives or solve every rare-event gradient problem. Focal loss and uncorrected class weights are alternative objectives, not the default probability-estimation loss.

Whether probabilities need postprocessing depends on what selection the classifier has learned. If  $r(x)$  is the posterior among selected examples and retention probabilities  $a_y(x) > 0$  are known, Bayes’ rule gives

$$\text{logit } p_{\text{pop}}(x) = \text{logit } r(x) + \log \frac{a_0(x)}{a_1(x)}. \quad (7)$$

A scalar offset is justified by class-only selection, or an equivalent constant retention ratio, and only when the predictions have that selected-posterior meaning. Local retrieval and selective label availability generally violate these conditions [48]. Here ICL already targets population queries conditional on enriched support, so applying another case-control correction would usually be inappropriate.

A three-state diagnostic recovered population probabilities to floating-point precision with the feature-dependent ratio. The scalar class correction had maximum absolute error .439, and the weighted-query diagnostic reproduced finite-population BCE exactly. These checks verify the identities rather than a trained fraud model’s quality.

The effect of normalization varies with the current prediction. At  $\pi_{\text{ref}} = 10^{-4}$ , the entropy multiplier is 979.4. Under the balanced binary proposal, weighted logit derivatives are  $2(1 - \pi)p$  for negatives and  $2\pi(p - 1)$  for positives. With calibrated  $p = \pi$ , multiplication by 979.4 gives roughly +0.196 and −0.196. At  $p = 0.5$ , however, the negative derivative is roughly 979.3. The multiplier alone therefore does not establish instability; initialization from a valid reference logit matters.

Normalization changes the relative contribution of complete-reference and incomplete-adjudication worlds to finite capacity and global clipping. The comparison records gradient norms before clipping and cosine similarities by standard/finance task, head, reference availability and positive count. Both arms keep the importance ratio. We examine these quantities before choosing Muon versus AdamW because task-gradient allocation interacts with optimization.

The probe in `research/probes/finance_gradient_probe.py` measures actual autograd derivatives at initialization without optimizer updates. `research/results/finance_gradient_probe.json` stores seeds, configurations, paired inputs, source hashes and all cases. It does not measure learning curves or production fraud performance.

## 9.7 Gradient scale and fraud specialization

With  $p = \pi = 10^{-4}$ , each balanced-proposal weighted binary logit derivative has magnitude  $2\pi(1 - \pi) \simeq 2 \cdot 10^{-4}$ . Balanced binary CE at  $p = .5$  instead gives magnitude .5, a factor of 2,500 before profile mixing. These gradients are added at shared parameters before AdamW or Muon acts, so schedule frequency alone does not determine finance’s share of parameter updates.

A tiny-model probe at fixed initialization illustrates the scale difference. A finance world with a complete historical reference produced trunk gradient norm  $7.82 \cdot 10^{-6}$  under unit weighting, versus .00954 for a P1 binary task. Without that reference, mean predicted probability was about .496 and the gradient norm was .105. No optimizer steps were taken: the larger norm reflects initial base-rate error, not faster measured learning. This motivates an explicit allocation comparison. More query draws reduce sampling noise, whereas removing importance weights changes the posterior target.

Separate fraud continuation isolates finance updates from standard-task gradients and retains the standard checkpoint. It keeps the architecture, uses binary risk-partition worlds with hidden H, and starts with unit world normalization and unchanged importance weights. Peak learning rates fall tenfold to  $8 \cdot 10^{-5}$  for Muon and  $3 \cdot 10^{-5}$  for AdamW. Unit versus entropy normalization remains a controlled comparison.

`-initialize-from` copies weights into a separately accounted run, creates a fresh optimizer and schedule, and records the parent checkpoint hash. `-resume` instead restores an existing run exactly.

## 9.8 Muon and AdamW: partition, numerics and schedule

The optimizer follows the [original Muon implementation](#)’s division between hidden matrices and embeddings/outputs updated by a conventional optimizer. [Moonlight](#) motivates shape scaling and decoupled decay. The [PyTorch Muon specification](#) distinguishes original scaling from `match_rms_adamw`, so a learning rate must be interpreted with its convention. These references guide the starting configuration without establishing an optimum for this model or AMD implementation.

Muon updates each MAB’s four attention projections and three SwiGLU matrices. Orthogonalization treats Q, K and V separately even when their forward kernel is fused. AdamW handles input, label and output projections, embeddings, inducing and summary tokens, norms, frequencies and scalar temperatures.

The starting Muon settings are peak base learning rate  $8 \cdot 10^{-4}$ , momentum 0.95 with Nesterov momentum, five Newton–Schulz iterations using coefficients (3.4445,  $-4.775$ , 2.0315), epsilon  $10^{-7}$  and zero decay. The original control uses decay 0.01. After gradient synchronization, momentum and update matrices are  $B \leftarrow 0.95B + g$  and  $G \leftarrow g + 0.95B$ .

We transpose  $G$  when it has more rows than columns and divide it by its Frobenius norm plus epsilon.

Five iterations compute  $A = XX^\top$  and  $X \leftarrow aX + (bA + cA^2)X$ , followed by transposing back if needed. Decay uses the base learning rate; the matrix update uses scale  $0.2\eta\sqrt{\max(m, n)}$ . Orthogonalization initially uses FP32. BF16 would require separate numerical validation as a speed optimization.

AdamW uses peak learning rate  $3 \cdot 10^{-4}$ , betas (0.9, 0.95), epsilon  $10^{-8}$  and standard bias correction. Projection and head matrices have decay 0.01 in the control and zero in the candidate; embeddings, norms, frequencies and scalars always have zero decay. Both optimizers receive synchronized gradients clipped to global norm 10. Matrix products use BF16, while master weights, optimizer states, reductions, losses, softmax and normalization accumulators use FP32.

Model and optimizer states are replicated, with gradients averaged after the complete logical batch. Parameter-activity flags are synchronized first. If a parameter is used on another rank, a locally inactive branch contributes zero; a globally unused parameter receives no state update or decay. Muon then orthogonalizes each complete original matrix. Applying it to arbitrary tensor-sharding slices would change the optimizer.

Local microbatch size and accumulation can change to fit memory, but each logical batch still contains 256 macroepisodes with the specified finance weights and route denominators. Learning rates increase linearly for the first  $\lceil 0.02T \rceil$  valid updates and then decay cosinusoidally to 10% of peak at update  $T$ .

Measured stage-specific update costs and the run budget determine  $T$  once, before launch; the manifest fixes it together with curriculum boundaries. Calibration and pilot work count toward the 50,000 GPU-hour ceiling. A nonfinite gradient skips the entire update and schedule increment, and three consecutive skips abort with diagnostics. EMA starts disabled. The earlier all-AdamW comparison at peak rates  $10^{-4}, 3 \cdot 10^{-4}, 8 \cdot 10^{-4}$  is deferred under the current allocation. Separately funded reproductions retain the reference systems' published recipes.

## 9.9 Shape and class curriculum

Each declared run horizon uses one warmup/cosine schedule as defined in Section 9.8. The current Q run uses the native 90/9/1 episode allocation in Section 10.3. Measured costs may shorten it, but  $T$  and stage boundaries are fixed before launch. Stage transitions restart neither optimizer nor learning rate. The charged GPU-hour cap remains binding if actual costs exceed estimates.

Every endpoint in a compute-scaling comparison must finish its own schedule or receive a matched terminal cooldown. An intermediate checkpoint from a much longer unfinished decay is not equivalent. Recovery checkpoints save model, optimizer, sampler and compute state at least hourly and at stage boundaries, and restart tests verify episode IDs and loss masks.

The remainder of this shape schedule belongs to the *legacy replacement-bank control*, not the native Q curriculum. It spends 60% of training GPU-hours on first-stage shapes, 25% on medium contexts and 15% on long or wide cases. These fractions allocate measured cost, not episodes or prior families. Stage one samples the three 32–2,048 support buckets equally. Stage two uses short buckets for 40% of episodes and discrete log-uniform 2,049–8,192 support for 60%. Subject to memory and development results, stage three assigns 30% short, 30% medium, 25% long (discrete log-uniform 8,193–32,768) and 15% wide (discrete log-uniform 1,024–4,096) support. Queries remain capped at 128 unless a separate supervision-density comparison supports a change.

In this control, short-bucket replay keeps the base width distribution. Medium widths {64, 128, 256, 512} have probabilities .25/.35/.25/.15; long widths {128, 256, 512} have .25/.375/.375; wide widths {512, 1024} have .35/.65. A separate wide bucket is necessary because the final cell cap excludes width 1,024 at  $n_s \geq 8193$ . Width follows the initially drawn bucket even if a class-dependent floor later raises support size.

The control’s stage cell caps are  $2^{19}, 2^{21}, 2^{23}$ . After drawing support size and applying its class floor, we renormalize width probabilities over values satisfying  $(n_s + n_q)p \leq C_{\text{cell}}$  and draw from that conditional distribution. This explicitly changes the shape prior rather than deleting columns after sampling. Logs retain realized widths, contexts and padding; any lower cap must be accompanied by the resulting distribution and frequencies.

If a long stage fails its accuracy or throughput criterion, its remaining allocation goes to the best confirmed shape mixture, with the revised schedule recorded before final evaluation. Family probabilities stay fixed within each stage of the first controlled run. Any curriculum change is a separate ablation within the same budget.

**Legacy-control many-class curriculum.** This class schedule also belongs to the replacement-bank control. The native reference-derived Q schedule keeps its class-budget law and stated envelope extensions. The control uses 3–10 classes in stage one. Stage two chooses intervals  $[3, 10]$ ,  $[11, 32]$  and  $[33, 128]$  with probabilities .70/.20/.10 and samples  $K$  uniformly within the chosen interval. Stage three adds  $[129, 256]$  and uses probabilities .70/.20/.08/.02.

After enforcing  $n_s \geq 16K$ , the control recomputes its bounded query count and logs the altered context histogram. From initialization, the shared head supports 256 classes, masking inactive logits from the declared or support vocabulary. Tasks above this limit are reported as unsupported. Supporting a dynamic class head would require an architecture extension absent from this recipe.

## 10 Model size and allocation of 50,000 GPU-hours

The project has eight nodes with eight MI355X GPUs each, or 64 GPUs in total. Its 50,000 GPU-hour limit equals 781.25 hours—32.552 days—of full-cluster use. Research, failures, model selection, final training and evaluation all share this allowance. AMD specifies 288 GB HBM per device and approximately 2.5 PFLOP/s of dense BF16 matrix peak performance [29]. Memory is distributed across devices, and actual throughput must be measured for this workload.

### 10.1 Model-size recommendation

The size study begins with the implemented 317M model, including reduced query caches and revised representations. Comparators include the 315M compressed-row control, the released TabICLv2 backbone and the 41M `persistent_small` cell model. EXAONE’s roughly 21M CAST, which retains feature detail through depth, and LimiX-2’s persistent feature/target architecture provide relevant alternatives [11, 22].

Equal-cost system comparisons ask which design performs best within this budget. To study capacity within a design, we will also compare two sizes of the selected architecture family. MoE and a 1B model have no initial funding.

Within a family, size variants keep frontend and input/output semantics fixed and vary trunk width and depth. The available width/layer pairs are (512, 12), (768, 12), (1024, 24), (1280, 24) and (1536, 32). Head count is width/64, and SwiGLU width is  $64 \lceil (11d/4)/64 \rceil$ . For widths other than 1,024, a projection maps the eight concatenated cell summaries to trunk width; trunk label, task and head dimensions change accordingly.

Measured parameter counts define these variants; the pairs are not exact 50/100/315/500/1,000M models. The grid makes later comparisons possible without committing to every size. Only two sizes receive funding after the architecture family is chosen.

A tabular model can learn how to infer from labeled context without storing each task’s facts in its weights. Its cost still includes long contexts, repeated cell operations, ensembles, preprocessing and retrieval, so this observation does not locate capacity saturation. LimiX-2 reports gains from 12.5M to 406.2M under its recipe, while some depth increases in Mitra and TabICLv2 showed weaker or saturating gains [22, 30, 8]. Architecture, prior, optimization and data scale jointly determine the result. Varying size alone does not establish the compute-optimal frontier.

## 10.2 Training volume and computational units

Chinchilla inferred a language-model size/training-token balance from many runs; its 70B/1.4T example gives 20 tokens per parameter [31]. That empirical ratio is not a universal law, even across language-model objectives. PFN accounting must distinguish raw cells, compressed rows, supervised targets and independent synthetic worlds, each of which measures a different kind of exposure.

For every optimizer update, compute accounting records:

1. the number of accepted episodes and distinct world/mechanism IDs;
2. support and query rows, including the query targets actually scored;
3. observed and padded cells, and feature-cell activations at each stage;
4. row tokens entering the ICL trunk, under an explicit token definition;
5. measured GPU-seconds for the whole update, generation time and rejection cost;
6. estimated training FLOPs for the encoder, attention, FFNs, heads and rematerialization.

Layerwise work belongs in FLOP estimates, but input-token counts should not be multiplied by depth; doing so would make them incomparable with ordinary language-model tokens. Logs also distinguish repeated views, repeated epochs and multiple episodes from one world.

Ordinary tasks use an independent world per episode by default. Finance uses each finite world for four macroepisodes, as in Section 6, while separately counting route forwards and repeated support. A throughput experiment could likewise draw four independent support/query sets per calibrated world, permitting at most one use in each logical batch and none across training/validation namespaces. Equal reuse preserves the marginal prior but alters gradient correlation and the count of independent mechanisms, so it must be reported as a separate comparison.

An episode with 1,024 support rows, 128 queries and 64 features contains 1,152 row tokens, 73,728 raw cells and 128 supervised targets. None of these is an automatic count of independent examples. A separate 4,096-row calibration sample also costs generation time even though it contributes no model-training tokens; sampler profiles include that cost.

## 10.3 Planned volume and reference-pilot exposure

The provisional Q configurations and earlier reference-pilot plan target **64,000,000 accepted standard tasks** and then **1,024,000 finance macroepisodes** for a separate specialist. A task means a support/query problem, not a row or cell. These are planning targets rather than scaling-law optima. The 9,000 and 3,000 GPU-hour limits remain binding, and source-specific counts below describe only the historical R/P1 pilot.

Deterministic seeds and ordered prefetch generate tasks on demand without materializing the complete bank. Validation uses fixed, separate namespaces whose worlds never enter training.

At global batch 256, the standard target requires 250,000 updates. Both native schedules allocate 90/9/1% of accepted tasks to stages one, two and three, unlike the legacy replacement control’s 60/25/15 shares of GPU-hours. Native shapes follow Appendix A, with the declared envelope extensions in later

stages. The table below gives the R/P1 pilot’s expected source counts; Q uses the independently specified five-way mechanism mixture.

| Stage | Updates | Accepted tasks | Expected R | Expected P1 |
|-------|---------|----------------|------------|-------------|
| 1     | 225,000 | 57,600,000     | 54,909,091 | 2,690,909   |
| 2     | 22,500  | 5,760,000      | 5,488,596  | 271,404     |
| 3     | 2,500   | 640,000        | 610,801    | 29,199      |
| Total | 250,000 | 64,000,000     | 61,008,488 | 2,991,512   |

Table 8: Standard exposure targets. Source expectations account for requested-width eligibility and are rounded to whole tasks; they are not fixed quotas.

In the R/P1 pilot, width eligibility changes realized source proportions. If stage  $s$  has  $E_s$  requested-width slots with at least eight features, expected P1 exposure is exactly  $0.05 \sum_s E_s$ , with R filling the rest. Before eligibility, the five-percent branch would request 3.2 million P1 tasks. Integrating rounded-uniform width sampling and long-row caps gives about 2.992 million accepted P1 tasks and 61.008 million R tasks. For ceiling  $F_{\max}$ ,  $P(F \geq 8) = (F_{\max} - 7.5)/(F_{\max} - 1)$ ; stage three averages this over log-uniform row sizes and their width caps.

These expectations assume the specified envelopes, no diagnostic shape overrides and no failed, skipped or replayed updates. They are neither exact finite-run quotas nor hard bounds. P4 contributes no reference-pilot tasks and remains an unfunded count/amount control. R’s eight function types occur inside graphs rather than defining eight table families. Equal allocations to MLPs, trees, products and other node functions would change the prior, so the pilot retains the pinned graph/function sampling laws and audits their consequences.

Native package draws choose classification and regression equally, giving 32 million tasks each in expectation. The native integer class-budget law, modified on extension slots, does not make binary and multiclass equally frequent. Under the proposed envelopes, expected counts are approximately 3.503 million binary, 28.497 million multiclass and 32 million regression tasks. Native generation may still produce fewer observed classes than requested.

A 20% extension rate yields 1,152,000 stage-two slots and 128,000 stage-three slots in expectation. In the pilot these are subsets of R/P1 draws, not extra episodes. Accounting separates requested class budgets from observed classes, requested from accepted widths, and records eligibility and every rejection reason.

The finance target takes 4,000 updates: 3,600/360/40 by stage, or 921,600/92,160/10,240 macroepisodes. Four uses per finite world give 230,400/23,040/2,560 worlds, totaling 256,000, provided no attempts are skipped or replayed and reuse groups stay within stages.

With 128 query targets per use, the finance target contains 131,072,000 nominal query uses. Repeated positive IDs remain repeated observations of finite outcomes, and additional route forwards do not increase macroepisode counts. Lazily representing a population of several million rows likewise does not put all of those rows into transformer context.

On 64 GPUs, 9,000 standard GPU-hours provide 140.625 wall hours and 3,000 finance GPU-hours provide 46.875. Ignoring setup and checkpoint reserves, those ceilings require aggregate throughput of 126.420 standard tasks/s and 6.068 finance macroepisodes/s. At batch 256, the maximum weighted mean update times are 2.025 and 42.1875 seconds.

For measured stage times  $t_s$ , the appropriate mean is  $0.90t_1 + 0.09t_2 + 0.01t_3$ , including generation, every route and synchronization. These numbers state what the target would require, not expected MI355X performance.

`scripts/plan_prior_volume.py` writes the unmeasured targets to `research/results/prior_volume_plan.json`. Given timing measurements in JSON, it calculates target cost and the largest smaller horizon fitting each phase after reserved overhead. It rounds standard horizons to 100-update blocks and finance horizons to 400-update blocks, preserving 90/9/1 stage shares and four-use world boundaries.

The measured horizon and its cosine schedule are fixed together; stopping an unfinished longer schedule at the budget limit is not equivalent. Until timings are available, these configurations are capped provisional runs. `docs/prior_volume.md` gives the measurement schema and arithmetic.

```
python scripts/plan_prior_volume.py \
  --output runs/provisional_volume.json
python scripts/plan_prior_volume.py \
  --measurements runs/actual_stage_timings.json \
  --output runs/measured_volume.json
```

Published counts put this volume in context. Released TabICLv2 scripts imply 35.2 million table presentations per classifier or regressor, while Kumo reports roughly 35/71/137 million tables across sizes [7, 12]. These suggest an order of magnitude, but do not establish equal independence, equal compute or the optimal volume here. The stopping point requires measured learning curves and cost.

## 10.4 Compute model and profiling protocol

For a full row transformer, a first approximation to forward-plus-backward cost uses dense projection parameters  $P$ , depth  $L$ , width  $d$  and processed rows  $r$ :

$$C_{\text{row}} \approx 6Pr + 12Ldr^2. \quad (8)$$

The second term accounts for attention products and can be large. Support-only keys and values reduce pairs to  $n_s^2 + n_s n_q$  only if the implementation avoids query K/V work; a dense masked kernel may still evaluate larger padded products. The asymmetric path also makes  $6Pr$  an approximation.

Frontend ISABs, within-row feature attention, categorical gathers, heads, optimizer steps and activation recomputation add work beyond this estimate. We therefore use kernel profiles and measured runtime alongside FLOP accounting.

In a dual-axis cell architecture, projection positions scale approximately as  $rF$  per block and attention pairs as  $rF^2 + Fr^2$ , under that model’s widths and masks. ICL-315 instead combines narrow cell operations with a wide row trunk. Multiplying raw cells by all 315M parameters would assume every parameter processes every cell; counting only row tokens would omit the frontend.

A fixed batch-times-row count does not fix computation. Doubling rows and halving batch size approximately preserves projection work but doubles full row-attention work. Flash-style attention reduces storage and memory traffic while retaining quadratic arithmetic. Thus a curriculum with “constant tokens per step” may still have changing update times.

We will profile each representative shape mixture after 200 warmup steps, timing at least 1,000 steps or a ten-minute stable interval, whichever is longer. Timing includes generation, queues, backward passes, optimization, communication, calibration and periodic checkpoints. One-GPU, one-eight-GPU-node and intended multi-node runs record allocated devices, accepted episodes and all counters.

A small model may spend much of a 64-GPU update communicating. If profiles show that limit, independent research runs on separate nodes may use the project budget better.

Let  $c$  be measured allocated GPU-seconds per accepted episode. A budget of  $H$  GPU-hours then

permits

$$E = \left\lfloor \frac{3600H}{c} \right\rfloor. \quad (9)$$

The table applies the conditional 12,000 GPU-hour final-training allowance to the fixed example of 1,152 rows, 64 features and 128 queries. It is a **sensitivity calculation only**; none of the rates is an MI355X measurement. The range illustrates why a token ratio cannot determine affordable

| Assumed GPU-s/episode | Episodes | Row tokens | Query targets | Row tokens/300M params |
|-----------------------|----------|------------|---------------|------------------------|
| 0.1                   | 432M     | 497.664B   | 55.296B       | 1,658.88               |
| 1                     | 43.2M    | 49.7664B   | 5.5296B       | 165.888                |
| 10                    | 4.32M    | 4.97664B   | 0.55296B      | 16.5888                |

Table 9: Illustrative throughput sensitivity, not throughput forecasts or recommended data budgets. Raw cell counts are 64 times row counts in this example.

volume without hardware measurements. The calculator recomputes volume from real counters, treating different candidate plans as alternatives within the same allowance.

## 10.5 Model size and training-volume experiments

The size/volume study fixes prior, architecture family, optimizer convention, task/shape schedule and validation distribution. Of its 10,000 GPU-hours, 2,000 first compare two sizes with two seeds at 500 hours per size/seed. Two distinct finalist packages then receive 3,000 hours each, leaving two 1,000-hour runs to confirm another seed or resolve an uncertain ranking.

Each package pays for all of its classification and regression checkpoints and any specialist. Each endpoint completes its schedule and intended short/medium/long curriculum, after measured stage costs establish feasibility. If architecture remains unresolved, the endpoints compare two systems rather than a pure size effect.

A 3,000-hour comparison is still fourfold smaller than the full 12,000-hour final package. Its learning curve can inform selection and reallocation, but cannot precisely identify a compute optimum. Equal-FLOP diagnostics supplement the primary equal-cost comparison because preprocessing and kernel efficiency differ across systems.

Every completed endpoint is evaluated on independent synthetic validation and the fixed real development panel, with binary, multiclass and regression results separate. We fit a learning-curve surface only when the measured points support a stable trend. One candidate diagnostic is

$$L(P, D) = L_\infty + AP^{-a} + BD^{-b}, \quad A, B, a, b > 0, \quad (10)$$

Here  $D$  has a fixed documented exposure definition and all points use the same shape mixture. Under the idealized cost  $C = 6PD$ , an interior optimum obeys

$$aAP^{-a} = bBD^{-b}, \quad P^* = \left[ \frac{aA}{bB} (C/6)^b \right]^{1/(a+b)}. \quad (11)$$

Even this simplified optimum depends on estimated coefficients and exponents, not a universal ratio of 20. The hybrid architecture has a different cost surface, so feasible allocations need measured costs and numerical comparison. Changing the prior requires re-estimating the synthetic-loss exponent, and likelihood alone does not establish predictive accuracy.

We estimate uncertainty by resampling independent dataset lineages and training seeds, then test sensitivity to removing sizes or data endpoints. If extrapolation is unstable, selection follows the observed performance–cost frontier. Intermediate continuation decisions can reallocate compute when learning curves cross.

Predictive performance takes priority within the total budget. A larger model may be preferred if it improves final scores despite higher prediction cost. Inference comparisons include single passes, fixed ICL ensembles and permitted retrieval, all with frozen weights. Per-task gradient fine-tuning is a separately identified adaptation track.

## 10.6 Allocation of the project budget

| GPU-hours | Purpose                                  | Scope and conditions                                                                                                              |
|-----------|------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| 5,000     | Systems, reference and evaluation checks | 2,500 reference inference/reproduction; 1,500 backend/profiling/recovery; 1,000 benchmark integration and cost forecasts.         |
| 15,000    | Controlled screening experiments         | Current breakdown in Section 5; it replaces the earlier allocation in Appendix C.                                                 |
| 10,000    | Size and longer finalist comparisons     | Two sizes/two seeds at 500 hours; two 3,000-hour bridges; two 1,000-hour confirmations or ranking-resolution runs.                |
| 12,000    | Conditional final training               | Total across selected checkpoints after longer-run and confirmation comparisons. A shared model is one option, not a requirement. |
| 5,000     | Locked evaluation                        | Estimate the cost of the official roster and finance periods before launch, including failed runs and ensembles.                  |
| 3,000     | Uncommitted reserve                      | For failed runs, unresolved rankings or an evidenced need for specialization; record the allocation before use.                   |
| 50,000    | Total ceiling                            | 32.552 full-cluster days at 64 allocated devices.                                                                                 |

Section 5 defines the funded screen. Appendix C preserves the preceding plan and controls for reproducibility, without adding to the allocation.

**Conditions shared by the current and earlier allocations.** A new final run receives a fixed new horizon. Changing a recipe or horizon during continuation creates a separately accounted experiment; exact-configuration recovery continues the original run. Failed reproduction, negative transfer or unresolved finalist comparisons rule out automatic use of final-training funds. Unassigned reserve stays unspent.

The trainer replicates model and optimizer states and synchronizes gradients after each logical batch. At the starting size, optimizer states alone are unlikely to require aggressive parameter sharding; long-context activations may consume more memory. Activation checkpointing will be used where measurements support it. Muon continues to operate on complete matrices because orthogonalizing shards independently changes the method.

Large HBM capacity cannot remove CPU-generation bottlenecks or ROCm attention limitations. Parameter count alone therefore establishes neither feasible batch size nor attainable throughput.

## 10.7 Compute accounting tool

```
python research/probes/compute_plan.py --gpu-hours 50000 --gpus 64 \
  --output research/results/compute_plan_50000.json
python research/probes/compute_plan.py --measurements measured_profiles.csv \
  --output research/results/measured_compute_plan.json
python -m unittest discover -s tests -p test_compute_plan.py -v
```

Each measurement CSV row identifies its run by `config_id`, `params` and `allocated_gpus`, with elapsed time in `wall_seconds`. Totals for the timed interval include `accepted_episodes`, `rows`, `cells`, `query_targets` and `core_tokens`; `core_token_definition` states the token convention. The script rejects missing or zero measurements and does not infer an optimum from throughput alone.

Four local tests passed for budget conservation, allocated-GPU time, quadratic attention cost and invalid inputs. GPU throughput and model learning curves have not yet been measured.

## 11 Evaluation and model selection

Evaluation must establish both broad performance with frozen ICL on ordinary tables and useful rare-event prediction against trees trained on all eligible history. Generator checks and mathematical identities validate the setup but cannot establish either result. We measure latency on the same pipelines used to assess accuracy, including context construction and ensembles.

### 11.1 Generator and model acceptance checks

Before a substantial run, we check reproducible generation and the prediction interface. Tests cover deterministic replay, finite values and stable moments, nominal categories, support-only preprocessing, legal class masks, query-label mutation invariance, packed-episode isolation and query-batch independence.

Regression tests check density normalization and analytic means at bin boundaries, extreme tail scales, constant targets and extreme outliers. We compare Muon with a small unfused FP32 reference and verify parameter counts and optimizer partitions. These tests concern implementation correctness, not predictive quality.

After class floors and cell caps are applied, distributional audits record support/query shapes, unique worlds, active dimensions, tree occupancy, category concentration, missingness, coarsening, class entropy and prevalence, target tails, and accepted family/head frequencies. Stage-specific plots are compared with development-table descriptors without using locked test labels. Noisy labels, null signals and identical visible rows with different outcomes remain valid draws.

We first reproduce the unchanged reference’s reported evaluation behavior to make comparisons with the new bank interpretable. Prior, architecture, optimizer and head are separate factors: changing the architecture may also change preprocessing, throughput and loss scale. A lower training loss alone cannot establish a prior improvement.

Comparisons pair world seeds where possible and reserve independent namespaces for evaluation. Combinations that survive selection receive at least three training seeds across the funded screening and scaling runs. Any additional uncertainty from a single final long run is reported.

## 11.2 Standard benchmarks and inference tracks

TabArena is the primary standardized endpoint. Its published design uses three outer folds, repeated ten times for tables smaller than 2,500 rows and three times otherwise. Binary ROC-AUC, multiclass log loss and regression RMSE are the primary metrics [32]. The current model protocol uses eight inner folds and distinguishes default, tuned and ensembled variants; system entrants control their pipelines within a budget [33].

The comparison pins repository revision, dataset IDs, split hashes, wrappers and checkpoint versions. TabArena-Lite supports integration and screening; final results use the full applicable protocol.

We report frozen ICL as both a single pass and a fixed ensemble. The screen compares 1, 4, 8 and 16 independent feature, category and class-encoding views, averaging class probabilities or predictive distributions before computing means. Validation chooses the ensemble size. Reported latency and memory include that setting, with single-pass results retained.

For tasks larger than the validated context limit, we compare uniform or stratified support subsampling with feature-based or learned retrieval using permitted labels and features only. Validation determines retrieval policy and context length. Scored queries do not guide those choices, and a 32k training stage does not by itself validate million-row contexts.

The reference roster comprises TabPFN-3.5, TabICLv2, Kumo Tabular, Xiaomi, EXAONE, current TabFM, LimiX-2 and suitable TabDPT/Mitra variants, plus tuned CatBoost, LightGBM and AutoGluon in the appropriate model or system tracks. Where possible we reproduce official recommended defaults. Every result states inference mode and cost, including whether it uses a bare frozen predictor, adaptation or an ensemble. Predictive quality is the priority, but tuned-system and equal-budget frozen-ICL comparisons answer different questions.

TALENT, BCCO and CTR23 extend task coverage, ScoringBench assesses distributional regression, and applicable single-table BeyondArena slices test robustness. Each suite uses a fixed named task view. TALENT’s 300-table base, paper-specific subsets and extensions are different collections, and its threshold, HPO and support-cap policies must also be pinned [34]. BCCO uses released splits and a stated aggregation rule [35]. ScoringBench needs matched distribution adapters and metric-specific coverage [36]; better CRPS does not necessarily mean better RMSE.

Development, confirmation and locked evaluation are separated by datasets and source lineages. Related OpenML exports and targets are deduplicated across suites. Once a benchmark repeatedly guides prior changes, it is development evidence even if all pretraining data are synthetic.

Reports retain failures, difficult classes, wide tables, memory limits and timeouts. Missing competitor results remain unavailable rather than being replaced by another model’s scores.

## 11.3 Finance evaluation against full-history trees

The main finance baselines are task-trained XGBoost, LightGBM and CatBoost using the complete eligible history, identical point-in-time features and matching label cutoffs. Limiting their training to the ICL context would narrow the claim. Native missing-value support and point-in-time 7/30-day aggregates are included when all methods can access those features. Calibration and threshold selection have an explicit budget. Any tree/ICL blend is reported as a separate system with its fitting cost.

For each tree library, the development screen samples 32 seeded configurations including documented defaults, then confirms the best four at later development origins. All methods receive the same stated wall-clock tuning allowance; unfinished trials are reported.

XGBoost uses histogram boosting, learning rate  $LU(.01, .2)$ , equally likely depths 3, 5, 7, 9, child

weight LU(1,100), row and column sampling U(.6,1) and L2 penalty LU(.1,100). LightGBM draws 15/31/63/127 leaves equally and a rounded minimum leaf count from LU(20,2000), with the same learning-rate, sampling and L2 ranges. Row subsampling enables its documented bagging frequency. CatBoost draws depths 4/6/8/10 equally, uses the same learning-rate range and L2 penalty LU(1,100), and records how it handles categories.

Trials permit up to 5,000 boosting rounds and stop after 100 rounds without improvement on a later adjudicated validation period. Half the non-default trials use no class weights. The other half draw a positive multiplier log-uniformly from 1 to  $\min(1000, N_- / \max(M, 1))$ , with the corresponding per-class rule for multiclass tasks. Regression and degenerate intervals omit class weighting.

Class weighting is distinct from probability calibration. LightGBM’s official documentation warns that imbalance weights can worsen probability estimates [50, 51, 52]. These ranges are fixed before evaluation and do not claim an optimal HPO allocation. CPU infrastructure and tree tuning count as operational costs, and as GPU-hours only while GPUs are allocated.

Finance evaluation advances through rolling chronological origins, distinguishing event, arrival, prediction and label-availability times. Training uses labels available by each cutoff. Later adjudicated development windows set calibration and thresholds; untouched future windows are evaluated at natural prevalence.

The primary deployment test allows recurring customers. Unseen-customer and unseen-campaign splits provide additional stress tests, with unresolved labels masked. Selectively reviewed test data cannot establish population precision without representative audits or justified observation weights. When positives are scarce, we use a small predeclared calibrator or threshold family and report uncertainty instead of fitting flexible isotonic calibration or searching many thresholds.

Finance reports average precision under an explicit convention, precision and recall at a daily review budget, recall at a predeclared precision requirement, false positives per million negatives, log loss, calibration and amount-weighted captured loss. It distinguishes transactions, accounts and campaigns and includes raw TP/FP/FN counts.

Thresholds are fixed from past validation. Choosing them to meet precision on the test set would not simulate an operational decision. Uncertainty uses blocked analyses that respect shared time, entity and campaign effects, and results include periods without positive events.

Prevalence, true-positive rate and false-positive rate jointly determine precision:

$$\text{Precision} = \frac{\pi \text{TPR}}{\pi \text{TPR} + (1 - \pi) \text{FPR}}.$$

At prevalence  $10^{-4}$ , illustrative precision and recall targets of .8 require FPR about  $2.0002 \times 10^{-5}$ , or roughly 20 false positives per million negatives. If no false positives are observed, the one-sided 95% IID upper bound is  $1 - .05^{1/n_-}$ . About 149,771 adjudicated negatives are needed for that bound to reach the illustrative operating point.

A million-row test at natural prevalence contains about 100 frauds in expectation. Detecting 80 of 100 independent frauds gives a two-sided exact 95% recall interval of roughly [.708, .873]. These targets illustrate evaluation scale; they are not company requirements. Dependence can weaken both calculations, so holdouts need enough adjudicated negatives and uncertainty analysis that respects campaigns.

The public Bank Account Fraud suite offers six synthetic datasets, each with one million rows, 30 features and eight months of data, covering controlled imbalance and shifts [44]. It is a useful development proxy. Its approximately percent-level prevalence and published TPR-at-5%-FPR operating point do not establish one-in-ten-thousand performance. Official results and any additional base-rate stress test are reported separately. An operational claim still needs company data and sufficiently long adjudicated

holdouts.

## 11.4 GPU batch-latency measurements

Initial latency profiles use one MI355X per replica. The 64-GPU cluster can process independent datasets or estimators without assuming a single table benefits from 64-way inference. Each profile pins checkpoint, ROCm/PyTorch/kernel revisions, precision, preprocessing, selector, ensemble count and output type.

End-to-end timings include the CPU codec and transfers, with startup reported separately. A cold context starts without a support cache but with weights resident; it excludes downloading a new checkpoint.

The profile grid uses  $n_s \in \{512, 2048, 8192, 32768\}$ ,  $F \in \{16, 64, 256\}$  and  $n_q \in \{128, 1024, 8192\}$  with binary, 10/100/256-class and regression heads. Stress cases add 65,536/131,072 support rows at  $F = 64$  and  $F = 1024$  at  $n_s = 2048$ . Some exceed training shapes and test capabilities that have not yet been established. Real-table timings accompany the synthetic grid, with failures, OOMs and every fallback recorded.

For every feasible shape, five untimed warmups precede twenty timed repetitions. Cold repetitions clear the support cache, and GPU synchronization brackets wall-clock intervals. Diagnostic timings cover codec, host/device transfer, support frontend/trunk, query frontend/trunk and outputs. Overlap means component times may not sum exactly.

We report median and p95 cold-context batch time, warm-cache time, rows per second and peak device memory. Ensemble timings include all members and aggregation, stating sequential and feasible parallel settings separately. The profiling budget includes warmups and failed trials.

The systems grid and memory availability determine chunk size from  $\{128, 512, 2048\}$ , without scored labels. Execution chunks are distinct from the total query batch. Before comparing speed, we test cached versus uncached predictions, changed chunks and reordered queries against FP32 reference tolerances. Reports include observed BF16 error and any benchmark prediction changes. Faster kernels are considered only after attention-mask equivalence is verified.

For each context size and width, selection compares quality with total batch time and reports binary, multiclass and regression separately. Full dense context provides the accuracy reference where feasible. A nonsignificant difference does not establish equal quality.

A noninferiority margin must be chosen before its confirmation run. Until that product criterion is set, lower latency alone cannot justify selecting a model with worse point-estimated quality. The higher-quality version remains available, with uncertainty reported and no claim of lossless compression.

## 11.5 Statistical criteria and controlled comparisons

Statistical reports separate binary, multiclass and regression scorecards while retaining official overall aggregation. Comparisons pair independent source datasets; repeated folds are repeated measurements, not independent datasets. Reports include raw metrics, benchmark ranks and uncertainty. Elo scores from different competitor rosters are not combined into a universal score. Proper losses and calibration supplement the official primary metrics.

Current prior selection uses the paired proper-loss rule in Section 5. The previous plan sought gains on all three panels, at least 1% improvement on a predeclared loss scale and at most 2% deterioration on important slices. Those historical thresholds document the earlier plan and add no requirements to the

current search.

The previous binary ranking used  $1 - \text{AUC}$ . Current prior selection uses log loss, while official benchmark reporting still uses AUC. Other metrics retain their original scales. Analysis accounts for candidate selection and multiple comparisons. Gains on two task types accompanied by a loss on the third are reported as a tradeoff and may motivate separate specialists.

`research/specs/ablation_plan.csv` preserves the earlier comparison catalog; `research/specs/prior_selection_v1.json` defines currently funded experiments. Section 10.6 and `research/specs/prior_bank_spec.json` specify their allocation. Before confirmation we fix comparator roster, dataset lineages, per-task normalization, practical margins, candidate set, inference and failure policies, and analysis times.

We average repeated folds within lineages before paired lineage resampling, incorporating training-seed variation rather than counting folds as independent. Multiple separately selected claims require a familywise procedure such as Holm. One claim that requires improvement on all three task types must pass each of its three constituent tests. Failure to reject a difference is not equivalence.

The paired standard-panel evaluator implements the specified proper-loss comparison, source-lineage bootstrap and confirmation rule. The official cross-suite protocol, broader multiplicity analysis and finance inference respecting time and campaign dependence still require separate work. Finance thresholds also need evidence beyond point estimates: one positive alert with no false positives cannot establish high population precision.

The planned finance analysis fixes a small threshold family on validation, applies an uncertainty criterion respecting time and campaign dependence, and evaluates once on untouched future data. With too few independent events, the result remains unresolved; more pretraining cannot supply missing evaluation evidence.

The optional 5% real-data continuation study uses source-lineage-disjoint development data, excludes locked benchmarks and keeps frozen inference. It preserves a synthetic-only checkpoint and discloses extra supervision. Task-centric TabDPT results motivate the comparison [24, 25], without supporting contaminated evaluation or attributing all gains to procedural priors.

## 11.6 Joint-model selection within the budget

The starting sequence retains the standard checkpoint and trains a separate binary-fraud specialist. The finance module also permits a pure-standard versus 80/20 joint-training comparison. A second experiment compares the full bounded selector with a context control while fixing model, worlds, search coverage and optimization.

Before choosing the two costly confirmation arms, we evaluate intermediate controls on the same development suite: rare prevalence alone; reservoir plus available positives; full selection and metadata; and structured masks and delay. An equal-world-count diagnostic supplements the main equal-compute comparison to distinguish inference improvements from extra data-processing work.

Replacing the starting sequence with a joint model requires preserving the standard binary, multiclass and regression criteria and improving finance results over tuned full-history baselines at declared operating points. Finance utility and benchmark rank stay separate so an aggregate cannot hide a failure.

The provisional final package allocates 9,000 hours to standard training and 3,000 to binary specialization while retaining both checkpoints. Replacing it with a successful joint model requires a recorded selection decision. All final checkpoints share the 12,000-hour limit, and any further continuation or recovery needs an identified reassignment within the project budget.

## 12 Implementation and validation

The codebase implements the integrated Q prior, reference and legacy-control generators, finance generation, dense ICL and the size grid, AdamW/Muon training, mixed tasks, checkpoint recovery, exact support caches, bounded finance inference and data/evaluation commands. The root `README.md` points to `docs/training.md` and `docs/data_and_evaluation.md`; `docs/tabarena.md` describes the optional TabArena wrapper.

### 12.1 Map from the recipe to the implementation

Table 10 connects the Q candidate, reference pilot and finance workflow to modules in `src/tabular_foundation/`. Q and its independent observation law use the common episode interface and trainer. `schema.Episode` implements the records in Section 9.2, and `model_inputs()` explicitly lists permitted forward fields. New priors and retrieval policies use the same boundary, keeping hidden simulator state out of prediction.

Table 10: Scientific components and executable code. Module paths are relative to `src/tabular_foundation/`.

| Component                | Module                                                   | Responsibility                                                                                                                             |
|--------------------------|----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Challenger bank          | <code>challenger_prior.py</code>                         | Draws task and shape before R/F/H/S/I; separate random streams, H fallback and retries within a source.                                    |
| Response and observation | <code>selection_laws.py</code>                           | Calibrated class scores, regression noise, missingness and coarsening; diagnostics remain outside model inputs.                            |
| Reference bank           | <code>reference_prior.py</code>                          | Verified upstream source, task/shape-first R/P1/P4 dispatch and generation audit.                                                          |
| Ordered producer         | <code>producer.py</code>                                 | Seeded CPU process generation, bounded ordered prefetch and finite-world cache.                                                            |
| Earlier control          | <code>static_prior.py</code>                             | <code>generate_episode()</code> ; frozen worlds, response laws, rendering and optional context controls.                                   |
| Finance prior            | <code>finance_prior.py</code>                            | <code>generate_finance_world()</code> and <code>sample_macroepisode()</code> ; finite identities, availability and future-query proposals. |
| Input boundary           | <code>schema.py</code> , <code>codec.py</code>           | Allowed forward fields, nominal types, support-fitted statistics, ranks and value payloads.                                                |
| Context selection        | <code>retrieval.py</code>                                | Reservoir codec, shared centers, streamed candidate search, deduplication and query routing.                                               |
| Predictor and heads      | <code>model.py</code> ,<br><code>distributions.py</code> | Cell encoding, both architectures, attention masks, metadata, support caches, quantile and density heads.                                  |

| Component        | Module                                                                                    | Responsibility                                                                            |
|------------------|-------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Optimization     | <code>optim.py</code> , <code>train.py</code> ,<br><code>runtime.py</code>                | Muon/AdamW partition, macroepisode loss, synchronized gradients, horizon and recovery.    |
| Inference        | <code>inference.py</code> , <code>predict.py</code>                                       | Frozen standard/finance predictors and prediction command.                                |
| Data and scoring | <code>data.py</code> , <code>metrics.py</code> ,<br><code>evaluate.py</code>              | Input validation, temporal contracts, task metrics and validation-selected thresholds.    |
| Paired selection | <code>selection_evaluation.py</code>                                                      | Declared data panels, exact split coverage, paired losses and source-lineage uncertainty. |
| Comparisons      | <code>profile.py</code> , <code>baselines.py</code> ,<br><code>tabarena_adapter.py</code> | Timing, task-trained tree baselines and optional official-benchmark integration.          |

The scientific recipes are stored in `research/specs/static_recipe_v3.json`, `research/specs/icl_architecture_v3.json`, `research/specs/finance_recipe_v3.json`, `research/specs/latency_recipe_v3.json` for latency/MoE, and the coordinating `research/specs/prior_bank_spec.json`. Executable code is under `src/tabular_foundation`, run configurations are in `configs`, and `README.md` provides operating instructions.

The coordinating specification’s `prior_selection` block describes the current research plan. Its `main_candidate` block and `docs/main_recipe.md` preserve the earlier runnable pilot, while the v3 specifications define legacy controls. These research JSON files describe methods rather than trainer configuration syntax. This standalone  $\text{\LaTeX}$  file contains the complete recipe without external input files.

Implemented methods include Q’s five mechanism families and response/observation laws, the compact persistent-cell comparator, paired standard-panel evaluation, base and context-conditioned standard priors, eligible conditional-classification targets, finance generation and selection, and the dense architecture. Options include the reference prior, hash-bit codec, alternative heads, logarithmic scaling, query KV grouping, risk-partition finance law and ordered process producer.

MoE, general local/global attention, feature feedback, auxiliary objectives, adaptive weights and real-data continuation remain proposed comparisons. They are not trainer options. Manifests and results must distinguish those proposals from executable methods.

**Generation and recovery.** `producer.py` can run serially or with persistent spawned CPU workers per training rank. The new candidate configurations use two workers per rank and a four-task prefetch queue. Futures are consumed in scheduled order, so faster generators do not alter source frequencies. Bounded finance caches are keyed by world, stage, task and overrides. They may retain raw data, but model states are recomputed. Worker exceptions propagate with audit records.

Reference runs set `PYTHONHASHSEED=0` because the unchanged upstream sampler depends on set iteration as well as numerical RNGs. CPU tests obtained identical trained parameters under serial and prefetched generation. Rank-zero generation and input-wait counters provide diagnostics rather than whole-cluster utilization measurements.

Completed-attempt logs aggregate source, eligibility, rejection and shape/class counts globally. Replayed attempts increase consumed presentations. Partial failed batches require the generator’s exception audit

and scheduler accounting because their attempt records are incomplete.

## 12.2 Running the candidate and selection experiments

The Q sampler is enabled with `standard_prior=selection`. Its `selection_options` independently controls five mechanism weights, five observation weights and context-conditioned complexity probability. Native task and shape sampling precede dispatch, so a new source does not need to generate and discard an R graph. At a fixed seed, R with identity observation reproduces unchanged native output. New sources retain valid no-signal tasks and absent classes.

`configs/selection_candidate_standard.json` and `configs/selection_candidate_fraud.json` provide the starting runs. They specify provisional 250,000- and 4,000-update horizons, batch 256, 90/9/1 stage shares, candidate Muon/AdamW rates and zero decay, under 9,000 and 3,000 GPU-hour ceilings. Fraud continuation loads a compatible standard checkpoint and creates fresh optimizer state. These configurations make the hypothesis executable; no winner has been selected, and hardware feasibility remains unmeasured.

`scripts/materialize_prior_selection.py` creates phase-specific trainer configurations and a manifest. P can begin without earlier results. M, A, V and F require recorded decisions from preceding comparisons, including chosen mechanism/observation weights and a nonempty evidence reference. Architecture and continuation phases also require the selected model and, when needed, an actual compatible parent checkpoint. The script validates these records without judging the scientific evidence or inventing a winner. Standard horizons are multiples of 100 updates; finance horizons are multiples of 400, preserving stage proportions and four-use world groups. `-steps` supplies common lengths for equal-episode diagnostics. For primary equal-budget comparisons, `-horizons` supplies a complete mapping of trial IDs to measured full-schedule lengths, allowing different episode counts when task costs differ. Manifests record equal package limits; reports must also give actual consumption.

```
export PYTHONHASHSEED=0
python -m tabular_foundation.train \
  --config configs/smoke_selection.json \
  --output runs/selection_smoke
python scripts/materialize_prior_selection.py \
  --phase P --steps 1000 --allocated-gpus 64 \
  --output runs/phase-P
```

The smoke run uses small diagnostic shapes to exercise generation, both task types, optimization and checkpointing. The 1,000-update P command demonstrates configuration generation; research horizons must come from measured stage costs. The manifest gives exact configuration and output paths. `scripts/submit_training.py` converts a resolved configuration to a Slurm command, printing by default and submitting only with `-submit`. It uses the active Python environment, bounds scheduler wall time by remaining run/project budgets and requests producer CPU resources alongside GPU ranks. The PyTorch code supports CUDA and ROCm builds, neither of which has been profiled here.

`-evaluation-reserve-gpu-hours` reserves a declared per-trial allowance inside the package and deducts it from training. Its zero default is explicitly marked unreserved. Separate evaluation jobs do not update the ledger automatically: `scripts/record_allocation.py` records their cumulative scheduler-billed time under the manifest's evaluation ID and can reconcile killed or resumed training under its original run ID. Repeating a cumulative total is idempotent, decreases are rejected, and overspending remains visible. Training plus evaluation must fit each package.

The project ledger tracks consumed time rather than reservations for concurrent jobs. Full-cluster runs should therefore be submitted serially, with scheduler charges reconciled after failures or kills. A

run stopped by its budget before the declared horizon is recorded as incomplete. Deterministic online generation and bounded ordered prefetch supply tasks without a precomputed corpus.

**Real-data evaluation.** `selection_evaluation.py` reads a declared development, confirmation or final panel. Each split specifies dataset, source lineage, task, fold, legal class count and separate support/query files. Stable original-record IDs establish disjointness; if absent, an explicit verified split-provenance record is required. The runner rejects identical files, ID overlap, changed paired data and incomplete split coverage. Lineage still requires a provenance audit: software cannot establish independence from dataset names.

Evaluation records both checkpoint training seed and inference-view seed. It times context setup separately from cached prediction and uses 1, 4 or 8 paired views. Views execute sequentially, so peak memory refers to that policy rather than simultaneous caches. The comparison averages folds and training seeds within datasets, then applies Section 5’s objective and source-lineage bootstrap. Reduced diagnostic bootstrap or lineage counts cannot produce a confirmation-evidence flag. Failures remain explicit and prevent hidden subset comparisons. Timings include hardware/software identities; finance evaluation separately enforces chronological label availability and validation-only threshold choice.

`docs/prior_selection.md` documents panel and phase-decision schemas, launch commands, recovery and tests. `scripts/audit_selection_priors.py` streams native-shape generation summaries and failures, clearly marking any small-shape diagnostic. Teacher learning curves from support cross-validation remain a separate research analysis, not an automatic acceptance filter.

### 12.3 From local checks to a cluster pilot

Local validation uses the environment in `README.md` and `docs/training.md`. Before allocating GPUs, we run the test suite and tiny standard-selection/finance configurations. Their small models, simple tables and increased finance prevalence test the pipeline rather than the research hypothesis.

```
python -m pytest -q
python -m tabular_foundation.train \
  --config configs/smoke_selection.json --output runs/selection_smoke
python -m tabular_foundation.train \
  --config configs/smoke_finance.json --output runs/finance_smoke
python research/probes/finance_design_probes.py \
  --output research/results/finance_design_probes.json
```

`docs/data_and_evaluation.md` exercises prediction and conversion, while `docs/tabarena.md` covers the intended benchmark adapter. Cluster work then pins a ROCm-compatible stack and profiles one replica and one node, recording stage-specific allocated GPU-seconds. Those measurements set horizons and cell/microbatch limits before a distributed pilot. Reference reproduction and controlled comparisons precede final model and mixture selection.

Each new run writes a separate directory under `runs/`. Recovery uses the interrupted run’s configuration and checkpoint, preserving sampler state and compute charges. The repository has no competitive pretrained weights; disposable tiny-run outputs can be regenerated. Exact CLI arguments, schemas and cluster procedures live in the operating guides rather than being repeated throughout this report.

## 12.4 Numerical and statistical checks

Several accounting details determine whether implementation matches the intended objective. Mixture weights apply to emitted macroepisodes: an 80/20 world split with fourfold finance reuse would yield 50% finance episodes. Route losses retain the original query denominator, and realized-loss regression represents zero with an explicit atom. Feature-dependent retrieval is trained on population queries because a scalar class-sampling correction cannot undo that selection.

Reference cohorts must be fully adjudicated, their sizes are visible to the model, and scalar reference smoothing is identical in training and serving. Finance widths follow the active stage, distributed budget decisions are synchronized, and recovery retains compute spent since the last checkpoint. Sparse multiclass tree baselines keep unseen-class examples in evaluation and report coverage.

Tests check parameter counts, generator replay and moments, query-label isolation, attention/cache invariances, density normalization and moments, optimizer partitions, checkpoint replay and distributed gradient reduction. `docs/audit.md` discusses findings and remaining risks; `research/results/code_validation.json` stores machine-readable results.

## 12.5 Design implications of the implementation audit

The audit found no direct query-label input leak and retained the support/query boundary, finite-population accounting, population-weighted loss, frozen-weight caches and optimizer partition. Counterexamples prompted higher-precision affine target transforms, finer heads, discrete identity inputs, logarithmic attention scaling and shared query KV. They also motivated hiding H from both the finance selector and predictor. Real-data comparisons are still needed to determine whether these changes improve accuracy.

One precision failure occurred before histogram quantization: FP32 casting made targets  $10^8$  and  $10^8 + 1$  both standardize to zero with center  $10^8$  and scale 0.5. The revised implementation retains FP64 affine quantities through standardization and inversion, while neural probability computations use FP32. Serving preserves the recovered original-unit precision.

Tests cover NLL and gradients, CDFs, quantiles, means and the positive-truncated finance head. Increasing bin resolution alone would not recover a target difference already lost during affine preprocessing.

These examples connect prior coverage to information preservation. More varied tasks cannot help if inputs lose feature identity, small target differences or rare historical evidence. Greater model capacity cannot recover structures absent from training. The comparisons first isolate these effects and then assess their combination against reference foundation models and full-history trees.

## 12.6 Validation and limitations

The earlier runtime audit passed **200 tests with no skips**, treating warnings as errors and including the two-process Gloo test. It covered source verification/replay, task and shape preservation, rejection counts, categorical encoding, heads, optimizer partitions, caches, recovery and per-prior counts. `research/results/code_validation.json` retains environment details and results, with earlier records in `docs/audit.md`.

A subsequent specification-only check added analytical identities, exposure accounting, dependency validation and invalid-input tests. Its focused run passed 45 tests with warnings as errors and changed no runtime generator or model. The later implementation added Q, common response/observation laws, the persistent-cell comparator, phase configuration and paired evaluation. These are successive

validation records, not revised interpretations of earlier tests.

The current implementation passes all 354 tests with warnings as errors, including the opt-in distributed test on permitted loopback sockets. A three-update training run completed all three diagnostic stages both serially and on two local Gloo ranks, with all 143 checkpoint tensors, losses and source counters exactly matching. `persistent_tiny` completed the same path. These small CPU runs do not validate accelerator kernels, 64-GPU communication or planned research volume. A separate generator check produced valid 512-feature, 256-class tasks from each of F, H, S and I with absent support classes retained; four worlds do not establish distributional coverage.

A two-worker audit accepted all 60 requested worlds at fixed shapes of 48 support rows, 24 query rows, 12 features and a four-class budget. It exercised all five sources across three stages without new-world numerical retries. This validates the audit command’s execution, not the planned thousand-world native-shape study, which must retain eligibility fallbacks and count actual accepted sources. Results are in `research/results/selection_training_integration.json` and `research/results/selection_prior_smoke_audit.json`.

Earlier integration tests trained tiny CPU models on ordinary and finance tasks. After three updates, a two-process mixed-profile run differed from serial by at most  $6 \times 10^{-8}$  in any parameter. Another three-update reference-prior run continued for four binary-specialist updates, testing weight initialization and world reuse. It lazily represented one million historical rows at prevalence  $10^{-4}$  and used a diagnostic selector with 256 candidates.

A three-million-row simulation checked population counts and row identities. Other tests exercised prediction, evaluation, profiling, CSV conversion and all three tree-library adapters. Their capped search pools do not measure full-history retrieval throughput.

Numerical probes cover learnability, conditional-gradient identities, rare-event weighting, cache arithmetic and MoE initialization. In small FP64 modules, dense-to-expert decomposition error was below  $7 \cdot 10^{-16}$ , widening error was zero, query-chunk error was below  $1.4 \cdot 10^{-15}$  and finite-difference gate-gradient error was below  $2.0 \cdot 10^{-11}$ . `research/results` stores detailed results and source hashes, with scripts in `research/probes`. These validate numerical identities, not full-model learning or accelerator kernels.

The 317M starting model has not been pretrained or tested on competitive benchmarks, and MI355X timings are unavailable. Potential bottlenecks include the CPU/FP64 codec and transfers. The bounded producer queue overlaps CPU work with training but has neither a pinned-memory transfer stream nor a byte-bounded queue. The persistent-cell comparator and paired standard-panel evaluator are implemented and tested locally; accelerator throughput and real-data transfer remain unmeasured.

The next experiments reproduce the released reference, measure stage-specific training costs and compare prior/architecture variants under fixed budgets. Ordinary benchmarks and chronological finance tests remain separate selection endpoints. They will determine whether the proposed task structure survives context selection and representation well enough to improve on reference foundation models and full-history trees.

## A Reference generator and earlier prior designs

This appendix documents the reference generator and the earlier prior designs retained as controls. The initial executable pilot mixed the released TabICLv2 generator with hierarchical category tasks on eligible draws. Its purpose was to test whether repeated identities improve inference while preserving most of the reference distribution. We first describe that reference and the R/P1 pilot, then specify the P0/P1/P4 replacement control in full. The current candidate mixture is specified in Section 5; the earlier mixtures below provide comparisons with it. The P0/P1/P4 configuration is stored in `research/specs/static_recipe_v3.json`. Unless a draw is explicitly row-specific, its parameters are sampled once per world and shared by calibration, support and query records.

The P0/P1/P4 bank includes smooth responses, sharp boundaries, joint causal dependence, local similarity, category pooling, and count or amount outcomes. Two additional controls change how the model learns from this bank. Context conditioning limits mechanism complexity while keeping the rest of the bank fixed. Conditional-label supervision changes the estimator of the training objective. Both are tested separately from changes to mixture weights.

### A.1 Reference prior and earlier extensions

The released reference provides a broad range of compositional structure: TabICLv2 combines eight function families in vector-valued graphs and samples their hyperparameters through a structured procedure [8]. By comparison, the `static-v3-base` SCM uses four node-function families and at most sixteen observed informative causal nodes. Its remaining columns are often proxies or distractors. This smaller bank is a controlled alternative, and its transfer performance must be measured against the broader reference.

Arm R uses an unchanged source revision of the released generator. In the earlier G control, an eligible episode comes from R with probability 95% and from hierarchical-category family P1 with probability 5%. Task type and shape are sampled *before* this choice. P1 requires  $F \geq 8$ ; for narrower tasks, its share goes to R. The separate C control uses 95% R and 5% count/amount family P4 for binary or regression tasks with  $F \geq 4$ . All other C-control tasks use R.

These controls preserve the classification/regression mix, context sizes and feature widths. We record the realized mixture after eligibility checks and filtering because the nominal branch probabilities do not fully describe accepted tables. If the separate G and C experiments support both additions, a 90/5/5 mixture can be tested on shapes that permit both. Any ineligible component again gives its share to R. The small initial additions were chosen to make their effects interpretable, rather than as estimates of optimal weights.

The adapter in `reference_prior.py` follows this sampling order, calls the pinned source, and records raw proposals, rejection reasons and realized shapes. R retains its released filters. P1 and P4 inherit the selected task, shape and class budget, and are accepted whenever they satisfy the validity checks. Applying the reference teacher filter to either would add another experimental change.

Two implementation details matter when comparing these controls with the published system. First, native R returns numerically encoded columns without categorical type flags. P1 and separately typed tasks therefore provide most of the explicit training for the nominal encoder. The model’s preprocessing is specified separately from the native generator.

Second, the current trainer requests one episode at a time. This preserves the sampling procedure for each table but omits the released first-stage grouping: four tables share their row count, support split and requested width. Even in those groups, class budgets and graphs are drawn separately. The batch

API preserves the grouping, and the later stages already use groups of one.

The earlier development plan first targeted category occupancy and conditional uncertainty, then count outcomes with zeros, varying exposure and dispersion. Its G/C controls remain specified here; Section 5 defines the current funded comparisons. Results on the relevant real-data strata determine whether an addition merits a larger share. Structured missingness and coarsening are also important coverage controls, although they are established prior ingredients. Context caps, conditional-label losses, adaptive weights and real-data continuation remain separate experimental factors. A failed G or C comparison leaves R in place for that choice; it does not initiate an unrestricted search for more families.

**Native and expanded envelopes.** The released curriculum starts with a fixed 1,024 rows, then uses 400–10,240 rows in stage two and 400–60,000 in stage three. Native R allows at most 100 features and ten class-budget slots, with tighter width caps for long tables. Under the expanded-envelope setting, 80% of later-stage requests retain this law. The remaining 20% allow up to 256 features and 32 classes in stage two, and 512 features and 256 classes in stage three. These settings change the generator distribution and are recorded as such.

A larger request limit does not guarantee a table of that size. Constant-column removal, realized class counts and native split repair still determine the accepted table. For example, one local diagnostic requested 64 classes but returned 25 observed classes after twelve raw attempts. Realized class and width histograms are therefore needed before expanding the range or committing a long run. Forcing classes into query sets or padding labels would change the sampling law, and would not demonstrate generation of a 256-class task.

## A.2 Generating an episode in the earlier R/P1 pilot

The earlier R/P1 pilot first chooses classification or regression, curriculum stage, total rows, support split, requested feature count and, for classification, class budget. It then draws the R/P1 branch. An eligible P1 task inherits all these choices; a P1 draw with width below eight goes back to R. Choosing the task and shape first prevents a generator’s own preference for particular shapes from changing its assigned mixture share.

**R: joint graph generation and released filtering.** R generates features and outcomes together in a hidden computation graph. The pinned implementation repeats the following procedure until it accepts a table, retaining the outer task and shape throughout.

1. **Specify the raw table.** Draw feature types, category cardinalities and shared distributional settings. The classification/regression task has already fixed the response type. Compatibility behavior in the released source caps native feature-category cardinalities at nine.
2. **Build a hidden acyclic graph.** Draw a graph with 2–32 nodes and assign the requested features and target to node outputs. A node can supply several observed columns, while other nodes remain hidden. Redraw the graph and column assignments until features and target share ancestors. This condition creates an opportunity for statistical dependence; it does not identify causal relationships in a real table.
3. **Assign functions to nodes.** Roots receive random vector inputs. Other nodes concatenate or aggregate their parent vectors, then apply sampled MLPs, oblivious-tree ensembles, discretization, linear or quadratic maps, random-Fourier GP maps, EM-like assignments, or products. A graph can combine different function types. Hidden node vectors can also have more dimensions than the observed columns.

4. **Generate all requested rows.** Evaluate nodes in topological order. Sampled converters produce numerical and categorical outputs and pass converted hidden vectors to descendants. Parameters and fitted transformations are shared across the raw table. Many transformations are fitted on those same generated rows because the released configuration has no separate IID fit-only pass. Random roots, hidden dimensions and categorical draws supply variation even though optional extra additive Gaussian node noise is disabled by default.
5. **Render and check the table.** Clip and standardize native values, permute columns and class IDs, and remove constant features. Reject a table if no features remain. For classification, try up to ten row permutations to obtain the same observed class set in support and query, with at least two labels. Then use a 25-tree, depth-six ExtraTrees OOB test with 200 bootstrap resamples to reject insufficiently predictable tables. The optional too-easy filter is disabled. A rejection starts a new internal-world draw under the same outer task, shape and source.
6. **Separate inputs from targets.** Return the accepted support and query rows. Native categorical columns already have numerical encodings and no type flags, so the local model processes them through its support-only codec. Its forward arguments exclude the hidden graph, query labels and rejection diagnostics.

The retained generator executes these steps through `reference_prior.py`. In `third_party/tabicl_reference/src/tabicl/prior/`, `_dataset.py` defines shapes and acceptance, `_graph_scm.py` builds raw tables, and `graph_lib/` provides graphs, node functions and converters. The source manifest pins every one of these files. Changing table-wide transformations, split repair or filtering would change the accepted distribution even if the graph-function list were unchanged.

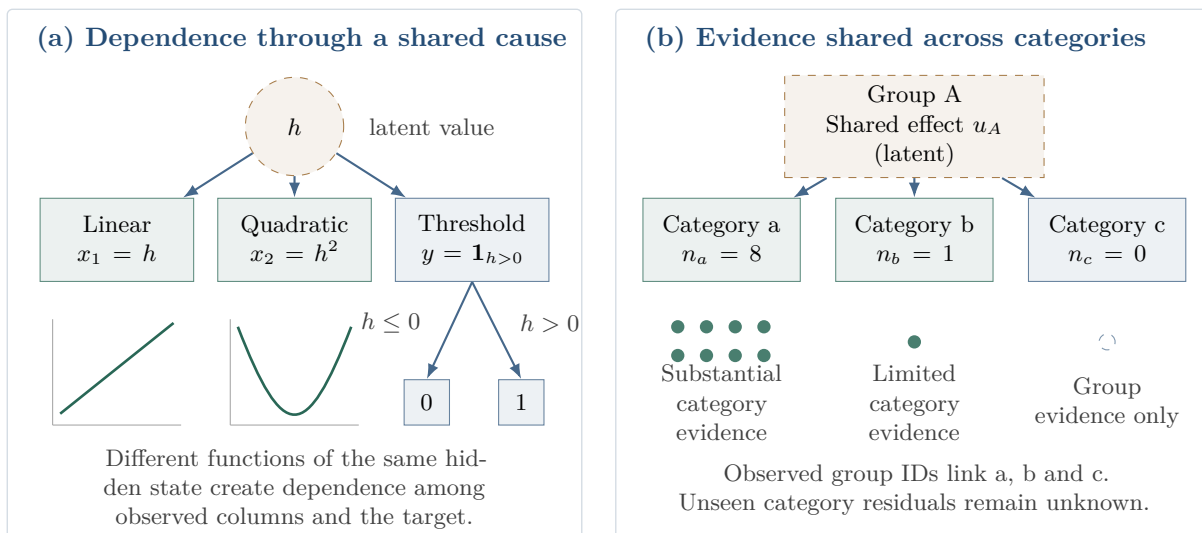


Figure 7: Two structures illustrated by the standard bank. (a) Features and target depend on a common hidden ancestor. (b) Observed group identifiers or metadata allow categories with few or no support rows to use evidence about a shared group effect. Counts refer to support rows. These are schematic examples, not native-generator outputs, and latent effects are never model inputs.

Four hand-chosen latent values make the left-hand schematic concrete. They produce the table below, where the same hidden mechanism relates two visible columns to the target. The generator knows all four outcomes; the predictor receives only the support outcomes. This small example omits the native row count, vector mechanisms and acceptance filter.

| Partition | Visible $x_1$ | Visible $x_2$ | Target $y$           |
|-----------|---------------|---------------|----------------------|
| Support   | −1            | 1             | 0, visible           |
| Support   | 1             | 1             | 1, visible           |
| Query     | −.5           | .25           | 0, withheld for loss |
| Query     | .5            | .25           | 1, withheld for loss |

**P1: entity and group effects.** In P1, repeated observations of a category carry information about a shared effect. Group membership and observed metadata also help when direct observations are scarce.

1. Reserve four requested columns for entity ID, parent-group ID and two observed metadata coordinates. The feature factory supplies the remaining  $F - 4$  columns. Independent streams generate 4,096 internal calibration rows, support rows and query rows. Calibration fits the generator and adds no rows to the model’s context.
2. Choose an entity-catalog size between eight and  $\min(4096, \max(16, 2n_s))$ , using a discrete-log-uniform draw. Choose an integer group count between two and  $\min(32, \max(2, E/2))$ . Entity frequencies are uniform with probability .2; otherwise they follow a power law with randomly assigned ranks. The catalog can therefore be balanced or have a long tail of rare entities.
3. Draw each entity’s metadata, intercept and optional slopes once. The intercept has a part predictable from metadata and a hidden residual. Entities in the same group share an additional group intercept. Appendix A.8 specifies the scales and probabilities.
4. Draw a row’s entity from the fixed frequency law. Combine a common GAM with the entity and group intercepts and entity-specific slopes on up to two ordinary coordinates. For output coordinate  $k$ , the score is

$$s_k(v, e) = b_k(v) + a_{e,k} + g_{\text{group}(e),k} + \beta_{e,k}^\top v_{1:d}, \quad d = \min(2, d_{\text{active}}).$$

Here  $v$  contains the latent features before rendering, masking and column permutation.

5. Use calibration to fix the response conversion, then sample the binary, multiclass or regression outcomes. Apply the shared rendering and missingness rules to obtain the observed table. IDs remain nominal, and support and query use the same effects. The native R teacher filter is not applied to this branch.

The illustration shows how the available evidence changes across categories. Eight support observations inform the effect of category a directly. Category c has no support observations, so prediction uses its group, metadata and other features while retaining uncertainty about its hidden residual. This is an inference problem of *partial pooling*: entities share some structure without having identical effects. Training presents this problem across tasks; it does not insert a programmed partial-pooling estimator into the network. The implementation is in `static_prior.py`, especially `_p1()`, the feature factory and the response compiler.

The earlier executable R/P1 pilot provides little missing-data training, which motivates the observation law in Section 5. Native R deliberately introduces no NaNs. P1 applies independent MCAR masking on 30% of its tasks, at cell-mask rate  $U(.01, .20)$ , and applies no such mask on the rest. Accounting for width eligibility, approximately 1.402% of all standard tasks enable masking: an expected 897,454 tasks in the 64 million-task plan. These are counts of generated training cases, not measurements of robustness. The finance specialist uses the structured missingness mechanisms described in Section 6, but its separate continuation cannot improve the retained standard checkpoint. Evaluation must test reliable prediction under different missingness mechanisms as well as the ability to accept missing inputs.

### A.3 P0/P1/P4 replacement control: mixture and variants

The earlier replacement control mixes P0, P1 and P4. The table reports their *marginal* episode probabilities. Within P0, the five subfamilies have conditional probabilities of 20/25/30/15/10 percent, summing to one. Multiplying each by P0’s 85% share gives its table entry.

| Generator                                 | Episode mass | Intended inference capability                                       |
|-------------------------------------------|--------------|---------------------------------------------------------------------|
| P0 linear/GLM/GAM and interactions        | 17.00%       | Sparse/additive effects and low-order composition                   |
| P0 sharp tree and tree-ensemble functions | 21.25%       | Discontinuities, thresholds and regime boundaries                   |
| P0 heterogeneous joint SCMs               | 25.50%       | Latent causes, redundant observations and arbitrary target position |
| P0 local/kernel functions                 | 12.75%       | Similarity, local smoothness and anisotropy                         |
| P0 gated regime compositions              | 8.50%        | Context-dependent mechanisms with shared structure                  |
| P1 hierarchical categorical effects       | 10.00%       | Partial pooling, rare levels and meaningful category metadata       |
| P4 counts and compound amounts            | 5.00%        | Zeros, dispersion, exposure and heavy positive tails                |

Table 11: Initial probabilities for the P0/P1/P4 replacement control. Conditional modifiers determine missingness, rendering, class prevalence and observation coarsening.

The first replacement comparison uses **static-v3-base**: fixed mixture weights, independently sampled mechanism complexity, hard labels, validity-only filtering and no auxiliary loss. The **static-v3-context** control changes only the specified 90% context-capped/10% base-replay rule. Conditional-distribution losses are tested separately again. Adding one change at a time allows the evidence to identify a useful combination; plausible ingredients alone do not establish one.

### A.4 Sampling conventions and base task shapes

We use  $\text{RU}(a, b)$  for a uniform integer on the inclusive set  $\{a, \dots, b\}$ , and  $\text{LU}(a, b) = \exp U(\log a, \log b)$  for a positive real. The discrete-log-uniform version is

$$\text{DLU}(a, b) = \min\{b, \max\{a, \lfloor \exp U(\log a, \log(b+1)) \rfloor\}\}.$$

All integer bounds are inclusive; continuous uniforms exclude their upper endpoint. Empirical quantiles follow NumPy’s linear interpolation between neighboring order statistics. Calibration moments use population denominators, without degrees-of-freedom corrections. At a numerical tree split, equality goes to the left child; at a quantile-category boundary, it goes to the upper bin. Unless probabilities are specified, “choose” means uniform sampling without replacement.

Several conventions make degenerate draws reproducible. Empty interactions contribute zero. Coefficients are divided by  $\max(10^{-12}, \|w\|_2)$ , leaving an all-zero vector at zero. Categorical probabilities are normalized in float64, and softmax subtracts the largest logit. Generation and calibration initially use float64; model inputs are cast afterward. Independent counter-based streams named **shape**, **parameters**, **calibration**, **support**, **query**, **labels**, and **rendering** include both world ID and retry number in every key.

The P0/P1/P4 standard profile gives equal nominal mass to binary classification, multiclass classification and regression. Finance uses a separate observation and selection process. Family probabilities are .85/.10/.05 for P0/P1/P4, but P4 itself chooses binary classification with probability .2 and regression

with .8. To compensate, P0 and P1 use task probabilities

$$(p_{\text{bin}}, p_{\text{multi}}, p_{\text{reg}}) = \left( \frac{1/3 - .05(.2)}{.95}, \frac{1/3}{.95}, \frac{1/3 - .05(.8)}{.95} \right) \approx (.340350877, .350877193, .308771930),$$

which assign exactly one-third of nominal standard-profile episode mass to each head. The default accepts support sets with missing classes. Arithmetic failures must still appear in reports comparing proposed and accepted distributions.

To sample context size, choose one of three bins with equal probability:  $[32, 127]$ ,  $[128, 511]$ , or  $[512, 2048]$ . Draw support size uniformly within the chosen bin and set  $n_q = \min(128, \max(16, \lfloor n_s/4 \rfloor))$ . Draw total width  $p$  from  $\{4, 8, 16, 32, 64, 128\}$  with probabilities  $(.10, .15, .20, .25, .15, .15)$ . Reserved columns require P1 to replace  $p$  by  $\max(p, 8)$  and P4 to replace it by  $\max(p, 4)$ .

Binary tasks have  $K = 2$  classes. Multiclass tasks draw  $K = \text{RU}(3, 10)$ , increase support size  $n_s$  to  $\max(n_s, 16K)$  and recompute  $n_q$ . This floor connects task complexity to the available context without requiring every declared class to appear in support. It also changes the realized context histogram, so reports retain both the proposed bin weights and the realized sizes. Dimensions count actual data and exclude hidden padding.

Before constructing model context, the generator fixes its constants using an independent pool of 4096 calibration rows. These rows are neither support examples nor scored queries, and their labels never enter the learner interface. The returned record contains the typed table, support/query indices, support labels and task kind. A separate evaluator object stores query labels, world parameters, achieved calibration statistics and provenance. Category IDs keep their nominal meaning. Model preprocessing fits only support data; calibration constants remain internal to the sampler.

## A.5 Shared features and the observation process

The shared feature factory determines how many coordinates carry information and how they are represented. For P0 outside the SCM branch, informative count  $d$  follows  $d = \text{DLU}(1, \min(p, 16))$  with probability .75 and  $d = \text{RU}(\lceil p/3 \rceil, p)$  otherwise. P1 and P4 apply the same rule to their nonreserved width. A world draws categorical fraction  $r_c \in \{0, .25, .5, 1\}$  with probabilities  $(.35, .30, .25, .10)$ ; each base coordinate then becomes categorical independently with probability  $r_c$ .

Numerical coordinates can be correlated within small blocks and can share a heavy-tailed scale. Draw world correlation  $r \in \{0, .3, .8\}$  with probabilities  $(.4, .4, .2)$ , randomly permute the numerical indices, and place them into consecutive blocks of four. For each row, draw a standard normal  $u_b$  per block and independent  $\epsilon_j \sim N(0, 1)$ , then set  $v_j = \sqrt{r} u_b + \sqrt{1-r} \epsilon_j$ . In a world selected with probability .2, multiply every numerical coordinate in a row by  $\sqrt{5/\chi_5^2}$ , using one row-specific denominator. The fixed degrees of freedom and shared denominator create heavy-tailed geometry across coordinates, rather than independent marginal noise.

Categorical coordinates carry effects associated with nominal categories. Draw cardinality  $C$  from

$$C \in \{2, 3, 5, 10, 32, 128\}, \quad P(C) = (.25, .15, .20, .20, .15, .05).$$

Draw exponent  $a = U(0, 1.5)$ , assign randomly permuted ranks, and make category frequencies proportional to  $r^{-a}$ . Give each category a fixed  $e_c \sim N(0, 1)$  effect, centered and scaled using its exact frequency-weighted mean and variance, with variance floor  $10^{-12}$ . Rows sample categories from those fixed frequencies. The score uses  $v_j = e_c$ , while the learner sees a separately permuted categorical code on which no arithmetic is performed. Category effects therefore have no imposed ordering of the kind created by discretizing Gaussian values.

The other  $p - d$  columns are either noisy proxies or unrelated distractors. Each independently copies a uniformly chosen informative coordinate with probability .3. A numerical copy adds  $N(0, \sigma^2)$  noise for  $\sigma = \text{LU}(.01, .3)$ . A categorical copy replaces its category with an independent draw from the fixed frequencies with probability .05 per row. Otherwise, the column is a fresh factory coordinate with no factor shared with the informative coordinates.

The learner sees transformed versions of the latent coordinates used by the score. Each numerical coordinate receives one transformation, fixed for the world: identity with probability .50,  $\text{asinh}(v)$  with .15,  $\text{sign}(v)|v|^q$  with .20 and  $q = \text{LU}(.3, 3)$ , or  $\exp(\text{clip}(v, -5, 5))$  with .15. It is then observed as  $x = AT(v) + B$ , where  $A = \text{LU}(.1, 10)$  and  $B \sim N(0, 3^2)$ . The score still uses prerendered  $v$ . Some maps are invertible; clipping can discard information.

Quantization can discard additional information. Independently, a world selected with probability .25 chooses  $\min(p_{\text{num}}, \lfloor p/4 \rfloor)$  numerical columns, where  $p_{\text{num}}$  is the available count. Each chosen column is rounded to the nearest grid point, with ties to even. Its grid width is  $\text{LU}(.05, .5)$  times its calibration standard deviation. Constant columns are valid, including those produced by nonlinear functions or particular categorical draws.

The baseline missingness rule leaves every cell observed with probability .70. Otherwise, it draws a world rate  $m = U(.01, .20)$  and masks cells independently with probability  $m$ . Values are returned with an explicit mask. MAR/MNAR mechanisms and query shifts have separately named ablations. Columns and category codes are permuted once per world and used consistently across splits. No transformation is inferred from query outcomes.

## A.6 Response distributions and task construction

P0 and P1 use a common procedure to turn mechanism scores into task outcomes, so their mechanisms can be compared under the same response definitions. Each supplies a fixed score function  $F$  with one output for regression or binary classification and  $K$  outputs for multiclass classification. Calibration centers each output and divides by  $\max(10^{-6}, s)$ , where  $s$  is its standard deviation. Variance below  $10^{-12}$  produces a zero standardized output and a logged degeneracy. This rescales an internal function; it does not normalize emitted targets using query labels.

Regression includes relations with no signal as well as noiseless ones. Null, weak, medium, strong and noiseless cases have probabilities (.05, .20, .45, .25, .05) and use, respectively,  $\rho = 0$ ,  $U(.02, .2)$ ,  $U(.2, .7)$ ,  $U(.7, .98)$ , or 1. For  $0 < \rho \leq 1$ , generate the pretransform response as

$$w = s(v) + \sqrt{(1 - \rho)/\rho} h(v) \epsilon.$$

The null case uses  $w = \epsilon$  and removes structural paths from the target into features. Each world draws one noise family: standard normal with probability .60, variance-normalized Student- $t_5$  with .20, variance-one Laplace with .10, or  $.9N(0, .5^2) + .1N(0, 2^2)$  divided by  $\sqrt{.625}$  with .10. Noise realizations are independent across rows.

With probability .6, the world has homoscedastic noise and  $h = 1$ . Otherwise, one informative coordinate sets  $h_0 = \exp(.5 \tanh(v_j))$ , which is divided by  $\sqrt{\text{mean}_C(h_0^2)}$ . Parameter  $\rho$  thus approximately controls the signal fraction of *pretransform* variance. It need not equal the final observed  $R^2$ .

The final response is transformed by identity, signed  $\log(1 + |w|)$ , or  $\exp(\text{clip}(w, -8, 8))$  with probabilities (.75, .15, .10). Scale  $B = \text{LU}(.1, 100)$  and location  $A \sim N(0, (10B)^2)$  then determine its units. Diagnostics record clipping and achieved calibration variance. The exponential branch supplies non-Gaussian targets and can create point masses.

Classification samples class balance independently of signal strength. Draw concentration  $\alpha \in \{.2, 1, 5\}$  with probabilities  $(.35, .40, .25)$  and  $u \sim \text{Dirichlet}(\alpha \mathbf{1}_K)$ . The desired prevalence is  $\pi_c = \delta + (1 - K\delta)u_c$ , with floor  $\delta = \min(2/n_s, 1/(2K))$ . Draw logit scale  $a = \text{LU}(.25, 8)$  and use base logits  $G = (0, as)$  for binary tasks or  $G_c = aF_c$  for multiclass tasks. Initialize  $b_c = \log \pi_c$  and perform exactly 64 calibration updates:

$$\bar{p}_c = \text{mean}_C \text{softmax}(G + b)_c, \quad b_c \leftarrow \text{clip}\{b_c + .5 \log[\pi_c / \max(10^{-8}, \bar{p}_c)], -12, 12\},$$

Recenter  $b$  to zero mean after each update. Calibration only approximates the desired prevalence, so diagnostics retain achieved prevalence and conditional entropy. Label-noise rate is  $q = 0$  with probability .6 and  $q = U(0, .15)$  otherwise. Draw labels independently from row probabilities  $(1 - q)\text{softmax}(G + b) + q\pi$ . Five percent of classification worlds are null: their logits before adding  $b$  are zero and target-to-feature paths are disabled. Apply one random permutation of class IDs per world.

The default P0/P1/P4 protocol declares  $K$  and its label alphabet before drawing rows. A declared class need not appear in support. The fixed classification head masks only logits outside the previously declared alphabet. For real-data evaluation, the permitted training protocol must supply the declared label universe; scored query outcomes cannot define it.

Comparisons that require every declared class in support can use the optional **reference-compatible-coverage** protocol. This is not the implemented default. It redraws support from the same world up to eight times, before generating any query. If all attempts fail, it can retry parameters at most four times within the same family, subfamily, head and shape, for five world attempts including the first. Exhaustion produces an explicit invalid episode. Queries are drawn only after support is accepted. This procedure conditions the distribution on support coverage, and its acceptance rate is reported. The default performs no coverage resampling.

Neither P0/P1/P4 standard-profile protocol uses query classes to rebalance, relabel or reject an episode. Finance has its own explicit importance-sampled training-query proposal in Section 9.6; finance evaluation keeps natural prevalence.

## A.7 P0: five conditional and joint mechanisms

P0 selects among the following five subfamilies with probabilities .20/.25/.30/.15/.10. Output heads share their input coordinates and structural sparsity but draw independent output coefficients. All use the common response compiler, which permits comparison of their mechanisms under the same response rules.

**Linear/GAM (.20).** These mechanisms combine additive effects and, optionally, sparse interactions. With probability .4, the score is exactly linear and has no interactions. Otherwise, each informative coordinate uses identity, tanh, hinge  $(v - k)_+$ , sine  $\sin(\omega v + \phi)$  or clipped square with probabilities  $(.4, .15, .2, .15, .1)$ . Hinge knots are calibration quantiles at  $U(.2, .8)$ . Sine parameters are  $\omega = \text{LU}(.5, 4)$  and  $\phi = U(0, 2\pi)$ . The square first clips  $v$  to  $[-5, 5]$ . Each basis is centered and scaled using calibration rows.

To vary relative effect sizes, draw exponent  $b = U(0, 2)$  and randomly permute coordinate ranks. Set  $w_j = \xi_j r_j^{-b}$  for  $\xi_j \sim N(0, 1)$ , then normalize  $w$  to unit Euclidean norm. Half of nonlinear worlds have no interactions. The rest choose  $I = \text{RU}(1, \min(6, \binom{d}{2}))$  distinct coordinate pairs, using zero pairs if  $d < 2$ . Add products of their centered and scaled bases with independent  $N(0, .5^2)$  coefficients, then pass the resulting  $F$  to the response compiler.

**Trees (.25).** Trees supply discontinuities and interactions. Draw tree count from  $\{1, 4, 16, 64\}$  with probabilities  $(.25, .25, .35, .15)$  and maximum depth from  $\{1, 2, 3, 4, 6\}$  with probabilities

(.10, .20, .30, .30, .10). Half the worlds use oblivious trees; the rest draw splits independently at binary nodes. A split chooses an informative coordinate uniformly. Numerical thresholds are calibration quantiles at  $U(.1, .9)$ ; categorical predicates use a uniformly chosen subset of size  $\text{RU}(1, C - 1)$ . Thirty percent of otherwise numerical predicates instead project up to three numerical coordinates with independent Gaussian coefficients normalized to unit norm. If fewer than two numerical coordinates exist, use the ordinary predicate.

An oblivious tree fits one predicate per level on the full calibration pool. An asymmetric tree fits a predicate using only calibration rows that reach its node and stops if fewer than eight arrive. Draw independent  $N(0, 1)$  leaf values for each output. Combine tree predictions using Gaussian weights normalized to Euclidean norm one. All thresholds and leaf values are fixed before generating support or query rows; the primitive fits neither labels nor boosted residual targets.

**Heterogeneous SCM (.30).** This subfamily generates  $X$  and the response jointly, allowing features and target to share causes and allowing features to descend from the target. With  $r = \min(d, 16)$  observed informative nodes, draw graph size  $M = \text{RU}(\max(8, r + 2), \max(16, 2r + 8))$ . Each node  $i > 0$  draws its parent count from  $\{1, 2, 3, 4\}$  with probabilities  $(.4, .3, .2, .1)$ , truncates that count to  $i$ , and chooses that many earlier nodes uniformly.

Choose target node  $t$  uniformly from  $1, \dots, M - 2$  before drawing rows. Select  $r$  distinct observed nodes other than  $t$ . With probability .7, require the observed set to contain one uniformly chosen parent of  $t$ , then select the remaining nodes uniformly. This requirement uses graph structure only, without inspecting target outcomes.

Each nonroot node uses a linear map with probability .25, an MLP with .35, a single depth-two axis-aligned tree with .25, or a 64-feature Gaussian RFF map with .15. A linear node with  $k$  parents draws weights from  $N(0, 1/k)$ . An MLP draws width 8/16/32 with probabilities .3/.4/.3, one/two hidden layers with .75/.25, and tanh/ReLU/sine activation with .5/.3/.2. Its weights have inverse-fan-in variance and its biases follow  $N(0, .5^2)$ . The tree is asymmetric and axis-aligned, has maximum depth two, and stops at nodes with fewer than eight calibration rows.

An RFF node uses all parents and 64 cosine features. It draws independent Gaussian frequencies divided by coordinate-specific length scales  $\ell_j = \text{LU}(.3, 3) \exp N(0, .35^2)$ , uniform phases, and Gaussian output coefficients divided by  $\sqrt{64}$ . Roots draw independent  $N(0, 1)$  values for each row. Ordinary nodes add independent  $N(0, .1^2)$  structural noise and then apply their calibration mean and scale.

Response generation occurs *at the target node*. That node emits the score dimension needed by the selected task and applies the response compiler there; no second response is generated afterward. Descendants of a regression target receive its generated value standardized by calibration moments. Descendants of a classification target receive a fixed  $N(0, 1)$  category embedding indexed by the generated class. Calibration proceeds in topological order, then freezes normalizers, thresholds, response intercepts and category embeddings. Support and query traverse the same graph with fresh row noise. Features can thus descend from the target or share its causes without directly copying a label.

Each observed node becomes categorical with probability  $r_c$ . Such a node draws cardinality and Zipf frequencies from the factory, then uses cumulative frequencies as calibration-quantile cut probabilities. Duplicate cuts and empty bins are allowed. Numerical nodes keep their values. Render the  $r$  observed nodes and fill the remaining width using the copy/distractor rule. In null worlds, use independent factory features and an independent target so that no descendant supplies target information. Graph nodes, the supervised target and output label encodings remain distinct.

**Local/kernel (.15).** These functions vary along a small subset of coordinates, chosen uniformly from the informative set with a maximum of eight. With probability .7, use 128 random Fourier features. Conditional on this choice, frequencies are Gaussian for an RBF kernel with probability .6 or multivariate

Student- $t_3$  with probability .4. For Student- $t_3$  frequencies, the independent Gaussian coordinates of each frequency vector share one  $\sqrt{\chi_3^2/3}$  denominator. Both versions use independent coordinate-specific length scales  $\ell_j = \text{LU}(.3, 3) \exp N(0, .35^2)$ , phases  $U(0, 2\pi)$  per feature, and Gaussian output coefficients divided by  $\sqrt{128}$ .

The other local branch draws 16 centers uniformly from calibration rows. With bandwidth  $h = \text{LU}(.3, 3)$ , assign radial weights  $\exp(-\|v - c_k\|^2/(2h^2))$  and normalize them across centers using log-sum-exp. Their weighted sum of fixed, independent Gaussian center values is the score. The actual query set has no role in selecting a nearest neighbor.

**Regime compositions (.10).** Different regions can follow different response functions. Draw regime count  $R = \text{RU}(2, 4)$ . With probability .7, assign regimes through a numerical informative coordinate split at calibration quantiles  $j/R$ . If all coordinates are categorical, randomly assign the first coordinate’s categories round-robin to  $R$  groups; some groups can be empty when cardinality is below  $R$ . With probability .3, use soft regime weights: Gaussian projections of up to three informative coordinates, divided by temperature  $\text{LU}(.2, 2)$  and passed through softmax.

Each regime adds an independent residual to a shared linear/GAM score. A residual is equally likely to be linear or a depth-two tree. The combined score is  $F = F_0 + A \sum_r g_r F_r$ , where  $A \in \{0, .5, 1, 2\}$  has probabilities (.2, .3, .3, .2). Choosing  $A = 0$  removes regime effects and supplies a control. The common response compiler determines final noise and prevalence.

## A.8 P1: hierarchical categorical effects

P1 repeats entity effects across rows so that evidence can accumulate within an entity and across entities in the same organization. Reserve four columns for nominal entity ID, nominal organization ID and two observed metadata coordinates. Draw  $E = \text{DLU}(8, \min(4096, \max(16, 2n_s)))$  entities and  $G = \text{RU}(2, \min(32, \max(2, \lfloor E/2 \rfloor)))$  organizations. Assign entities to organizations by a random permutation followed by round-robin placement. Each entity receives metadata  $z_e \sim N(0, I_2)$ .

With probability .2, entity frequencies are uniform. Otherwise they follow Zipf ranks with exponent  $U(.6, 1.8)$ , randomly assigned to entities. Calibration, support and query sample independently from this fixed frequency law. A query entity can therefore be unseen in support without any explicit holdout. Forced entity or group holdouts are disabled in this IID baseline.

The feature factory generates the other  $p - 4$  columns and a linear/GAM base score. For each output, add entity and organization effects:

$$F(x, e) = F_0(x) + b_e + u_{g(e)} + v_e^T v_{1:\min(2, d)}, \quad b_e = \tau \{ \sqrt{\omega} \beta^T z_e + \sqrt{1 - \omega} \epsilon_e \}.$$

Draw  $\beta$  as a two-dimensional Gaussian vector normalized to unit norm. Parameter  $\omega \in \{0, .5, 1\}$  has probabilities (.3, .4, .3) and controls the share of the entity intercept explained by metadata. Its overall scale  $\tau \in \{0, .25, 1, 2\}$  has probabilities (.15, .25, .4, .2). Organization-effect standard deviation is 0/.5/1 with probabilities .5/.3/.2. Slope standard deviation is 0/.25/.75 with probabilities .6/.3/.1. Draw independent Gaussian effects at these scales once per entity or organization. If an ordinary slope coordinate is categorical, use its encoded categorical effect.

Pass the complete score through the common response compiler. Independently permute ID codes and keep metadata observable. Reserved metadata columns undergo the same numerical rendering and cell masking as other numerical columns. Reserved IDs stay nominal and follow the same masking rule. Support and query share the sampled effects. Realized entity occupancy, unseen-query fraction and group-effect variances are returned only as diagnostics.

## A.9 P4: count and compound-amount outcomes

P4 generates counts and amounts accumulated over an exposure, such as a period at risk, while allowing event frequency and severity to vary separately. One observed column holds exposure in positive raw units before the common numerical rendering and masking. The count rate uses its latent value even when its observation is obscured. The factory generates the remaining  $p-1$  features and two independent standardized linear/GAM scores,  $s$  and  $s_V$ .

Draw exposure dispersion  $\sigma_E \in \{0, .5, 1\}$  with probabilities  $(.2, .5, .3)$ .

For each row, draw  $e = \text{clip}(\exp N(-\sigma_E^2/2, \sigma_E^2), .02, 50)$ . Draw desired calibration mean  $\mu_0 \in \{.2, 1, 5, 20\}$  with probabilities  $(.25, .35, .25, .15)$  and signal scale  $a = \text{LU}(.25, 2)$ . Calibration sets  $Z_C = \text{mean}_C[e \exp(\text{clip}(as, -4, 4))]$ , giving row mean

$$\mu(x) = \mu_0 e \exp(\text{clip}(as, -4, 4)) / \max(10^{-6}, Z_C).$$

Draw dispersion  $\kappa \in \{.5, 2, 10, \infty\}$  with probabilities  $(.2, .35, .25, .2)$ . For finite  $\kappa$ , use  $N^* \sim \text{NB}(\kappa, \kappa/(\kappa + \mu))$ ; for  $\kappa = \infty$ , use  $\text{Poisson}(\mu)$ . The negative binomial counts failures before successes and has mean  $\mu$  and variance  $\mu + \mu^2/\kappa$ .

A separate mechanism adds zeros. Draw world rate  $\zeta \in \{0, .2, .6\}$  with probabilities  $(.5, .35, .15)$ , then independently set  $N = 0$  for each row with probability  $\zeta$  and  $N = N^*$  otherwise. The count distribution can also produce zero, so this is a *zero-inflated* model, not a hurdle model.

Regression predicts count  $Y = N$  with probability .6 or compound amount with probability .4. For amount severity, draw  $\nu_0 = \text{LU}(1, 100)$ ,  $c \in \{-.5, 0, .5\}$  with probabilities  $(.2, .4, .4)$ , and severity shape  $k_V \in \{.5, 2, 10\}$  uniformly. Mean severity is  $\nu(x) = \nu_0 \exp(\text{clip}(cs + .5s_V, -4, 4))$ . When  $N > 0$ , draw

$$Y \sim \text{Gamma}(Nk_V, \nu(x)/k_V),$$

using shape-scale parameters; set  $Y = 0$  when  $N = 0$ . One such Gamma draw gives exactly the sum of  $N$  independent Gamma severities for any count. The construction requires no count cap, lognormal-sum approximation or query-dependent rejection.

Binary P4 tasks predict  $Y = \mathbf{1}[N > 0]$ . A fair coin drawn once per world can reverse the label IDs. P4 already generates stochastic outcomes, so it does not apply the P0 Gaussian noise compiler. Calibration records zero rate, mean and dispersion. Invalid numerical draws produce diagnostic failures instead of being replaced by zero.

## A.10 Generation order, shifts and validity checks

For a P0/P1/P4 episode, first choose the profile, family, head and shape, then draw world parameters and calibration rows. Calibration fixes function scales, predicates and response constants. Freeze the world, sample support, and apply only the permitted support conditions for acceptance. Draw independent query rows afterward. Apply the fixed rendering rules and masks, fit model preprocessing on support, and return model inputs separately from evaluator labels.

SCM calibration and row generation instead follow topological order, including response generation at the target node, while preserving the same separation of model inputs and evaluator information. A target’s descendants can already depend on its realized value. Applying a second independent label generator would therefore change the joint world.

All distribution-shift settings are **off** in the standard baseline. Their ablations distinguish changes in covariates from changes in measurement. Covariate shift chooses  $\min(d_{\text{num}}, \lceil d/4 \rceil)$  informative numerical coordinates and transforms query values as  $v'_j = a_j v_j + b_j$ , with  $a_j = \text{LU}(.7, 1.4)$  and  $b_j = U(-.5, .5)$ ,

before both score evaluation and rendering. This preserves the direct forward conditional mechanism. An empty selection is logged as a no-op. Measurement shift applies the same map only during rendering. Categorical-frequency shift replaces query frequencies with normalized  $\pi_c^\gamma$  for  $\gamma = \text{LU}(.5, 2)$  while retaining category effects and codes.

These three simple shifts are disabled in SCM worlds because changing generated graph nodes need not preserve the observational conditional. Root interventions need a separate specification and can change observed  $P(Y | X)$ . The first ablations test each shift individually. Independent reversal of the label mechanism is reserved for an uncertainty audit and excluded from ordinary static training.

P0/P1/P4 worlds are accepted when their arithmetic is valid and, if explicitly enabled, they satisfy the optional support-coverage condition. Null and constant-score worlds remain valid. No tree is fitted on scored queries to decide whether to retain a world. Checks also enforce shape limits and reference integrity. Diagnostics record null and constant-score draws, retries, masks, realized shapes and accepted task proportions.

Selecting parameters with a teacher would be a separate procedure. It would need an independent calibration world with its own support/query split, and could not inspect query labels from the episode later emitted for training. Teacher selection is therefore distinct from the base sampler’s validity checks.

## A.11 Context-conditioned complexity

The `static-v3-context` control tests whether the available support can identify the mechanisms drawn by `static-v3-base`. It keeps the same mixture and seed list. An independent 10% of candidate episodes follow the unchanged base law for replay and comparison. For the other 90%, replace informative count  $d$  by  $\min(d, \max(1, \lfloor n_s/8 \rfloor))$  and tree depth  $D$  by  $\min(D, \max(1, \lfloor \log_2(\max(2, n_s/4)) \rfloor))$ .

Apply the caps before drawing parameters in P0, P1’s ordinary feature mechanism and P4’s score mechanisms. In SCM worlds, the capped informative count also enters the rules for  $r$  and  $M$ . Width, categorical cardinality and target SNR remain fixed. For linear/GAM functions, the cap reduces active dimension. For a balanced full tree, the depth cap corresponds to about four observations per leaf. Unequal leaf probabilities can produce very different occupancy, which is measured using independent estimates of empty-leaf mass and learning curves. A tree-weighted ensemble diagnostic is also useful, but does not equal exact ensemble Bayes error.

When support size and mechanism complexity are independent, training can spend substantial effort on relations that the given context cannot identify. Caps may reduce that problem, but they narrow the prior and could impair transfer to large or sparse contexts. The comparison therefore matches sampled rows and training FLOPs, reports results by context length and task head, and retains the base arm.

Hierarchical effects make the identification issue concrete. Repeated categories reveal fixed shared effects; metadata determines what can be inferred for categories absent from support. Drawing a new effect for every row would eliminate the partial-pooling problem.

An optional *world-parameter selection* experiment could instead use teacher disagreement. Separately seeded diagnostic worlds, each with training and validation rows, would select a frozen parameter distribution for the next training tranche. The experiment would record diagnostic world IDs, candidate parameters, teacher versions, validation scores and selection probabilities. Its validation labels would be disjoint from all emitted training episodes and the final real-data evaluation.

This selection experiment needs an unselected comparison arm to test downstream transfer. Disagreement among synthetic teachers alone does not show that a world is learnable or useful. Teacher disagreement would not determine acceptance of scored queries and is not part of the nominal base sampler.

The linear special case motivates measuring difficulty at finite context size, but it does not justify a universal  $d \leq n_s/8$  rule. Sparse, regularized or compositional relations can remain learnable with many nominal variables. A 90% probability of choosing the capped branch also does not mean that 90% of episodes become easier: the rule changes only draws where a cap binds. Binding rates by task, stage and density, together with occupancy and teacher-learning curves, describe its actual effect. The uncapped reference remains the initial default for this control while each intervention is tested separately.

Conditional-label supervision asks a separate question about simulation noise. Exact integration removes one component of Monte Carlo variance while preserving the ideal expected objective. It does not establish improved transfer to real data.

## A.12 Conditional-law outputs and information boundaries

When a forward world has an analytic response law, an evaluator-only `label_law` hook can reduce simulation noise. P0’s four direct conditional subfamilies and P1 can return the full conditional class-probability vector before label-ID permutation, together with the permutation. They can also return an exact regression conditional mean when one is available. For binary P4, the unpermuted positive probability is

$$(1 - \zeta) \left[ 1 - \left( \frac{\kappa}{\kappa + \mu} \right)^\kappa \right]$$

for finite  $\kappa$ ; the Poisson case uses  $\exp(-\mu)$  in place of the power. Count and amount means are  $(1 - \zeta)\mu$  and  $(1 - \zeta)\mu\nu$ . The common regression compiler returns an exact mean only for the identity output transformation:  $A + Bs$  in nonnull worlds and  $A$  in null worlds. For other transformations it returns `unavailable`, rather than using an uncontrolled quadrature approximation.

These probabilities condition on the simulated latent state, which can contain more information than the learner sees. An observational prediction law would condition on the *actual learner-visible query information*. Masking, rounding and clipping can conceal score inputs, so  $P(Y | v, \theta)$  need not equal  $P(Y | X, \theta)$  after rendering. The exported quantity is named `latent_conditional_law` to make this distinction explicit.

The proposed training estimator takes the conditional expectation of ordinary hard-label loss given the full simulated state. It integrates only fresh query-outcome noise, leaving support labels, query inputs, predictions and the realized mask fixed. The resulting expected supervised loss remains unbiased. The latent state enters neither the predictor nor an observational Bayes oracle. P1’s hidden random effects obey the same distinction.

The first implementation disables this hook for *all* SCM worlds. If query features descend from a sampled target, averaging over forward target noise while holding those features fixed generally gives the wrong conditional distribution. The target’s forward law is not its posterior given descendants. Every arm retains hard sampled support labels and hard evaluation-query labels.

Soft conditional classification targets are therefore an optional estimator for training, to be compared with ordinary hard-label cross-entropy. They do not change the prior family, and their availability alone establishes no claim about benchmark performance.

The implemented `Episode` class restricts learner access with a `model_inputs()` allowlist. It returns support features and labels, query features and the declared task vocabulary. Query outcomes, conditional laws and hidden diagnostics remain outside that interface. Section 12.1 maps these records to the API. Target normalization uses support labels and retains the inverse transformation for scoring. Feature tables never include latent scores, graphs, entity effects, true rates, calibration labels or analytic laws.

Before a pilot is ready, deterministic replay and invariance checks must cover row order, category codes and query outcomes. In the P0/P1/P4 sampler, changing evaluator query outcomes must leave support acceptance, input values, fitted preprocessing and the model forward pass unchanged. Renaming unseen entities while preserving their generated metadata and effects must alter only nominal codes. A fresh diagnostic pool compares empirical P4 moments with the specified laws. These checks verify generation and information flow; downstream improvement still needs benchmark evidence.

## B Deferred architecture and training experiments

The methods below are possible follow-up experiments. They are not enabled in either architecture shown in Figure 1 or in the current standard and fraud training configurations. We retain their specifications so that each can be tested as an explicit change to the selected baseline. Their usefulness depends on measured predictive performance and cost.

### B.1 Optional feature feedback

The proposed feature-feedback experiment asks whether task inference partway through the trunk can recover detail lost in the initial row summary. Parameter counts below refer to the original nonfinance 315M control. The design retains stage-two feature cells and adds two untied refinement sites after ICL blocks eight and sixteen.

Each refinement reshapes the current row vector into eight width-128 slots and adds them to the saved summary tokens. A row MAB, a support-only column ISAB and a second row MAB update the representation. The new summaries are concatenated and added to the ICL row vector with multiplier  $\tanh(g)$ , where each site’s scalar gate starts at zero. Refined cells continue to the next site under the existing visibility masks.

Together the sites add eight MABs, two inducing banks and two gates, or 2,134,338 parameters. The resulting base has 317,155,794 parameters before finance extensions. Residual-output RMS gains start at  $1/\sqrt{16}$ .

The experiment tests recovery of feature information without claiming a novel mechanism. Paired runs use identical priors and report both equal-backbone and equal-GPU-hour comparisons. Ordinary frontend attention costs approximately  $O(FNSd + NF^2d)$  per column/row stage with  $S = 128$ ; each ICL block costs  $O((n_s^2 + n_s n_q)D)$  in attention with  $D = 1024$ , plus linear and FFN work. Feedback adds cell processing and storage. Flash-style kernels avoid storing full score matrices but leave quadratic arithmetic, so measured ROCm throughput at the planned shapes determines whether long-context training is affordable.

Before a substantial feedback run, implementation checks must cover query-label mutation invariance, query-batch independence, support-row permutations, episode packing, unseen categories, absent support classes, finite densities and analytic means, and exact parameter and optimizer partitions. These checks verify that feedback preserves the intended visibility and numerical behavior.

### B.2 Optional MoE: matched active work and extra capacity

MoE would test whether more stored FFN capacity improves prediction at approximately fixed active matrix work. It keeps the frontend, attention, heads and priors fixed, replacing only FFNs in one-indexed trunk layers 17–24. Each replacement has one shared and four routed bias-free SwiGLU experts of hidden width 1,408 and a bias-free  $4 \times 1024$  router. For the existing normalized FFN input  $z$ , the

proposed rule is

$$a = W_r z, \quad p = \text{softmax}(a), \quad j = \arg \max_e a_e, \quad F(z) = F_s(z) + 4p_j F_j(z).$$

Router logits and softmax use FP32 at temperature one, resolving exact ties by the lowest index. Routing acts on individual tokens, without jitter, dropout, capacity limits, token dropping, quota rerouting or batch-dependent normalization. The combined result passes through the existing residual-output RMSNorm. Activating one shared and one selected half-width expert matches the matrix arithmetic of the original width-2,816 FFN, with router overhead added.

The factor four gives the routed half unit gain under uniform routing and bounds its selected multiplier between one and four. This initialization choice must be assessed experimentally. Away from ties, training differentiates through the *full* four-expert softmax:

$$\frac{\partial(4p_j)}{\partial a_e} = 4p_j(\mathbf{1}\{e = j\} - p_e)$$

Normalizing only the selected singleton would give it weight one and remove this task-loss gradient to the router. Switch routing, shared experts, router z-loss and dense upcycling have precedents [38, 39, 40]; the particular configuration above is a proposed experiment.

| Candidate                               | Stored parameters | Nominal active accounting |
|-----------------------------------------|-------------------|---------------------------|
| Dense ICL-315                           | 315,021,456       | 315,021,456               |
| ICL-MoE-419                             | 418,863,248       | 315,054,224               |
| Dense final-eight FFNs widened to 7,040 | 418,830,480       | 418,830,480               |

Table 12: Base-model counts, excluding the 153,728-parameter finance extension. Active accounting includes all non-MoE parameters conventionally; it is not a complete forward FLOP count or a latency measurement.

A SwiGLU has  $3dh$  weights. Each half expert therefore has 4,325,376 weights and each router 4,096, adding  $8[5(4,325,376) + 4096 - 8,650,752] = 103,841,792$  stored weights across the replacements. Different rows may collectively use all experts, so replicated gradients and optimizer states scale with total stored capacity. Smaller dispatched GEMMs may also be less efficient. Attention still contains the  $n^2 + nq$  term; this MoE design does not reduce long-context attention work.

**Initialization and optimizer control.** For each seed, all three arms start from a common dense prefix whose training cost is included in the budget. For every replaced FFN, a fixed random permutation is applied to gate/value rows and matching down-projection columns. Half the resulting channels initialize the shared expert, and the other half initialize each of the four routed experts. Router entries use standard deviation  $10^{-3}/\sqrt{1024}$ . The conversion approximately preserves the function, preserving it exactly only at uniform zero logits. We measure the conversion difference.

The wide dense control keeps the original channels and appends 4,224 independently initialized gate/-value rows with variance  $1/1024$ . Their down-projection columns start at zero, exactly preserving the initial function without permanently identical duplicated channels.

All three continuation arms initially use AdamW so the architecture comparison does not also change optimizer geometry. They reset optimizer state and use peak learning rate  $3 \cdot 10^{-4}$ , betas (0.9, 0.95), epsilon  $10^{-8}$ , projection decay 0.01, no decay for norms/embeddings/scalars, global clipping at 10 and the specified 2% warmup/cosine schedule. Muon orthogonalizes half-size expert matrices differently from the original dense matrices; introducing it simultaneously would confound the comparison. A later experiment can test Muon on the preferred architecture, with AdamW retained for routers.

**Explicit balancing scope.** Load balancing uses the *synchronized distributed microbatch*, weighting support and queries so that their sizes do not determine an episode’s contribution. With  $B_m$  complete episodes, a support row in episode  $b$  has weight  $1/(2B_m n_{s,b})$  and a query row has weight  $1/(2B_m n_{q,b})$ ; padding contributes nothing. Each MoE layer computes the following global microbatch statistics:

$$f_e = \sum_t w_t \mathbf{1}\{j_t = e\}, \quad P_e = \sum_t w_t p_{t,e}, \quad Z = \sum_t w_t \left[ \log \sum_e \exp(a_{t,e}) \right]^2.$$

Hard assignments have no gradient. Soft probabilities retain gradients through distributed reductions scaled consistently with the loss. The auxiliary term is  $0.01 \text{mean}_\ell[4 \sum_e f_{\ell e} P_{\ell e}] + 0.001 \text{mean}_\ell Z_\ell$ , with each accumulated microbatch loss weighted by  $B_m/256$ . The layer reduction averages the eight layers.

This averages separate microbatch balancing losses, which differs from applying the nonlinear balancing loss to one combined 256-episode batch. Paired runs therefore record and fix microbatch packing. Balancing is not imposed separately within tasks or classes, and inference omits the auxiliary term.

The proposed deployment of the roughly 419M model replicates every expert on every MI355X under data parallelism. It communicates gradients without inter-node expert dispatch. Tokenwise routing without dropped tokens preserves support caching and independent query chunks; capacity limits or query-batch load adjustments would not. AMD’s AITER repository provides ROCm/MoE operators and lists MI355X support [41]. A fused kernel would be chosen only after checking forward/backward correctness and speed for these small expert shapes.

**Budget and decision.** The proposed screen compares MoE with D315 at similar active FFN arithmetic and dense-wide419 at similar stored capacity. Three arms, two paired seeds and 250 GPU-hours of continuation per arm/seed total 1,500 GPU-hours. The revised allocation assigns this experiment zero hours, so prefix training and continuation both require a documented budget reassignment before launch.

Results will report accepted episodes and actual GPU-hours. Selection requires better real development quality at measured batch cost; synthetic loss or routing entropy alone is insufficient. A successful MoE model could occupy an already funded confirmation slot. Long-context attention changes must first be tested separately.

### B.3 Deferred local/global attention and cache experiments

The deferred *local/global attention path* keeps individual support rows for local evidence and uses learned global summaries to carry information between regions. It retains the frontend, task heads, priors, width and 24-layer trunk, with full dense attention as the reference. Keeping individual rows can help rare examples, but their contextual states differ from dense attention, so predictions need not match. Inducing summaries and feature retrieval are established methods [42, 24]; this combination still needs a controlled accuracy comparison.

**Routing index, built from support features only.** Routing uses observed features alone, allowing selection to be reproduced without labels or task-conditioned hidden states. The 128-dimensional index combines numerical support-rank coordinates  $2u-1$ , missing flags and feature-specific nominal-category one-hot indicators with a missing level. A fixed seeded sign projection, with entries  $\pm 1/\sqrt{128}$ , maps this implicit representation before division by  $\sqrt{F}$ .

A stable hash of the index seed, feature identity, category identity and output coordinate supplies categorical projection signs without a materialized one-hot matrix. Numerical coordinates and missing

flags use separate hash identities. Queries use support-fitted ranks and the frozen projection, including for unseen categories. Neither target labels, label-conditioned frontend states nor generator metadata enter the index. Evaluation must still test failures caused by irrelevant features and hash collisions.

We partition support indices deterministically to define local key sets. Each node splits on the coordinate with greatest within-node population variance, resolving variance ties by the lowest coordinate index. Stable sorting uses persistent random support IDs to break value ties; these IDs are independent of values and labels. A split at  $\lfloor n/2 \rfloor$  recurses until every leaf has at most  $b = 2048$  rows. IDs stay with records during permutation tests and cache reuse, and each leaf stores its centroid and stable ID.

Each estimator also samples  $\min(256, n_s)$  uniform support anchor IDs without replacement, independently of queries. A query selects the two nearest leaf centroids by squared Euclidean distance, resolving ties by leaf ID, or all leaves if fewer than two exist. Queries on the same route can share GPU execution while retaining exactly the same key sets.

**Layer computation and parameters.** The global path starts from a trainable  $G_0 \in \mathbb{R}^{128 \times 1024}$  initialized with standard deviation 0.02. Its 131,072 parameters bring this variant of the original base control to **315,152,528**. Before local updates, layer  $\ell$  aggregates all support evidence into global summaries:

$$G_\ell = \text{MAB}_\ell(G_{\ell-1}, S_{\ell-1}),$$

Here  $S_{\ell-1}$  is the previous-layer state of all support rows. The same layer’s MAB then updates each support row using the previous states in its own leaf, previous states of global anchors and current  $G_\ell$ . Queries read two selected leaves, the same anchors and  $G_\ell$ , but never serve as keys or values.

Global and ordinary substeps share Q/K/V/output/FFN/norm parameters within a layer; parameters remain untied across depths. Each MAB call includes its usual FFN and residual norms. Global tokens receive no artificial labels or extra role embeddings.

Before softmax, deduplication gives each support row only one entry even if it appears in both an anchor set and a selected leaf. The 128 learned global slots stay distinct, and the existing length scale uses the actual visible key count. Globals are learned representations, with no claim to be sufficient statistics or unbiased subsamples, so there is no log-mass correction, multiplicity weighting or extra loss. Each layer caches support and global K/V. Queries remain independent while receiving information from outside their selected leaves through the globals.

Including global aggregation and local attention bounds the per-layer attention pairs by

$$n_s(b + R + 2g) + n_q(2b + R + g), \quad b = 2048, \ R = 256, \ g = 128,$$

This includes the step where global summaries read all support rows. Dense attention has  $n_s^2 + n_s n_q$  pairs. At  $n_s = 32768$ , the bound gives 12.8 times fewer support pairs and 7.31 times fewer query pairs. These ratios count pairs, not speed: frontend work, projections/FFNs, routing, gathers, padding and global-token computation also take time.

This full-head variant retains about 96 KiB of core cache per support row and adds 12 MiB of global K/V. It reduces attention work but keeps the full support cache.

**Train the graph that will be used.** The proposed model would train on its deployment graph, rather than first introducing local selection after dense-only pretraining. Independently of query outcomes, training uses the local/global path on 20% of episodes with  $n_s \leq 8192$  and all episodes above 8,192; other episodes use dense attention. Episode-mean supervised loss, family weights, heads and the shape curriculum stay fixed. Inference uses dense attention for  $n_s \leq 8192$  and the hybrid graph beyond

that, with full-dense inference measured separately. Routing is nondifferentiable feature preprocessing without a straight-through estimator.

Two paired seeds and 250 GPU-hours per arm/seed would make the dense-versus-hybrid comparison cost 1,000 hours. Its current allocation is zero: finance-context comparisons received the former allowance. Launch requires an explicit reassignment within the 50,000 GPU-hour ceiling. MoE stays disabled during this test. If both changes help independently, an allocated confirmation run must test their combination.

Alongside aggregate real benchmarks, evaluation tests rare classes and regions, distractor features, global additive mechanisms and localized discontinuities. Dense attention with reduced support provides a control at equal end-to-end time. Full-context dense inference remains available when routing misses relevant evidence or lowers accuracy.

**Additional efficiency experiments that remain separate.** As context grows, irrelevant keys can dilute useful attention. The legacy length scale stops changing above 4,096 keys because its log-length argument is clipped at log 4. The implemented candidate instead uses Section 7.4’s logarithmic scale; increasing the bounded clip is another possible control. Dilution tests and stability measurements at the longest contexts are needed to assess extrapolation. A learned temperature by itself does not establish it.

For a simple dilution example, let one useful anchor have fixed logit advantage  $\Delta$  over  $n - 1$  irrelevant keys. Its attention mass is  $[1 + (n - 1)e^{-\Delta}]^{-1}$ , and mass at least  $\alpha$  requires  $\Delta \geq \log((n - 1)\alpha/(1 - \alpha))$ . A gap giving mass 0.5 at 4,096 keys gives only 0.111 at 32,768. The model may learn a different gap, so this is a stress test rather than an impossibility result.

The audit prioritized length scaling and dilution diagnostics over MoE. The revised allocation nevertheless defers the dedicated GPU comparison and keeps the current logarithmic rule fixed during prior selection. Any experiment named QASSMax must reproduce the released rule exactly: the clipped control and implemented logarithmic alternative are different mechanisms [8].

An expensive support prefill could provide more supervision if it served more scored queries. At 32,768 support rows, increasing queries from 128 to 512 adds only  $384/32768 = 1.171875\%$  of the dense support-square pair count. Those gradients share a world and are correlated. The comparison must therefore retain episode-mean loss, match GPU-hours, count unique worlds and targets separately, and recompute cell-cap conditioning for the actual query count.

Unlike frozen-weight inference, this training experiment cannot reuse detached K/V. Query losses must differentiate through support states and parameters, with the shared graph accumulated or recomputed before the optimizer step. Independent predictions at deployment do not imply independent training gradients.

If profiling finds cache bandwidth limiting, a separate experiment could use 16 query heads and four KV heads in the original nonfinance 315M control. This version reduces K/V projection size, unlike the implemented query-only grouping. Its trunk cache would be fourfold smaller—0.75 GiB at 32,768 support rows—while its 16 query heads retain quadratic pair arithmetic. The projection reduction is  $24 \cdot 2(1024^2 - 1024 \cdot 256) = 37,748,736$ , leaving 277,272,720 stored parameters with other components fixed.

Each KV head would start from the average of a consecutive group of four dense head projections, followed by continuation training before evaluation. This changes function and capacity; language-model GQA results do not establish retained tabular accuracy [43]. The variant is outside the initial hybrid/MoE comparisons unless profiling supports a documented budget reassignment.

**What bounded context can lose.** Selection can remove outcome-relevant evidence even when the model is trained correctly for the selected contexts. For a deterministic selector  $S$ , the optimal log-loss risk difference between complete labeled history  $D$  and its selected representation is

$$H(Y_q | X_q, S(D)) - H(Y_q | X_q, D) = I(Y_q; D | X_q, S(D)) \geq 0.$$

With randomized selection, the identity additionally conditions on public routing and selection randomness. It quantifies information removed by selection; it does not imply that every finite full-history model beats every bounded model. Training on the selector cannot recover evidence the selector discards. The 7,168-row finance context is thus an empirical cost/accuracy choice, without a sufficiency claim.

We compare reservoir/positive, nearest-center and feasible query-specific selectors using the same frozen checkpoint, together with full-history trees given all permitted evidence. Separate timings for cold history preparation, route construction, each prefill and warm prediction make the cost of changing the available evidence explicit.

**Cache correctness and lifecycle.** A cache remains valid only while all inputs to its construction are unchanged: support values and labels, feature types and order, fitted codec, declared label mapping, model and kernel precision, augmentation and index seeds, routing IDs and attention configuration. Adding or changing a support row can alter preprocessing, inducing summaries, routes and mutually contextualized support states. The ordinary full cache must then be invalidated; no exact incremental insertion algorithm is specified.

Correctness checks precede timing comparisons. They verify independent query chunks, cached/uncached agreement, support permutations with persistent IDs, global-update order, route deduplication and episode isolation. These establish the conditions under which cache reuse preserves the specified prediction.

## B.4 Optional consistency and masked-conditional objectives

The starting candidate uses only supervised task scores. The optional consistency study described here applies to the histogram control. It gives 25% of episodes a second view by permuting features before grouping, reseeding category codes and drawing another injection of declared labels into 256 output slots. We align predictions by semantic class and average the two supervised losses. The consistency coefficient increases from zero at  $0.2T$  to 0.05 at  $0.4T$ , then stays fixed.

For classification, consistency is symmetric stop-gradient KL. For regression, it is KL over the 130 bin masses plus 0.1 times the mean squared difference of the two log tail scales, under a shared target transform. This regularizes consistency but is not KL between full continuous densities. Removing an informative feature is not assumed to preserve predictions. InfoNCE stays disabled because rows or classes from different synthetic mechanisms need not be valid negatives.

A separate masked-conditional study would add an auxiliary task to 10% of episodes at weight 0.1. It uniformly selects an observed feature among those eligible by support availability, removes that feature from both input tensors and uses it as the auxiliary target. The copy discards the original  $y$  and refits preprocessing after removal.

A categorical auxiliary target needs an independently declared complete vocabulary of at most 256 values. Support-only vocabulary cannot guarantee that query targets are legal, and query values may not determine eligibility or skipping. An ineligible copy is skipped while its original episode remains. This objective stays disabled unless equal-compute comparisons show a benefit.

## B.5 Optional adaptive prior sampling

The initial run fixes family weights. The optional adaptive sampler would examine each active subfamily every 2% of the run budget on 256 independent candidate worlds. Each calibration episode retains its sampled support size and class-size constraint and uses  $n_q = \min(256, \max(16, \lfloor n_s/4 \rfloor))$ . A fixed teacher portfolio—constant, linear/logistic, shallow-tree and nearest-neighbor predictors—receives the model’s feature interface. It is scored by classification cross-entropy or support-standardized regression MSE.

The adaptation score measures whether a teacher exposes room for improvement: subtract the best teacher’s calibration loss from the current PFN’s, then divide by the constant predictor’s loss floored at  $10^{-3}$ . Subfamily means are shrunk toward zero by  $m/(m + 256)$  and clipped to  $[-1, 1]$ . Within each family, weights then update as

$$\tilde{w}_j \propto w_j^{(0)} e^{0.5g_j}, \quad w_j^{\text{new}} = 0.9w_j + 0.1\tilde{w}_j.$$

We project the new weights onto the probability simplex with bounds of one-quarter to four times each initial active weight, finding the Euclidean projection’s clipping offset by bisection. A 10% stream retains the original fixed distribution. The complete update policy is frozen before confirmation. This is a conservative proposed configuration within the performance-driven mixture adaptation studied by RTFM and DoReMi [27, 28], not a novelty claim.

Teacher preferences may bias adaptation, so the unchanged-weight control and adaptive sampler are compared on real, lineage-disjoint development tasks. Calibration worlds stay separate from scored worlds: using a scored query label to decide whether its episode is emitted would change the distribution in an uncontrolled way. Evaluation considers whole-task transfer and classification/regression separately, rather than treating low raw loss or predictor disagreement as the end goal.

## C Earlier screening allocation and controls

**Previous allocation.** The table and controls below preserve the previous screening plan. Section 5’s current program replaces it with phase budgets of 4,800, 1,800, 3,200, 2,800 and 2,400 GPU-hours. Both plans cannot run within the same allocation. These older experiments remain available only through an explicit budget reassignment.

| Earlier screen                                                                              | Arithmetic              | GPU-hours |
|---------------------------------------------------------------------------------------------|-------------------------|-----------|
| Prior triage: R, R+P1, R+P4, authored replacement                                           | $4 \times 2 \times 250$ | 2,000     |
| Prior/backbone crossing: two priors; released, authored compressed, compact persistent-cell | $6 \times 2 \times 350$ | 4,200     |
| Regression head: reference quantiles versus finer histogram                                 | $2 \times 2 \times 150$ | 600       |
| Category codec: scalar Fourier versus hash-bit candidate                                    | $2 \times 2 \times 150$ | 600       |
| Length scaling: current cap versus one chosen alternative                                   | $2 \times 2 \times 150$ | 600       |
| Shared versus classifier/regressor checkpoint packages                                      | $2 \times 2 \times 200$ | 800       |
| Muon/AdamW versus all-AdamW, including rate triage                                          | $2 \times 2 \times 150$ | 600       |
| Six finance controls                                                                        | $6 \times 2 \times 200$ | 2,400     |
| At most two complete combinations, two new seeds each                                       | $2 \times 2 \times 800$ | 3,200     |
| Total                                                                                       |                         | 15,000    |

Each row multiplies arms, seeds and *total package* hours. Initially, task-specific packages divide time equally between classification and regression under a fixed binary/multiclass rule. All checkpoints share the package allowance.

The prior/backbone experiment fixes heads, supported classes, feature/target interface and optimizer convention. Reference-generator and head adapters are executable. When this plan was written, faithful native-backbone and compact CAST training interfaces still needed implementation. The current **persistent\_small** comparator is implemented but does not claim to reproduce CAST. Cross-family capacity differences remain part of the system comparison. Smaller head, codec and attention studies require fresh paired controls; a partial checkpoint from a longer cosine schedule cannot substitute for a completed run at the comparison horizon.

The second prior in the factorial comparison would be a graft supported by triage, or the replacement bank if a recorded decision assigned it that role. If neither is supported, R remains and unused comparison cells receive no automatic funding. A combined P1/P4 graft likewise has no automatic allowance.

The readout comparison tests finer histograms against reference quantiles, retaining the coarse histogram as an inexpensive falsification control. An analytic stress test would choose one length-scaling alternative for paired GPU runs. Short learning-rate trials all fit within the optimizer study’s 600 hours.

The six historical finance controls keep the legacy class-conditional mechanism, population seeds, arrival process and observation law fixed except for each named intervention. The risk-partition candidate enters complete-configuration confirmation and specialist runs. A matched legacy run within those slots would be needed to isolate its mechanism.

The controls comprise standard-only F0; joint 80/20 training with full selection and unit normalization F1; entropy-normalized F2; F2 with H removed from selector and model F3; F2 with a nondiscriminative generative H law F4; and reservoir-plus-positive F5 with normalization fixed before launch. Every applicable arm retains population importance weights. Identifying F5’s selection effect requires a comparator with the same normalization.

In F0, finance-only adapters need consistent neutralization so untrained metadata parameters do not introduce random effects. Results separate reference-present and reference-absent worlds. The two final-confirmation slots pay for a challenger and its required full-selector control, rather than two challengers plus another control.

This earlier plan would confirm at most two complete configurations, including required paired controls, on finance and all three ordinary task types. Testing the combination matters because separate gains need not add. Short runs guide triage without proving final superiority. If measured throughput cannot cover intended contexts in 150–350 hours, arm counts or allocations must change before launch. A partial early curriculum in one arm cannot be compared with a completed curriculum in another.

Costs include generation, indexing, calibration, every routed prefill and failed allocations. Reference generation, candidate heads/codecs and grouped-query caches are runnable. The compact persistent-cell model and paired standard-panel evaluator, still pending when this allocation was drafted, are now implemented. The complete statistical and cross-suite confirmation procedure remains future work.

MoE, general sparse attention, adaptive priors and auxiliary losses remain deferred. Implemented query-only KV grouping is tested within the existing architecture/confirmation allowance. Final training shares 12,000 hours across 9,000 standard hours and 3,000 fraud hours; each checkpoint does not receive a separate allowance.

## References

- [1] Hollmann et al. *TabPFN: A Transformer That Solves Small Tabular Classification Problems in a Second*. ICLR 2023. Supplied `tabpfn.pdf`. <https://arxiv.org/abs/2207.01848>.
- [2] Hollmann et al. *Accurate Predictions on Small Data with a Tabular Foundation Model*. Nature 637, 2025. Supplied `tabpfn2.pdf`. <https://doi.org/10.1038/s41586-024-08328-6>.
- [3] Prior Labs Team. *TabPFN-2.5: Advancing the State of the Art in Tabular Foundation Models*. 2025. Supplied `v2, tabpfn2.5.pdf`. <https://arxiv.org/abs/2511.08667>.
- [4] Prior Labs Team. *TabPFN-3: Technical Report*. 2026. Supplied `tabpfn3.pdf`. <https://arxiv.org/abs/2605.13986>.
- [5] Jäger et al. *TabPFN-3.5: Technical Report*. September 2026. Supplied `tabpfn3.5.pdf`. <https://arxiv.org/abs/2609.17895>.
- [6] Qu et al. *TabICL: A Tabular Foundation Model for In-Context Learning on Large Data*. ICML 2025. Supplied `tabicl.pdf`. <https://github.com/soda-inria/tabicl>.
- [7] Qu et al. *TabICL official pretraining scripts and release notes*. Accessed October 3, 2026. <https://github.com/soda-inria/tabicl#pre-training>.
- [8] Qu et al. *TabICLv2: A Better, Faster, Scalable, and Open Tabular Foundation Model*. 2026. Supplied `v2, tabicl2.pdf`. <https://arxiv.org/abs/2602.11139>.
- [9] Kong et al. *TabFM: A Zero-Shot Foundation Model for Tabular Data*. September 29, 2026. Supplied `tabfm.pdf`. <https://arxiv.org/abs/2609.37959>.
- [10] Xiaomi-TabLDM Team. *Xiaomi-TabLDM: A Tabular Foundation Model Technical Report*. September 2026. Supplied `v2, xiaomi.pdf`. <https://arxiv.org/abs/2609.03880>.
- [11] Eo et al. *EXAONE Tabular 1.0: Technical Report*. August 2026. Supplied `exaone.pdf`. <https://arxiv.org/abs/2608.25774>.
- [12] NVIDIA. *NVIDIA Kumo Tabular Sets a New Accuracy-Efficiency Frontier for Tabular Prediction*. September 29, 2026. Supplied print `kumotabular.pdf`. <https://huggingface.co/blog/nvidia/kumo-tabular>.
- [13] *Mitra-v2 Technical Report*. September 2026. Secs. 2.2–2.4, Table 2, Appendix C. <https://arxiv.org/abs/2609.04540>.
- [14] Bouadi et al. *Shaping the Prior: How Synthetic Task Distributions Determine Tabular Foundation Model Quality*. 2026. Sec. 3.1, Table 2. <https://arxiv.org/abs/2605.18971>.
- [15] Davies et al. *Mind the Gap? A Distributional Comparison of Real and Synthetic Priors for Tabular Foundation Models*. 2026. <https://arxiv.org/abs/2605.06343>.
- [16] Türkmen et al. *Towards Evaluating Data Priors for Tabular Foundation Models*. 2026. Secs. 3–4. <https://arxiv.org/abs/2606.29241>.
- [17] Zhao et al. *From Synthetic Priors to Model Behavior: Structural Coverage in Tabular Foundation Models*. September 2026. <https://arxiv.org/abs/2609.06912>.
- [18] Qu et al. *Official TabICL checkpoints*. Metadata inspected October 2, 2026; exact revisions in the model-scale audit. <https://huggingface.co/jingang/TabICL>.
- [19] NVIDIA. *Kumo Tabular official checkpoints*. Metadata inspected October 2, 2026. <https://huggingface.co/nvidia/Kumo-Tabular>.
- [20] Google. *TabFM 1.1.0 official PyTorch checkpoints*. Metadata inspected October 2, 2026. <https://huggingface.co/google/tabfm-1.1.0-pytorch>.

- [21] Google. *TabFM 1.0.0 official PyTorch checkpoints*. Metadata inspected October 2, 2026. <https://huggingface.co/google/tabfm-1.0.0-pytorch>.
- [22] LimiX Team. *LimiX-2: A Contextual Mechanism Network Towards General Structured-Data Intelligence*. September 2026. <https://arxiv.org/abs/2609.17488>.
- [23] Layer 6. *TabDPT official checkpoints*. Version 1.3 metadata inspected October 2, 2026. <https://huggingface.co/Layer6/TabDPT>.
- [24] Ma et al. *TabDPT: Scaling Tabular Foundation Models on Real Data*. NeurIPS 2025. <https://arxiv.org/abs/2410.18164>.
- [25] Shaheen et al. *Understanding the Surprising Generalization Properties of Tabular Foundation Models*. August 2026. <https://arxiv.org/abs/2608.17957>.
- [26] Biswas and El Karoui. *Distillation of Synthetic Data for Time Series Foundation Models*. September 2026, v2. <https://arxiv.org/abs/2609.09586v2>.
- [27] Peroni et al. *Robust Tabular Foundation Models*. 2025. <https://arxiv.org/abs/2512.03307>.
- [28] Xie et al. *DoReMi: Optimizing Data Mixtures Speeds Up Language Model Pretraining*. NeurIPS 2023. <https://arxiv.org/abs/2305.10429>.
- [29] AMD. *AMD Instinct MI355X GPUs*. Official specifications, accessed October 2, 2026. <https://www.amd.com/en/products/accelerators/instinct/mi350/mi355x.html>.
- [30] Zhang et al. *Mitra: Mixed Synthetic Priors for Enhancing Tabular Foundation Models*. NeurIPS 2025. <https://arxiv.org/abs/2510.21204>.
- [31] Hoffmann et al. *Training Compute-Optimal Large Language Models*. NeurIPS 2022. <https://arxiv.org/abs/2203.15556>.
- [32] Erickson et al. *TabArena: A Living Benchmark for Machine Learning on Tabular Data*. NeurIPS 2025. <https://arxiv.org/abs/2506.16791>.
- [33] AutoGluon team. *TabArena repository: model/system protocols and TabArena-Lite*. Accessed October 2, 2026. <https://github.com/autogluon/tabarena>.
- [34] LAMDA-Tabular. *TALENT repository: datasets, evaluation, and tuning policies*. Accessed October 2, 2026. <https://github.com/LAMDA-Tabular/TALENT>.
- [35] LimiX team. *LimiX repository and BCCO dataset releases*. Accessed October 2, 2026. <https://github.com/limix-ldm-ai/LimiX>.
- [36] Landsgesell et al. *ScoringBench implementation and benchmark data*. Accessed October 2, 2026. <https://github.com/jonaslandsgesell/ScoringBench>.
- [37] Dao et al. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS 2022. <https://arxiv.org/abs/2205.14135>.
- [38] Fedus et al. *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity*. JMLR 2022. <https://arxiv.org/abs/2101.03961>.
- [39] Zoph et al. *ST-MoE: Designing Stable and Transferable Sparse Expert Models*. 2022. <https://arxiv.org/abs/2202.08906>.
- [40] Komatsuzaki et al. *Sparse Upcycling: Training Mixture-of-Experts from Dense Checkpoints*. ICLR 2023. <https://arxiv.org/abs/2212.05055>.
- [41] AMD/ROCm. *AITER: AI Tensor Engine for ROCm*. Official operator repository, accessed October 2, 2026. <https://github.com/ROCm/aiter>.

- [42] Lee et al. *Set Transformer: A Framework for Attention-based Permutation-Invariant Neural Networks*. ICML 2019. <https://arxiv.org/abs/1810.00825>.
- [43] Ainslie et al. *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*. EMNLP 2023. <https://arxiv.org/abs/2305.13245>.
- [44] Jesus et al. *Turning the Tables: Biased, Imbalanced, Dynamic Tabular Datasets for ML Evaluation*. NeurIPS 2022. [https://proceedings.neurips.cc/paper\\_files/paper/2022/file/d9696563856bd350e4e7ac5e5812f23c-Paper-Datasets\\_and\\_Benchmarks.pdf](https://proceedings.neurips.cc/paper_files/paper/2022/file/d9696563856bd350e4e7ac5e5812f23c-Paper-Datasets_and_Benchmarks.pdf).
- [45] Dal Pozzolo et al. *Credit Card Fraud Detection: A Realistic Modeling and a Novel Learning Strategy*. IEEE TNNLS. Author manuscript. [https://dalpozz.github.io/static/pdf/TNNLS\\_2017.pdf](https://dalpozz.github.io/static/pdf/TNNLS_2017.pdf).
- [46] Le Borgne et al. *Reproducible Machine Learning for Credit Card Fraud Detection: Practical Handbook*, validation strategies. [https://fraud-detection-handbook.github.io/fraud-detection-handbook/Chapter\\_5\\_ModelValidationAndSelection/ValidationStrategies.html](https://fraud-detection-handbook.github.io/fraud-detection-handbook/Chapter_5_ModelValidationAndSelection/ValidationStrategies.html).
- [47] Elkan and Noto. *Learning Classifiers from Only Positive and Unlabeled Data*. KDD 2008. <https://cseweb.ucsd.edu/~elkan/posonly.pdf>.
- [48] Fithian and Hastie. *Local Case-Control Sampling: Efficient Subsampling in Imbalanced Data Sets*. Annals of Statistics, 2014. <https://arxiv.org/abs/1306.3706>.
- [49] Gorishniy et al. *TabR: Tabular Deep Learning Meets Nearest Neighbors*. ICLR 2024. <https://arxiv.org/abs/2307.14338>.
- [50] LightGBM authors. *Official parameter reference*. <https://lightgbm.readthedocs.io/en/latest/Parameters.html>.
- [51] XGBoost authors. *Official parameter reference*. <https://xgboost.readthedocs.io/en/stable/parameter.html>.
- [52] CatBoost authors. *Common training parameters*. <https://catboost.ai/docs/en/references/training-parameters/common>.
- [53] Prior Labs. *TabPFN-3.5 implementation*, revision 15f5e6b, inspected October 3, 2026. [https://github.com/PriorLabs/TabPFN/blob/15f5e6b2b629b905879b9be907261416f20d0df5/src/tabpfn/architectures/tabpfn\\_v3\\_5.py](https://github.com/PriorLabs/TabPFN/blob/15f5e6b2b629b905879b9be907261416f20d0df5/src/tabpfn/architectures/tabpfn_v3_5.py).
- [54] NVIDIA. *Kumo Tabular ICL implementation*, revision b982fbf, inspected October 3, 2026. <https://github.com/NVIDIA/structured-data-models/blob/b982fbf4188ea8db17249fe81338723106100b63/sdm/models/kumo/tabular/icl.py>.
- [55] Xiaomi. *Xiaomi-TabLDM public source*, revision c87efdd, inspected October 3, 2026. <https://github.com/xiaomi-research/xiaomi-tabldm/tree/c87efddedb244d1a21ac07b40b1453466329e5c5>.
- [56] AutoGluon. *Mitra-v2 fine-tuning source*, revision b4701e8, inspected October 3, 2026. <https://huggingface.co/autogluon/mitra-finetune/tree/b4701e8148dc33b00ed15d7086ff59816957cde4>.
- [57] Zhao et al. *Architecture Alignment With Sparse Priors in Tabular Foundation Models*. September 2026. <https://arxiv.org/abs/2609.36883>.
- [58] Luo et al. *Memory Efficient Tabular Foundation Models*. July 2026. <https://arxiv.org/abs/2607.27546>.
- [59] Park et al. *Generating High-Diversity Synthetic Tabular Data via Less-Constrained Prior*. IJCAI 2026, pp. 4714–4722. <https://www.ijcai.org/proceedings/2026/525>.